

学校编码: 10384

学 号: 23020231154138

厦門大學

硕 士 学 位 论 文

(学 术 学 位)

基于隐私计算的多管理域数据平面审计关键技术研究

Research on Key Technologies for Privacy-Preserving
Inter-Domain Data Plane Auditing

方明俊

指导教师姓名: 向乔教授

专 业 名 称: 计算机科学与技术

答 辩 日 期: 2026 年 5 月

学位授予日期: 2026 年 6 月

2026 年 6 月

摘 要

近年来,随着互联网规模的扩张与网络数据平面逻辑的复杂化,由配置错误引发的网络故障频繁发生。如何在故障发生后准确定位根因、进而支撑溯源与责任划分,已成为网络运维中的重要挑战。为此,研究人员提出了多种基于形式化方法的数据平面验证技术,用于事前配置验证与事后故障定位。然而,现有方法在多管理域场景中面临三方面挑战。其一,现有方法依赖集中收集全网配置与策略信息,而在多管理域场景下,各域缺乏互信基础且存在隐私保护需求,集中式验证方案无从适用。其二,事故发生后,特定路由前缀在不同时间点的可达性等数据平面语义结论往往需要多方反复核验,而现有方法缺乏对此进行可信、高效独立核验的支持。其三,路由与配置状态的持续变化会不断使历史语义结果失效,迫使系统全量重新生成,带来显著的计算开销。

针对上述问题,本文提出了多管理域数据平面审计框架 AegisPath, 包含以下三项关键技术:

1. 基于门限秘密分享的多管理域数据平面语义生成技术。各管理域在不公开本域配置与策略细节的前提下参与安全多方计算,能够围绕覆盖对象生成表征数据平面语义结论的数据平面见证。
2. 基于零知识证明的版本化查询审计架构。系统利用 MiMC 承诺与 Merkle 树,将特定时刻的网络转发状态锚定为不可篡改的版本,并通过零知识证明技术使外部审计者能够在不接触真实转发路径的前提下,高效地对数据平面语义结论进行独立核验。
3. 面向频繁状态变化的增量更新技术。系统仅对受影响覆盖对象局部重算其数据平面见证以减少时间开销,并在 Merkle 树上通过只更新对应叶子减少重建 Merkle 树的时间开销。

本文实现了 AegisPath 的原型系统,并从数据平面语义生成、查询验证与增量维护三个方面在至多 200 个自治系统的网络拓扑中进行了实验评估。结果表明,数据平面语义见证生成的时间在最大规模拓扑下达到 10^4 秒量级,保证作为单次生成任务的可行性;而在在线查询阶段中,零知识证明生成时间在所有数据集上均控制在 2.1 秒以内,相较于现有代表性方案 Segull 的时间开销优化至多 190 倍;在局部变化场景下,通过增量更新优化,离线语义生成阶段在时间开销上获得至多 20 倍的加速,版本维护在时间开销上获得至多 47 倍的加速。

关键词: 网络数据平面; 多管理域网络; 安全多方计算; 零知识证明

Abstract

In recent years, with the expansion of the Internet and the increasing complexity of network data-plane logic, network failures caused by configuration errors have occurred frequently. Accurately identifying root causes after failures and further supporting trace-back and responsibility attribution has become an important challenge in network operations. To address this issue, researchers have proposed various formal-method-based data-plane verification techniques for pre-deployment configuration checking and post-failure diagnosis.

However, existing approaches face three major challenges in inter-domain settings. First, these methods rely on centralized collection of network-wide configurations and policy information, whereas in inter-domain environments different domains lack mutual trust and have privacy protection requirements, making centralized verification inapplicable. Second, after an incident, data-plane semantic results such as the reachability of a specific routing prefix at different time points often need to be repeatedly verified by multiple parties, while existing methods do not support such verification in a trustworthy, efficient, and independent manner. Third, the continuous evolution of routing and configuration states constantly invalidates historical semantic results, forcing the system to regenerate them from scratch and thereby incurring significant computational overhead.

To address the above problems, this thesis proposes AegisPath, an inter-domain data-plane auditing framework, which includes the following three key techniques:

1. A threshold-secret-sharing-based technique for inter-domain data-plane semantic generation. Each administrative domain participates in secure multi-party computation without disclosing its local configurations and policy details, and collaboratively generates data-plane witnesses for covered objects to characterize data-plane semantic results.
2. A zero-knowledge-proof-based versioned query auditing architecture. The framework uses MiMC commitments and a Merkle tree to anchor the network forwarding state at a specific time into a tamper-evident version, and further enables external auditors to independently and efficiently verify data-plane semantic results without accessing the actual forwarding paths.
3. An incremental update technique for frequently changing network states. The framework recomputes data-plane witnesses only for affected covered objects to reduce

the time overhead, and updates only the corresponding leaves in the Merkle tree to reduce the time cost of rebuilding the Merkle tree.

We implement a prototype of AegisPath and evaluate it from three aspects, namely data-plane semantic generation, query verification, and incremental maintenance, on network topologies with up to 200 autonomous systems. The results show that the time cost of data-plane witness generation reaches the order of 10^4 seconds on the largest topology, which demonstrates the feasibility of one-shot offline generation. In the online query phase, the zero-knowledge proof generation time is bounded within 2.1 seconds across all datasets, achieving up to a 190-fold reduction in time cost compared with the representative baseline Segull. Under local change scenarios, the incremental update optimization achieves up to $20\times$ speedup in offline semantic generation and up to $47\times$ speedup in version maintenance.

Keywords: Data Plane; Inter-Domain Networks; Secure Multi-Party Computation; Zero-Knowledge Proof

目 录

摘 要	I
Abstract	III
目 录	V
图索引	XIV
表索引	XV
第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	6
1.2.1 数据平面验证	7
1.2.2 多管理域场景的网络验证	9
1.2.3 可验证审计与证据机制	11
1.3 相关理论与技术	13
1.3.1 多管理域数据平面语义与验证对象	13
1.3.2 安全多方计算	14
1.3.3 零知识证明	18
1.3.4 MiMC 承诺	23
1.3.5 Merkle 树	23
1.4 研究内容	24
1.5 论文章节安排	26
1.6 本章小结	28
第 2 章 AegisPath 系统概述	29
2.1 系统架构	29
2.1.1 功能角色划分	30
2.1.2 系统执行阶段	31
2.2 工作流程	32

2.3 安全性分析	35
2.3.1 威胁模型	35
2.3.2 隐私风险分析	37
2.4 本章小结	44
第3章 基于门限秘密共享的数据平面语义生成	45
3.1 问题定义与生成目标	45
3.1.1 多管理域数据平面语义生成问题定义	45
3.1.2 覆盖对象与生成期输入输出	46
3.2 基于秘密共享的多管理域数据平面语义生成	47
3.2.1 异构配置的标量化编码与门限分片	48
3.2.2 生成期全局调度流程	49
3.2.3 逐对象语义推演	52
3.2.4 版本化见证的结构化封装	57
3.3 安全性分析	58
3.4 本章小结	59
第4章 基于零知识证明的版本化数据平面审计	61
4.1 问题定义与生成目标	61
4.2 版本化证据结构	63
4.2.1 语义见证的哈希承诺生成	63
4.2.2 Merkle 聚合与版本根生成	64
4.3 数据平面不变量验证电路	66
4.3.1 电路输入规格与约束框架设计	66
4.3.2 见证合法性与版本一致性约束	67
4.3.3 网络语义不变量的算术化编码	67
4.3.4 审计证明的生成与验证流程	68
4.4 安全性分析	69
4.5 本章小结	71

第 5 章 面向频繁状态变化的增量更新机制	73
5.1 生成期增量更新	74
5.1.1 增量输入接口与驱动方式	74
5.1.2 增量推演算法	75
5.2 证据结构增量维护	80
5.2.1 受影响叶子的承诺更新	81
5.2.2 Merkle 分支更新与新版本根计算	82
5.3 安全性分析	84
5.4 本章小结	85
第 6 章 系统实现与性能评估	87
6.1 系统实现	87
6.2 实验设置	89
6.2.1 测试环境	89
6.2.2 数据集	90
6.2.3 对比工具	91
6.3 实验一：隐私协作计算的性能表现	93
6.4 实验二：零知识证明审计的性能表现	95
6.5 实验三：增量更新机制的性能表现	99
6.6 本章小结	104
第 7 章 总结与展望	107
7.1 总结	107
7.2 未来展望	108
参考文献	111
致 谢	117
在学期间完成的相关学术成果	119

Contents

Abstract-Chinese	I
Abstract-English	III
Contents	V
List of Figures	XIV
List of Tables	XV
Chapter 1 Introduction	1
1.1 Research Background and Significance	1
1.2 Related Work	6
1.2.1 Data-Plane Verification and Scalable Optimization	7
1.2.2 Interdomain Network Verification and Privacy-Preserving Collaboration	9
1.2.3 Verifiable Auditing and Evidence Mechanisms	11
1.3 Related Theories and Technologies	13
1.3.2 Secure Multi-Party Computation	14
1.3.3 Zero Knowledge Proof	18
1.3.4 MiMC Commitment	23
1.3.5 Merkle Tree	23
1.4 Research Content	24
1.5 Arrangement of Content	26
1.6 Summary	28
Chapter 2 AegisPath System Overview	29
2.1 System Architecture	29
2.1.1 Functional Roles	30
2.1.2 Execution Stages	31
2.2 Workflow	32

2.3 Security Analysis	35
2.3.1 Threat Model	35
2.3.2 Privacy Leakage Analysis	37
2.4 Summary	44
Chapter 3 Data-Plane Semantics Generation Based on Threshold Secret Sharing	45
3.1 Problem Formulation and Generation Objective	45
3.1.1 Problem Definition of Interdomain Data-Plane Semantic Generation	45
3.1.2 Coverage Objects and Generation-Phase Inputs/Outputs	46
3.2 Secret-Sharing-Based Interdomain Data-Plane Witness Generation	47
3.2.1 Encoding and Threshold Sharding	48
3.2.2 Global Scheduling Workflow of the Generation Phase	49
3.2.3 Per-Object Semantic Deduction	52
3.2.4 Packaging Versioned Witnesses	57
3.3 Security Analysis	58
3.4 Summary	59
Chapter 4 Version-Bound Data-Plane Auditing with Zero-Knowledge Proofs	61
4.1 Problem Formulation and Generation Objective	61
4.2 Construction of the Versioned Evidence Structure	63
4.2.1 Hash Commitment for Semantic Witnesses	63
4.2.2 Merkle Aggregation and Version Root Generation	64
4.3 Data Plane Invariant Verification Circuit	66
4.3.1 Input Specification and Constraint Framework	66
4.3.2 Witness Legitimacy and Version-Consistency Constraints	67

4.3.3	Arithmetic Encoding of Network-Semantic Invariants	67
4.3.4	Proof Generation and Verification Workflow	68
4.4	Security Analysis	69
4.5	Summary	71
Chapter 5 Incremental Update Mechanism for Frequent State Changes		73
5.1	Incremental Updates in the Generation Phase	74
5.1.1	Incremental Input Interface and Driving	74
5.1.2	Incremental Semantic Deduction Algorithm	75
5.2	Incremental Maintenance of the Evidence Structure	80
5.2.1	Updating Commitments of Affected Leaves	81
5.2.2	Updating Merkle Branches and Computing a New Root	82
5.3	Security Analysis	84
5.4	Summary	85
Chapter 6 Implementation and Evaluation		87
6.1	System Implementation	87
6.2	Experimental Setup	89
6.2.1	Test Environment	89
6.2.2	Dataset	90
6.2.3	Baseline	91
6.3	Experiment I: Performance Evaluation of Privacy-Preserving Collaborative Computing	93
6.4	Experiment II: Performance Overhead of Proof Generation and Verification in Query-Phase Zero-Knowledge Auditing	95

6.5 Experiment III: Performance of the Versioning and Incremental Update Mechanism under Localized Change Scenarios	
99	
6.6 Chapter Summary	 104
Chapter 7 Conclusions and Future Directions	 107
7.1 Conclusion	 107
7.2 Future Work	 108
References	 111
Acknowledgements	 117
Relevant Academic Achievements Completed During the Academic Period	 119

图索引

图 1.1	多管理域中控制面路径失真与外部审计需求示意图	5
图 1.2	现有相关工作与本文方法的对比	7
图 1.3	数据平面验证领域代表研究	8
图 1.4	多管理域场景网络验证领域代表研究	10
图 1.5	Shamir 秘密共享的基本流程	16
图 1.6	ZK-SNARK 的基本实现流程	19
图 1.7	论文组织架构图	27
图 2.1	AegisPath 的系统结构	30
图 2.2	工作流程示例图	33
图 3.1	全局对象槽表中多个覆盖对象并发细化的示意图	50
图 3.2	局部细化函数中规则扫描示例图	55
图 4.1	覆盖对象在 Merkle 树内的排序规则示意图	64
图 4.2	版本 v 下的 Merkle 树结构示例图	66
图 5.1	版本 v 下的网络拓扑与覆盖对象示例图	76
图 5.2	配置变更后的确认覆盖对象是否需要重算	77
图 5.3	配置变更后的版本演化与新版本输出	79
图 5.4	新增前缀对象场景下增量更新流程示例图	80
图 6.1	生成期隐私协作计算的时间开销	93
图 6.2	生成期隐私协作计算的通信量	93
图 6.3	生成期构造 MiMC 承诺和版本根的时间开销	94
图 6.4	查询期证明者生成证明的时间开销	96
图 6.5	查询期时审计者进行验证的时间开销	96
图 6.6	查询期时证明者的电路约束数量	97
图 6.7	Seguall 查询方案的时间开销	97
图 6.8	Seguall 查询时产生的通信发送量	98
图 6.9	AegisPath 和 Seguall 时间开销对比图	99
图 6.10	查询阶段时间开销累积分布图	99
图 6.11	配置变更场景下生成期增量更新与全量更新的时间开销对比	100

图 6.12 配置变更场景下生成期增量更新与全量更新时间加速比累积分布图 .	100
图 6.13 覆盖对象增加场景下生成期增量更新与全量更新的时间开销对比	101
图 6.14 覆盖对象增加场景下生成期增量更新与全量更新时间加速比累积分布图	101
图 6.15 增量更新 Merkle 树和重建 Merkle 树的时间开销对比	102
图 6.16 增量更新 Merkle 树相对于重建 Merkle 树的加速比累积分布图	102
图 6.17 增量更新 Merkle 树和重建 Merkle 树的电路约束数量对比	103
图 6.18 增量更新 Merkle 树和重建 Merkle 树的电路约束数量对比累积分布图 .	104

表索引

表 1.1	可验证审计与证据机制相关工作总结	12
表 3.1	全局对象槽表中的主要状态字段	49
表 5.1	版本更新时叶子层的处理规则	82
表 5.2	Merkle 树局部更新时的处理规则	84
表 6.1	生成期 SMPC 框架选型比较	87
表 6.2	查询期零知识证明框架选型比较	88
表 6.3	实验中公开数据来源及其作用	90
表 6.4	实验中附加数据面规则的注入比例设定	91

第 1 章 绪论

本章首先介绍多管理域网络环境下隐私约束、可扩展性以及运行期变更复核等方面面临的关键问题，并据此说明开展多管理域数据平面审计研究的现实意义 (§1.1)。然后梳理数据平面验证、多管理域协作验证与可验证审计等方向的研究进展，总结现有方法的局限性 (§1.2)。接着介绍本文涉及的基本概念与相关技术，包括多管理域数据平面语义与验证对象、安全多方计算、零知识证明、MiMC 承诺与 Merkle 树 (§1.3)。在此基础上，概述本文的研究内容与技术路线 (§1.4)。最后给出全文的章节安排与组织结构 (§1.5)。

1.1 研究背景与意义

近年来，信息技术的持续迭代与网络基础设施的加速演进，为数字经济与社会运行提供了关键底座。2025 年上半年我国互联网基础资源保有量保持稳定、信息基础设施持续夯实，移动互联网接入流量延续增长态势：截至 2025 年 6 月，5G 基站总数达 454.9 万个，域名总数为 3262 万个，其中国家顶级域名“.CN”数量为 2085 万个，IPv6 地址数量为 68567 块；同时，2025 年上半年移动互联网累计流量达 1867 亿 GB，同比增长 16.4%^[1]。基础资源与接入能力的持续增强，使得云服务、移动互联网、关键公共服务与跨境互联之间的联系不断加深，网络的可用性与安全性也因此变得更加重要。

随着用户规模、接入流量与跨境互联需求的增长，多管理域网络 (Interdomain Network) 作为全球通信基础设施的核心，其稳定性与安全性对公共服务、云平台以及关键业务连续性产生直接影响。多管理域互联由不同自治系统 (Autonomous System, AS) 通过边界网关协议 (Border Gateway Protocol, BGP) 交换路由信息并形成复杂拓扑。这种分布式自治架构在带来灵活互联与规模扩展能力的同时，也使得路由策略、协议交互、设备实现以及运行期变更高度复杂，从而使配置错误与策略失配成为长期存在的系统性风险。行业研究亦表明^[2]，故障与中断事件在宏观层面呈现高频与高影响并存的特征：其 2024 年调查中，超过半数受访者 (54%) 报告其遇到的最近一次显著 (或严重) 中断成本超过 10 万美元，且五分之一超过 100 万美元，并明确指出由 IT 与网络相关问题导致的影响性中断情况的占比在 2024 年上升到 23%)。

为了解决这些问题，研究人员和相关从业者提出了网络验证（Network Verification）这一重要研究方向。网络验证的基本目标，是检验报文转发行为是否与预期策略保持一致，从而帮助运维者在故障扩散前发现黑洞、环路、错误绕行等问题。围绕这一目标，现有研究大体可分为控制平面验证与数据平面验证两类：前者主要关注路由宣告、路径选择与策略传播结果是否满足预期；后者则进一步关注报文在真实转发过程中是否能够到达目标、是否经过指定中间节点，以及是否存在环路、黑洞或边界处被丢弃等现象，即直接面向真实转发语义下的可达性、必经点与无环性等数据平面不变量。

在单一管理域内部，这类验证之所以能够有效开展，一个重要前提在于验证者通常可以集中获取网络设备的明文转发表与策略配置。在这一前提下，已有数据平面验证技术能够在集中视图下计算真实转发结果，并据此判断相关性质是否成立。然而，这一前提在多管理域场景中通常难以满足。多管理域网络由多个自治系统独立运营，各 AS 往往将其内部转发状态、边界策略与运营规则视为敏感商业信息，不愿向外部完整披露。已有研究也指出，自治系统通常不会公开其内部转发信息，而 AS 之间的关系与策略本身也具有敏感性^[3-8]。这意味着，单管理域中集中获取明文状态后直接验证数据面的思路，在多管理域网络中往往难以直接成立。

正因如此，现有面向多管理域场景的验证框架通常都需要在隐私保护约束下工作，即不能直接依赖各自治系统公开其内部转发表项与边界策略，而必须在有限可观测信息基础上开展验证。在这一条件下，现有方法所能实际利用的信息主要来自控制平面，因此其验证对象也往往退化为控制平面上的路由选择结果、策略传播结果与公开可观测路径^[5-7]。研究者和运维人员也确实可以通过 BGP 路由采集器^[9]、Cogent Looking Glass^[10] 以及 RouteViews^[11] 等公开观测渠道，获得某一前缀在控制平面上的宣告信息与路径属性。

然而，在由边界过滤、黑洞处置、策略路由或其他数据面配置所引发的这一类事故中，控制平面证据并不足以唯一决定数据面中的真实交付结果。其根本原因在于，控制平面上的可见宣告与 AS-PATH 只反映了路由传播和选路结果，而数据面中的最终转发行为还会受到边界执行机制与本地运营策略的进一步约束，例如前缀级访问控制列表、远程触发黑洞、策略路由以及边界过滤等。因此，即便某一前缀在控制面上仍然可见，报文在真实转发过程中仍可能被丢弃、重定向，或沿着与控制面推断不一致的路径转发。

这种控制面可见而数据面不可达的偏差是真实网络中持续存在的工程问题。

Zeng 等人对某大型数据中心连续 14 个月的运维工单统计表明，与数据平面转发相关的问题在该时间窗内多次出现，其中转发表项缺失工单 35 起、转发环路工单 11 起、转发黑洞工单 11 起，平均每月约报告 4 起相关问题^[12]。在多管理域网络中，这种偏差还会因不同管理域对边界策略与安全机制的异构执行而进一步放大。以 ROA 相关事件为例，由于不同网络对 ROV 的部署程度和执行策略并不一致，同一前缀即使仍可在控制面上被观测到宣告与路径信息，也可能在部分网络中因边界侧过滤而无法被实际转发^[13]。这表明，在这一类由数据面配置与边界执行所主导的场景中，控制平面观测结果既不能充分解释异常来源，也不能直接验证真实的数据面行为。

由此可以看到，当数据面配置偏差或边界执行失效引发转发事故时，问题的核心就不再只是控制面是否存在某条宣告路径，而是各域当前数据面配置实际生效之后，相关流量是否能够按预期到达、经过或避免某些路径节点。可达性、必经点与无环性等性质所对应的，并不是控制面宣告语义，而是由转发表项、边界过滤、黑洞处置与策略改写等数据面配置共同决定的实际转发语义。

这一点在数据平面配置引发的事故处理中尤为突出。现实中的多管理域故障处置往往并非一次性完成，而是伴随着多轮修复、策略调整与责任界定。一旦异常由边界过滤、黑洞处置、路由拒绝或其他数据面策略触发，后续真正困难的往往不再只是发现故障，而是如何围绕某一时刻的真实数据面行为进行事后核验，并据此判断修复是否生效、责任应如何划分、以及外部各方是否能够对相关结论进行独立复核。在这一过程中，真正需要验证结果的利益相关方并不只包括触发过滤或实施策略变更的一侧网络运营方，还包括受影响前缀的拥有者、与事件处置相关的其他自治系统、国家级应急响应机构（例如 CERT/CSIRT）、监管部门以及需要判断业务连续性与责任边界的外部审计者。于是，不同参与方往往会在不同时间反复追问：某一时刻某个前缀是否真实可达，某条路径是否经过指定中间域，某次策略调整是否确实改变了数据面行为，以及后续修复是否真正消除了黑洞、绕行或环路。

这意味着，多管理域场景中的问题已经不再只是一次性的验证计算，而进一步演化为围绕同一对象在不同时刻持续发生的核验需求。对于这一类事后核验任务，至少存在两方面挑战：其一，真实数据面语义依赖各管理域内部的转发表项、边界过滤规则与策略配置，而这些信息通常被运营商视为敏感隐私，难以集中收集进行核验；其二，事后核验并非一次性过程，而是围绕特定前缀、路径性质与责

任边界反复发生的持续性需求，若每次查询都重新组织一次完整的多管理域隐私协作计算，则会带来较高的协同开销与响应延迟。

俄罗斯封禁 Telegram 事件可以直观体现这一点。围绕 Telegram 在俄罗斯境内被限制访问及其后续解除过程，相关公开报道表明，监管侧曾持续推动边界处的前缀级阻断与过滤，并一度通过大规模封禁云服务相关地址段压制业务连通性^[14-15]。在这类事件中，外部各方真正关心的并不只是某一时刻控制面上是否仍能观测到相关前缀宣告，而是某一阶段特定前缀是否真实可达、限制措施是否已经生效、解除之后是否确实恢复，以及这些结论在事后能否再次被独立核验。因此，事件本身体现出的不是一次简单的路径观测需求，而是一个随时间持续演化的数据面核验需求。

由网络初创资源中心（NSRC）运营的 Route Views 项目在其 2025 年更新报告中，也从公共控制面观测平台的实际使用情况反映了这种持续性需求。该报告指出，其路由收集节点的传统命令行访问方式每天都会被成千上万的自动化脚本持续访问，同时成千上万的网络运营者与研究人员长期依赖 Route Views 数据开展分析与判断^[16]。这表明，围绕特定前缀与路径的反复查询并非偶发现象，而是现实网络运维中的持续性需求。与此相一致，Route Views 官方接口文档进一步表明，该接口专门面向需要经常访问当前 Route Views 数据的网络运营者与研究人员，并将常见查询作为主要使用场景；相应地，未认证访问被限制为每秒 1 次接口调用，认证访问被限制为每秒 10 次接口调用^[17]。这些事实共同说明，在多管理域故障分析、策略变更跟踪与事后复盘过程中，围绕同一前缀或路径进行持续观测、反复询证和多轮核对，本身就具有明确且持续存在的现实规模。

为了直观展示控制面与数据面语义的偏差和外部核验需求，本文给出了一个典型的多管理域转发场景。如图 1.1 所示，设公开路由观测系统在某一时刻观测到：前缀 202.112.0.0/16 的可见控制面路径为 $AS\ S \rightarrow AS\ A \rightarrow AS\ B \rightarrow AS\ C$ 。基于这一公开可观测结果，外部机构只能获得该前缀在控制面上似乎经由 AS A、AS B 到达 AS C 这一表象信息。例如，APNIC 希望查询该前缀对应的真实数据面路径是否经过某个特定自治系统，而 Cloudflare 则可能更关注该前缀在当前状态下是否仍能到达目标自治系统。

然而，公开控制面路径并不等于真实数据面转发结果。图中进一步给出一个典型场景：虽然外部观测到的控制面路径包含 AS B，但 AS B 的本地数据面配置中存在针对前缀 202.112.0.0/16 的拒绝规则（deny 202.112.0.0/16）。这意味着，实

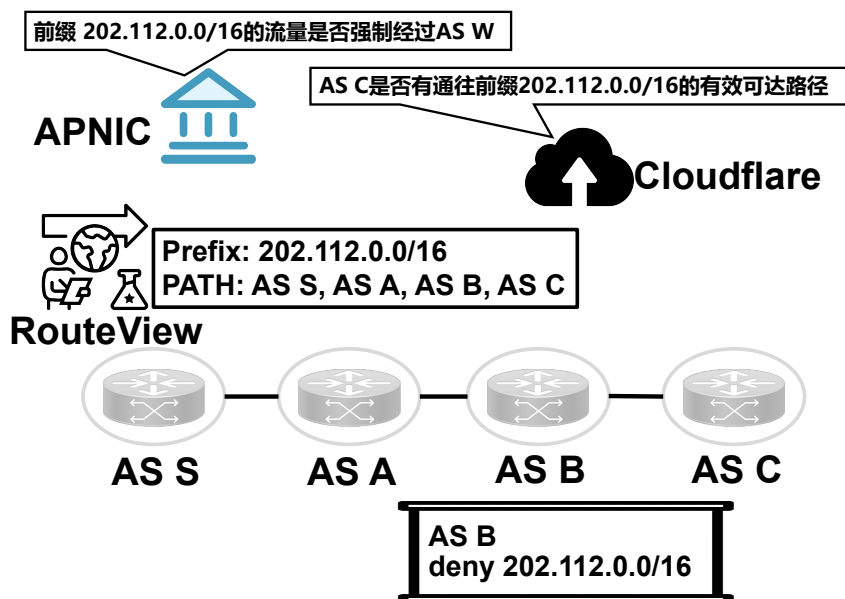


图 1.1 多管理域中控制面路径失真与外部审计需求示意图

际转发过程中流量在到达 AS B 后可能被边界策略直接丢弃，从而导致控制面上看似存在到 AS C 的路径，但数据面上实际上不可达。因此，对于同一个前缀，公开观测系统提供的是路径线索，而真正决定可达性、必经点与无环性等性质的，仍然是各管理域内部的数据面配置与策略。

该示例说明了本文所关注的问题场景：当多管理域事故由边界过滤、黑洞处置、路由拒绝或其他数据平面配置触发时，相关方往往需要围绕特定前缀、特定时刻与特定性质，对真实数据面行为进行事后复核。一方面，外部相关方确实可能需要判断某一前缀在某一时刻是否真实可达、某条路径是否经过指定中间域，以及某次策略调整之后相关异常是否已经消除；另一方面，决定这些结论的转发表项、边界过滤与策略规则又分散在多个自治系统内部，既无法仅凭公开控制面观测准确推出，也难以由各域以明文形式集中公开。因此，本文关注的并不是一般意义上的一次性验证计算，而是在隐私约束下如何为这一类事故后的复核过程提供可被后续引用和核验的数据面证据。

围绕这一问题，一种直接思路是公开各自治系统的转发表快照与边界规则，并以此作为复核依据；然而，这类数据通常具有明显的商业敏感性^[3-4, 6]，难以在多管理域环境中被各方接受。另一种思路是完全依赖安全多方计算，由各 AS 在不泄露本地输入的前提下联合计算数据面语义，并在每次需要重新核验某项结论时再次运行协议。该方案虽然能够保护输入隐私，但会使复核过程持续依赖在线多方协作，从而使后续查询成本随复查次数不断累积。也就是说，前一种方案难以满

足多管理域环境下的隐私边界要求，后一种方案则难以满足事故处置过程中反复核验的实际开销要求。

为了解决上述挑战，本文提出以验证为核心手段、以审计为应用目标的设计思路。首先，在不公开各自治系统明文配置的前提下，协同生成由真实数据面状态决定的转发语义结果；其次，将该结果与特定版本的系统状态进行密码学绑定，从而支持外部相关方对同一版本下的数据平面不变量进行独立核验，有效规避了重复执行高开销在线协同计算的必要性。在该框架中，数据平面不变量作为最终审计对象，而验证过程则负责为其提取可信语义并生成可公开核验的证据基础，并进一步将这些结果组织为可公开核验、可持续引用的审计证据。

1.2 国内外研究现状

随着网络规模持续扩大、设备功能不断增强以及多管理域协作需求日益频繁，网络验证面临的挑战已不再局限于单一管理域内的正确性检查。在多管理域场景下，一方面，验证目标要求尽量贴近真实数据面行为；另一方面，各自治系统又普遍不愿公开本域配置、策略与边界规则，这使得传统依赖集中收集语义输入的验证方式难以直接适用。与此同时，多管理域治理、故障复盘与持续核验还进一步提出了外部可复验与证据留存的要求。

围绕这些问题，现有研究大致可以归纳为三类。图 1.2 对上述三类工作的基本思路及其主要局限作了概括。本节将在此基础上分别介绍数据平面验证、多管理域场景下的网络验证，以及可验证审计与证据链机制的相关研究现状，并对现有工作的主要不足作进一步总结。第一类是数据平面验证方法 (§1.2.1)，其优势在于能够直接围绕最终转发行为建立形式化模型，并对可达性、必经点、无环性等网络不变量进行系统化检查；但这类方法通常默认验证方能够获得或集中汇聚数据面语义输入，因此在多管理域场景下容易受到隐私约束与自治边界的限制。第二类是多管理域场景下的网络验证方法 (§1.2.2)，其优势在于已经开始考虑自治域之间的信息隔离、协作验证以及隐私保护计算，说明多管理域条件下的联合验证具有现实可行性；但现有方案往往仍主要围绕配置验证、控制面导出的转发抽象或较粗粒度的转发图展开，对于边界过滤、黑洞处置、策略改写以及它们与最长前缀匹配共同作用下形成的对象级有效语义，仍缺少统一、可组合的表达接口。第三类是可验证审计与证据链机制 (§1.2.3)，其优势在于能够通过透明日志、可验证目录、状态承诺与证明机制，为外部观察者提供可复验、可追溯且难以篡改

的证据表示；但这类工作通常更关注日志、目录、路由状态或宣告-转发一致性等对象的可验证记录，而不直接生成由真实数据面配置与边界策略共同决定的对象级语义见证。

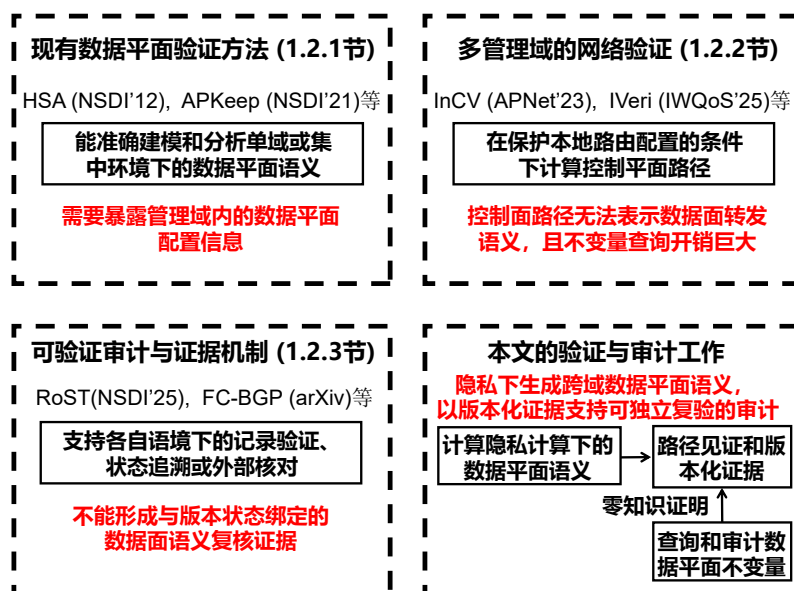


图 1.2 现有相关工作与本文方法的对比

1.2.1 数据平面验证

数据平面验证（Data Plane Verification, DPV）直接以网络设备的最终转发行为为对象^[18]，检查可达性（reachability）、隔离性/安全分段（isolation）、无环性（loop-freeness）、黑洞（blackhole）以及必经点（waypointing）等网络不变量。与 ping、traceroute 等探针式方法相比，DPV 的核心优势在于：能够以形式化方式刻画报文集合的转发语义，并输出系统性的验证结论与可复现的反例路径。图 1.3 列出了数据平面验证领域的代表性工作，以下按时间顺序对这些代表性方法及其技术演进进行介绍。

早期工作首先回答了如何将数据平面语义转化为可判定问题。Anteater^[19] 将数据平面路径行为编码为布尔可满足性问题，并利用 SAT 求解器检查环路、黑洞以及转发一致性问题，其代表性意义在于较早将网络配置错误检测系统化地转化为逻辑求解问题，从而奠定了后续形式化数据平面验证的基础。随后，Header Space Analysis（HSA）^[20] 提出以头空间与转移函数统一建模网络设备的转发、过滤与改写行为，使可达性失败、隔离性破坏及环路等问题都能够在统一框架下表

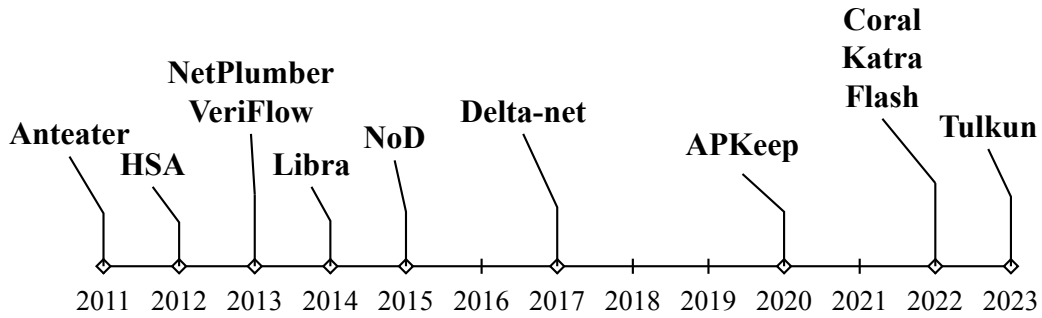


图 1.3 数据平面验证领域代表研究

达与检查。围绕 HSA 的在线化与工程化需求，NetPlumber^[21] 将网络属性验证推进到增量更新场景：它在线监听状态变化并局部更新依赖关系，使验证可以随配置变化持续执行，而不必每次全量重算。这一阶段的代表工作共同确立了一个基本方向，即以符号化方式表示报文集合及其在网络中的传播与变换，从而对数据平面不变量进行自动化分析。

在此基础上，研究者开始进一步关注表达能力与规格语言的扩展。NoD^[22] 提出以 Datalog 同时作为建模语言与规格语言，用以表达更高层的可达性策略、信念检查以及动态网络中的验证需求。与仅支持固定头格式和固定转发模型的系统相比，NoD 强调更一般的建模能力，能够描述网络故障、头部变化以及策略级不变量，并借助 Z3 支持大规模头空间上的求解。这类工作的重要意义在于，它将数据平面验证从固定模型上的快速检查进一步扩展到可表达动态网络与抽象策略信念的统一验证框架。

随着网络规模增长与配置更新频繁化，数据平面验证的研究重点逐渐从能否表达语义转向能否在可接受时间内完成验证与定位。一条代表性路径是利用并行化和任务分解提升规模能力，例如 Libra^[12] 采用分而治之的方式对大规模转发表进行一致性检查，将单点验证压力拆分到并行任务上，从而提升大规模网络中的验证吞吐。另一条路径则进一步强调实时性与增量更新能力。VeriFlow^[23] 在控制器更新路径上实时拦截并检查网络级不变量，强调验证系统必须跟上控制面更新速度。在此基础上，Delta-net^[24] 针对实时数据平面验证中的性能瓶颈，提出基于 atoms 的增量式图变换方法，通过维护覆盖全网流量的单一边标记图而非重复构造多张重叠转发图，显著降低了规则插入与删除时的检查开销，并使更大范围的 what-if 查询成为可能。相较于前述工作，Delta-net 的重点已不只是增量验证，而是进一步回答如何在高频规则变动和大规模前缀条件下保持准实时检查。

与此同时，随着网络设备功能演进，数据平面验证的研究对象也从仅转发与

过滤扩展到更复杂的数据面语义。例如, APKeep^[25] 提出端口-谓词映射 (Port-Predicate Map) 模型, 将转发、过滤、改写等功能统一抽象并支持高效增量更新, 重点面向真实网络中复杂更新场景下的实时验证。Katra^[26] 则面向多层封装网络提出新的验证机制, 用于处理封装、解封装以及跨层头部依赖带来的状态爆炸问题。这类工作表明, 数据平面验证的难点已经不再局限于基本的 reachability 检查, 而是进一步扩展到复杂头部语义、跨层行为与高频动态更新的组合场景。

近年的另一条重要路线则开始直接挑战集中式验证架构的边界。Flash^[27] 面向大规模网络中的更新风暴与长尾到达场景, 强调在高并发更新条件下保持一致、快速的数据平面验证, 其关注点已经从单次检查延伸到持续更新过程中的验证时延与一致性。Coral^[28] 所代表的工作提出将验证能力下沉到网络设备侧, 通过分布式、设备侧的数据平面检查来缓解集中式验证器在规模、路径长度和管理网络依赖方面的瓶颈。在此基础上, Tulkun^[29] 进一步系统化了这一方向: 它将数据平面验证转化为有向无环图上的计数问题, 通过设备侧协作显著降低中心验证器的性能压力, 并在真实大型数据集上展示出优于集中式方案的吞吐与增量性能。

总体而言, 数据平面验证领域已经形成了较完整的发展序列: 从 Anteater、HSA、NetPlumber 所代表的符号化建模与可判定分析, 到 NoD 所体现的统一建模与规格表达, 再到 VeriFlow、Libra、Delta-net、APKeep、Katra、Flash 所体现的实时性、增量更新、复杂语义与大规模优化, 最终进一步发展到 Coral 和 Tulkun 所代表的分布式、设备侧验证架构。这些工作共同推动了 DPV 从可表达、可验证走向可扩展、可在线、可部署。但它们大多仍默认验证方能够获得或集中汇聚数据面语义输入, 例如转发表、过滤规则、改写规则及相关策略。在多管理域环境中, 这一前提往往不成立。各自治系统通常不会向第三方明文开放本域配置与策略, 因此, 经典数据平面验证的难点并不只在如何验证, 还在于如何在隐私约束下联合生成可验证的数据面语义输入。

1.2.2 多管理域场景的网络验证

多管理域网络验证的核心矛盾在于: 验证目标要求尽量贴近真实网络行为, 而真实行为又由多个自治系统的本域配置、策略与边界规则共同决定; 但在工程实践中, 各管理域通常将这类信息视为敏感数据, 不愿明文披露给其他自治系统或外部验证者。因此, 多管理域验证不仅面临语义表达与规模问题, 还必须处理自治边界与隐私保护问题。图 1.4 列出了多管理域场景网络验证领域的代表性工作, 以

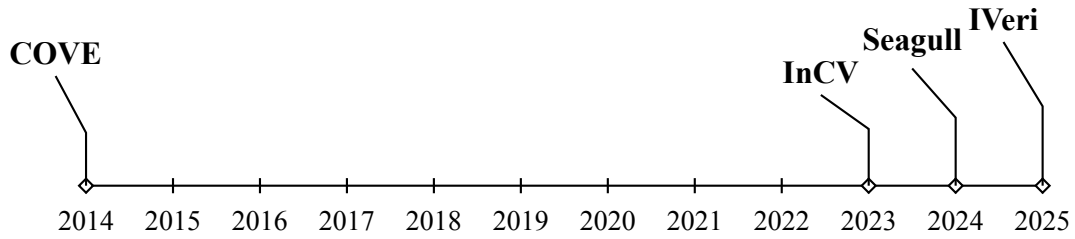


图 1.4 多管理域场景网络验证领域代表研究

下按时间顺序对这些工作及其技术演进进行介绍。

较早的代表性工作 COVE^[30] 将多管理域验证建立在递归协作检查的思路：各管理域部署本地验证器，当需要回答多管理域可达性时，由相关域之间通过递归询问共同构造正向与反向可达结构，并据此判断域间连通性。这类工作的意义在于，它较早说明了多管理域网络验证并不一定依赖于全局明文配置集中收集，也可以通过分布式、域间协作的方式完成。然而，COVE 主要围绕域间可达性这一较窄属性展开，更强调各域如何配合给出回答；对于复杂数据面语义、边界策略作用以及频繁变更下的持续复核问题，其覆盖范围仍较为有限。

在多管理域验证真正进入隐私保护语境之后，InCV^[7] 将多管理域配置难以集中收集直接作为问题出发点，提出在隐私保护条件下完成多管理域配置验证的方案。其核心价值在于：明确把多管理域网络中的隐私约束前置，说明多管理域验证系统首先必须回答在不看明文配置的情况下是否仍能完成联合判断这一基础问题。相较于早期依赖递归交互的协作式方法，InCV 更强调通过安全多方计算在不泄露本域敏感配置的前提下完成验证。与此同时，这类工作验证的重点仍然主要是多管理域配置或控制逻辑的一致性与正确性，即更接近多管理域配置验证意义上的检查，而不是进一步围绕对象级数据面语义结果建立后续审计接口。

Seagull^[6] 则进一步把隐私保护的多管理域验证推进到更接近转发结果的层面。该工作引入 MPC，在不暴露敏感路由信息的前提下验证配置正确性与若干网络性质，其建模核心是以前缀到下一跳自治系统的映射为基础构造转发图，并在该图上检查可达性、环路与必经点等性质。相较于仅停留在配置规则层面的验证，Seagull 已经更接近由转发表语义导出的多管理域转发性质检查，这使其在验证对象上较早跨出了只看配置、不看转发结果的界限。不过，Seagull 的语义接口仍主要围绕从前缀指向下一跳的转发图展开，对于边界处的前缀级过滤、黑洞处置、策略改写以及它们与最长前缀匹配共同作用下形成的对象级有效语义，仍缺少统一、可组合的表达接口。因此，它虽然比单纯的多管理域配置验证更接近数据面，但

与对象级有效数据面语义见证之间仍存在明显距离。

IVeri^[5] 则进一步关注多管理域隐私验证的可扩展性问题。该工作通过数据聚合等方式减少实际参与隐私计算的节点数量，并结合路由代数相关抽象与推理手段降低高代价过程带来的计算与交互开销，从而提高多管理域验证在更大规模网络下的可行性与吞吐能力。从研究定位上看，IVeri 回答的是多管理域隐私验证在规模上能否进一步推进的问题，因此其重点在于隐私协作计算的扩展性与工程可行性，而不是对对象级数据面语义进行更细致的表达。换言之，IVeri 进一步证明了隐私保护的多管理域验证不仅必要，而且可以朝更大规模场景扩展，但其核心贡献仍主要落在隐私下的联合验证计算，而非生成可版本化、可独立复验的对象级语义见证。

总体来看，多管理域场景下的网络验证已经从早期的递归协作可达性检查，发展到在隐私保护条件下完成配置验证与转发图性质验证，并进一步开始考虑大规模场景下的协作计算效率问题。这些工作逐步回答了多管理域验证是否可做，隐私约束下能否协作完成验证以及是否能够支撑更大规模网络的问题。然而，现有方案在验证对象与后续审计接口方面仍存在明显共性限制：其一，验证对象往往偏向多管理域配置、控制面导出的转发抽象，或以前缀到下一跳为核心的图模型；其二，对边界过滤、黑洞、策略改写以及它们与最长前缀匹配共同作用下形成的对象级有效语义，缺少统一且可组合的表达方式；其三，相关工作通常更关注如何完成一次隐私保护下的联合验证，而较少进一步回答如何把验证结果组织成可版本化、可公开复验、可重复查询的审计证据这一问题。

1.2.3 可验证审计与证据机制

随着多管理域网络规模扩大、运营主体增多以及配置与策略变更频率不断提高，网络故障处置、合规审查与责任追溯已不再局限于单次诊断，而往往伴随着事后复盘、重复询证与跨主体核验等需求。在这类场景中，相关参与方不仅关心某一时刻的验证结论本身，还关心该结论是否能够在事后被第三方独立复验，相关状态是否会在后续变更中被改口或回滚，以及是否能够形成可追溯、可校验的证据链。围绕这些问题，已有研究主要从透明日志、可验证目录、路由状态透明性、宣告-转发一致性认证以及公开观测支持下的外部核对等方向展开。

一类代表性工作来自透明日志与可验证目录机制。证书透明（Certificate Transparency, CT）^[31] 提出以公开、追加式日志记录证书存在性，并利用 Merkle 树为每

表 1.1 可验证审计与证据机制相关工作总结

工作	证明或观测对象	核心机制	优点	不足
RFC 9162	证书日志记录	Merkle 日志证明	可公开审计、支持版本一致性校验	不支持网络数据面语义
CONIKS	键绑定目录	目录根与成员证明	隐藏内部条目、支持一致性检查	不支持数据平面语义性质的证明
RoST	路由撤销状态	状态透明库	支持撤销状态公开追踪	不覆盖对象级数据面语义
FC-BGP	宣告-转发一致性	转发承诺	约束宣告与转发关系	不提供隐私保护下的对象级复核
BGPStream / Hubble / Trinocular	控制面归档或主动探测所见的异常症状	公开控制面分析与主动探测	支持异常发现与外部辅助核对	不提供版本化数据面语义结果，不能形成语义级可验证证据

条记录提供成员证明、为不同时间点的日志视图提供一致性证明，使外部观察者不仅能够检查某一证书是否已被纳入公开日志，还能够验证日志是否保持追加式增长而未发生回滚、分叉或静默删除^[32]。RFC 9162 在此基础上进一步标准化了证书透明第二版的日志格式、证明接口与一致性校验方式。CONIKS^[33] 面向端到端加密场景中的密钥透明性问题，提出一种可验证目录机制：服务端维护用户标识到公钥绑定关系的目录结构，并仅公开目录根及相应证明；外部用户无需获知全部内部条目，即可验证某个绑定关系是否存在、目录视图是否一致，以及服务端是否对不同用户返回了相互矛盾的结果。这类工作的重点在于对记录、目录和版本关系提供可公开校验的表示，但其证明对象主要是证书记录或键绑定映射，并不直接对应网络数据面语义。

另一类工作开始直接面向路由状态或宣告-转发一致性。FC-BGP^[34] 提出转发承诺机制，将自治系统的转发行为纳入可验证框架，使外部能够检查某一路由宣告是否与其声称的数据面转发行为保持一致，从而把宣告与转发之间的一致性关系转化为可校验对象。RoST^[35] 则聚焦于路由撤销状态，针对多管理域环境中可能出现的撤销抑制、状态不透明与历史追溯困难等问题，引入路由状态透明性机制，使自治系统能够检查某条路由是否确已被撤销，以及相关状态变化是否已被正确公开。相较于透明日志与目录机制，这类工作更接近网络运行状态本身，但其证明对象仍主要是路由状态或宣告-转发关系，而不是由真实数据面配置、边界过滤与策略共同决定的对象级有效语义结果。

除上述显式证明与状态公开机制之外，多管理域网络中还存在一类与公开观测相关的工作。BGPStream^[36] 统一支持对 RouteViews 与 RIPE RIS 的历史和实时 BGP 观测进行分析，其关注重点在于控制面公开归档数据的处理与异常分析。Hubble^[37] 通过互联网级主动探测发现可达性问题，并报告其能够发现全面探测本应发现问题中的 85%；Trinocular^[38] 则面向互联网级停机检测，在 11 分钟一轮的探测配置下，对持续一轮及以上的停机给出 100% 的检测率。此类工作的共同点在于，它们反映的是控制面归档或主动探测所见的外部异常症状，而不是与特定版本状态严格绑定的数据面语义结果。因此，它们是作为异常发现或外部核对的辅助依据，缺乏针对特定版本数据面语义的可追溯性保证。

相关工作的证明或观测对象、核心机制、优点与不足在表 1.1 中进行了归纳。总体来看，上述工作主要围绕四类对象展开：其一是日志记录或目录映射，其二是路由状态变化，其三是宣告与转发之间的一致性关系，其四是控制面归档或主动探测所见的异常症状。相应地，它们的核心能力主要体现在追加式记录、版本一致性校验、外部可验证查询、状态公开追踪以及异常发现与外部辅助核对。然而，这类机制通常并不直接生成由真实数据面配置、边界过滤与策略共同决定的对象级有效语义见证，也不直接面向可达性、必经点与无环性等对象级数据面不变量的版本化复核接口。换言之，这些研究重点解决的是记录、状态、宣告或外部症状如何被可信公开、观测与校验，而不是在隐私约束下如何计算并审计对象级数据面语义结果。

1.3 相关理论与技术

1.3.1 多管理域数据平面语义与验证对象

多管理域网络由多个自治系统（Autonomous System, AS）互联构成，域间路由通常通过边界网关协议（Border Gateway Protocol, BGP）交换可达性信息与路径属性，从而形成可观测的路由宣告与 AS 路径^[39]。在工程实践中，外部观察者可以借助公共路由收集与观测基础设施获取一定程度的控制面视图，例如 RouteViews 组织的 BGP 收集器与归档^[11]、RIPE RIS 的路由信息服务^[9]，以及运营方提供的 Looking Glass 查询接口^[10]。这些机制使得控制面宣告的路径在一定程度上可被观测与复现，但它们并不直接给出数据平面真实转发语义，也难以刻画边界过滤、黑洞处置、策略路由等数据面规则对可达性与路径性质的实际影响。

数据平面验证的基本任务是：首先刻画待验证的报文集合，然后检查该报文集合在实际转发过程中是否满足可达性、必经点、无环性等网络不变量。已有研究通常在报文头字段空间上定义验证对象，并在此基础上对转发、过滤与改写行为进行形式化建模与分析^[20-21, 25, 28-29]。

在这一建模范式下，验证对象的粒度可根据具体场景灵活选择。面向多管理域验证与审计场景，本文并不直接以原始报文头空间作为外部接口，而是将验证对象刻画为与目的前缀相关的覆盖对象。在初始输入阶段，该对象实例化为源自控制面路由宣告的二元组 (p, s) 。该二元组表征了从源自治系统 s 出发、面向特定目的前缀 p 的可达性覆盖范围，构成了语义计算的初始粒度。然而，受最长前缀匹配、边界过滤及策略改写等数据面转发逻辑的影响，初始二元组往往具有语义复合性，并非不可再分的原子单元。为此，本文在生成期对 (p, s) 对应的流量空间进行解构，将切割后形成的语义一致子空间重新索引为细化二元组 $(p^{(k)}, s)$ 。基于此，本文中的二元组对象蕴含两层语境：一是作为输入覆盖单位的原始对象；二是经由生成期细化后、与稳定数据面语义相对应的细化对象。若无特别说明，后续所称的二元组对象均指后者，即进入承诺与查询阶段的最终验证单元。

采用上述对象抽象的原因在于：一方面，多管理域场景中的观测、运维与审计活动通常以前缀及其域间转发行为为基本对象。无论是域间连通性分析、关键路径约束核验，还是运行期的故障复盘与策略核查，目的前缀都比更细粒度的报文头字段组合更直接、更稳定，也更符合多管理域治理中的实际使用方式。另一方面，仅以目的前缀刻画对象，仍不足以区分不同来源方向上的转发差异；而仅以原始前缀二元组 (p, s) 作为最终语义单位，又难以表达更具体规则、边界过滤与策略约束导致的内部语义分化。因此，本文采用原始二元组输入、生成期细化、细化后二元组输出的对象组织方式，以统一刻画从特定源自治系统出发、面向某一目的前缀，并在数据面规则约束下细化后的验证语义。

1.3.2 安全多方计算

安全多方计算 (Secure Multi-Party Computation, SMPC) 旨在解决如下问题：在不存在可信第三方的条件下，多个参与方如何在不公开各自私有输入的前提下，共同完成某个目标函数的计算^[40-42]。形式化地，设参与方为 $P_1, \dots, P_n$ ，其私有输入分别为 $x_1, \dots, x_n$ ，希望共同计算目标函数 f ，得到输出 $y = f(x_1, \dots, x_n)$ 。SMPC 的基本要求是：各方在得到输出 y 的同时，不额外泄露其他参与方的私有输入信

息，除非这些信息本身已经可以由输出推知。在本文场景中，各自治系统的本地数据面配置、边界过滤规则与策略信息均可视为私有输入，而多管理域数据面语义见证的生成过程则可建模为联合计算函数 $f(\cdot)$ 。因此，SMPC 为多管理域环境下无法集中收集明文配置、但仍需联合生成全局数据面语义这一问题提供了可行的技术路径。

现有安全多方计算可以通过多种底层技术实现。第一类是**混淆电路**（Garbled Circuits, GC）路线^[40, 43]。该方案由 Yao 提出，其核心是将参与方共同关注的计算逻辑（目标函数）表示为底层的布尔逻辑门电路，并通过对电路中每一个逻辑门执行真值表加密与置乱的混淆处理（Garbling Process），使得参与方能在不获知输入真实语义的前提下协同执行该逻辑并获得结果标签。第二类是**基于秘密共享**的路线，其基本思想是先将明文输入拆分为多个份额，再在份额域上完成联合运算；这一路线通常更适合多参与方场景下的大规模算术计算，也是本文采用的实现方式。第三类是**基于同态加密**的路线^[44-45]，其核心是在密文域中直接执行部分或全部运算，再由持有解密权的一方恢复结果；这类方案在特定聚合类任务中具有优势，但在多参与方交互式计算场景下往往需要额外机制配合。除此之外，实际系统还常结合不经意传输（OT）、预处理相关随机性或混合协议等技术，形成面向不同威胁模型与性能目标的实现方案。综合考虑多参与方扩展性、与算术计算的契合程度以及本文门限协作模型的一致性，本文采用基于**门限秘密共享**的 SMPC 实现路径。

本文采用的基础共享机制是 **Shamir 秘密共享**（Shamir's Secret Sharing）^[46]。其核心思想是：将一个秘密编码为有限域上的多项式常数项，再把该多项式在不同点上的取值作为不同参与方持有的份额。设秘密为 $S \in \mathbb{F}_p$ ，门限参数为 t ，则可随机选取一个次数为 $t-1$ 的多项式：

$$f(x) = S + a_1x + \cdots + a_{t-1}x^{t-1}, \quad (1-1)$$

其中 $a_1, \dots, a_{t-1}$ 为从有限域 $\mathbb{F}_p$ 中随机选取的系数。随后，向第 i 个参与方分发份额 $f(i)$ 。当收集到不少于 t 份有效份额时，可通过拉格朗日插值恢复 $f(0) = S$ ；而当观察到的份额少于 t 份时，攻击者无法唯一确定多项式的常数项，从而无法获得关于秘密的有效信息。

图 1.5 给出了 Shamir 门限秘密共享的基本流程示意。对于秘密 S ，系统首先构造一个以 S 为常数项的随机多项式 $F(X)$ ，并在**秘密分发阶段**通过该多项式

在不同点上求值，生成多个份额并分发给不同参与方；图中各参与方持有的 (i, S_i) 对应于第 i 个点上的份额。随后，在**秘密重构阶段**，当至少有 t 个参与方提交各自份额时，系统即可基于这些点值执行拉格朗日插值，恢复多项式常数项，从而重构出原始秘密 S 。

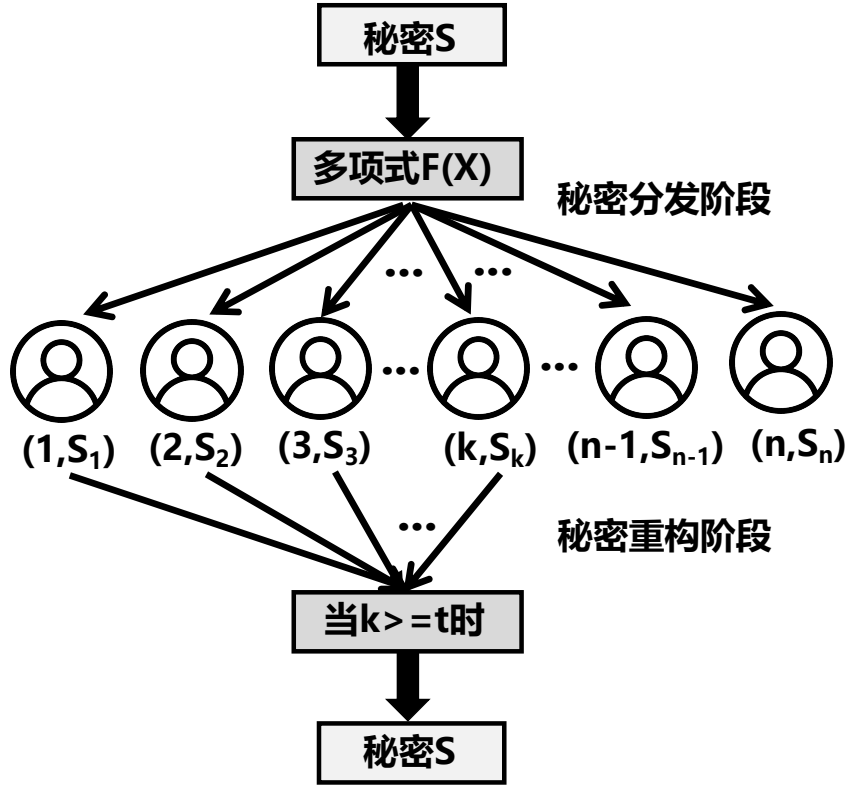


图 1.5 Shamir 秘密共享的基本流程

从实现角度看，本文后续生成期需要的算术运算有加法、乘法、比较、相等判断、前缀匹配中的按位逻辑运算等操作，下文将详细介绍本文如何在秘密共享框架下构造并实现这些关键协议。

基于门限秘密共享的一个重要优势在于：它天然支持加法的线性保持，并在结合交互协议后支持乘法运算。设两个秘密 S_1, S_2 分别由多项式 $f_1(x), f_2(x)$ 编码，各参与方持有的份额分别为 $f_1(i)$ 和 $f_2(i)$ 。对于加法，参与方可直接在本地将两份份额相加，得到：

$$f_3(i) = f_1(i) + f_2(i). \quad (1-2)$$

这些新份额对应的多项式为：

$$f_3(x) = f_1(x) + f_2(x). \quad (1-3)$$

由于多项式加法保持常数项线性关系，因此有：

$$f_3(0) = f_1(0) + f_2(0) = S_1 + S_2. \quad (1-4)$$

由此可见，秘密共享下的加法可以完全在本地完成，不需要额外交互。

相比之下，乘法运算不能像加法那样直接保持原有门限结构。若各参与方直接在本地计算份额乘积 $f_1(i) \cdot f_2(i)$ ，则所得结果对应的是多项式

$$g(x) = f_1(x)f_2(x), \quad (1-5)$$

其常数项满足：

$$g(0) = f_1(0)f_2(0) = S_1S_2, \quad (1-6)$$

但多项式次数会从 $t - 1$ 上升到 $2t - 2$ ，因而不能直接继续作为原门限结构下的共享结果使用。因此，实际系统通常不采用直接局部相乘然后长期保留高次数共享的方式，而是通过额外的交互式协议将乘法结果重新转换回原有门限共享形式。在实现上，本文使用的是预处理的乘法三元组（Beaver triples）^[47]。

位运算的实现依赖于对秘密共享整数的二进制分解。具体而言，本文将秘密共享形式的整数 x 拆解为 ℓ 个秘密比特 $\{x_k \in \{0, 1\}\}_{k=0}^{\ell-1}$ ，从而将位逻辑运算转化为份额域上的代数计算。对于按位与（AND），其结果可表示为：

$$b_1 \wedge b_2 = b_1 b_2. \quad (1-7)$$

对于按位异或（XOR），利用比特满足 $b_i^2 = b_i$ 的特性，可将其改写为：

$$b_1 \oplus b_2 = b_1 + b_2 - 2b_1 b_2. \quad (1-8)$$

对于按位非（NOT），则有：

$$\neg b = 1 - b. \quad (1-9)$$

这种转换确保了基本位运算均能归约为份额域上的加法与乘法组合。对于进一步涉及的比较、相等判断以及整数到比特串的转换，本文采用位分解与比较协议来实现。这类协议的基本思想正是先将秘密共享的整数转换为秘密比特表示，再通过份额域上的代数计算完成比较与相等测试。本文在实现上主要参考了 Nishide 等提出的比较与相等测试协议^[48]，以及 Toft 关于位分解的相关研究工作^[49]。

数据无关计算（Data-Oblivious Computation）是将 SMPC 算法工程化落地时必

须满足的重要约束。仅对数据做秘密共享并不足以保证隐私：若算法的分支选择、循环次数或访存模式依赖于秘密输入，仍可能通过执行行为泄露敏感信息。数据无关计算要求算法的执行轨迹仅依赖公开参数（如输入规模），而不依赖私有数据的具体取值^[50-51]。因此，在本文的多管理域数据面语义生成过程中，前缀匹配、规则选择与路径更新等操作都需要采用数据无关的实现方式，以避免在协作计算过程中通过控制流泄露本域策略信息。

1.3.3 零知识证明

零知识证明（Zero-Knowledge Proof, ZKP）用于解决如下问题：证明者如何在泄露秘密内容的前提下，向验证者证明某个命题成立。其基本定义由 Goldwasser、Micali 和 Rackoff 系统提出：一个证明协议若同时满足**完备性**、**可靠性**与**零知识性**，则验证者除知道该命题成立这一事实外，不应获得关于秘密见证的额外信息^[52]。在本文场景中，零知识证明用于在查询期向外部审计者提供可独立复验的证据接口：证明方在不公开真实转发路径与域内策略细节的前提下，证明某个覆盖对象在某一版本状态下满足或者不满足特定数据面不变量。

早期零知识证明主要以**交互式证明**的形式出现，证明者与验证者需要进行多轮消息交互，典型工作包括图同构、哈密顿环等 NP 语言上的零知识协议^[52]。这类协议在理论上奠定了零知识证明的基础，但在系统部署中存在交互轮次较多、难以离线验证以及不便公开转发等局限。为降低交互开销，研究界进一步提出了**非交互式零知识证明**（Non-Interactive Zero-Knowledge, NIZK）的思想，通过公共参考串或 Fiat-Shamir 启发式将交互过程压缩为单条证明消息，使验证者能够在离线条件下独立完成验证^[53-54]。在此基础上，又逐步发展出强调证明简洁性与验证高效性的**零知识简洁非交互式论证**（Zero-Knowledge Succinct Non-interactive Argument of Knowledge, ZK-SNARK）体系；这一方向能够在证明大小和验证时间上维持较低复杂度，因此被广泛用于可验证计算与隐私保护系统^[55-58]。

从实现角度看，现代 ZK-SNARK 通常包含以下几个组成部分。首先，需要将待证明的程序逻辑转化为算术约束系统；其次，需要执行参数生成过程，得到证明键与验证键。对于需要可信设置的证明系统，该过程既可以由受信第三方完成，也可以由多个相互独立的参与者联合执行多方参数生成仪式；在后一种情形下，只要参与者中至少有一方诚实且未保留有害中间材料，生成结果即可被视为可信^[59]。随后，证明方基于公开输入与私有见证生成证明；最后，验证方使用验证键对证明

进行独立核验。图 1.6 可用于展示这一基本流程。

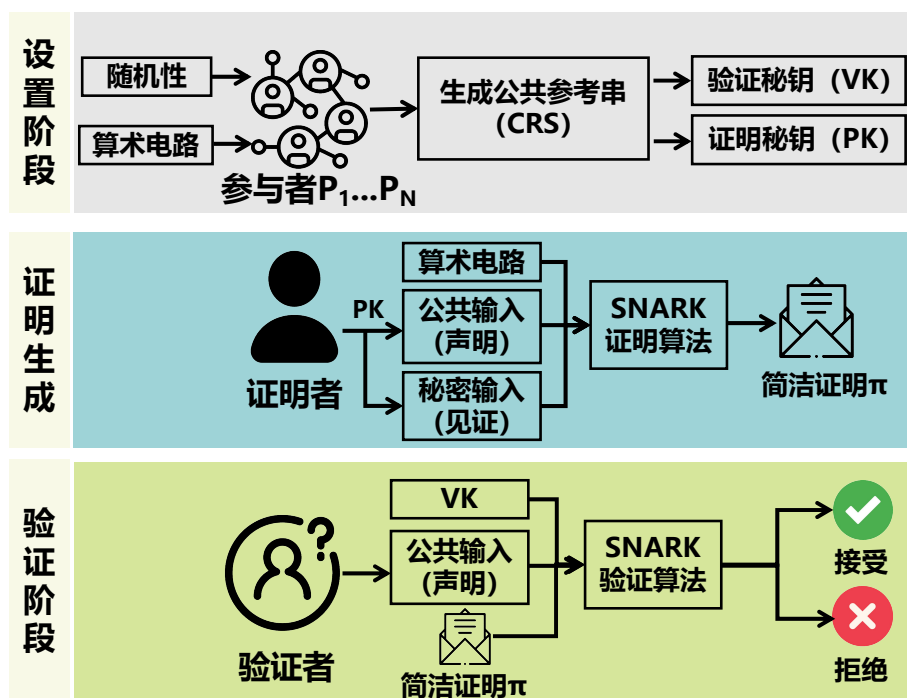


图 1.6 ZK-SNARK 的基本实现流程

更具体地说，ZK-SNARK 的实现过程可概括为四个阶段：

- **1. 电路构造阶段：** 设计者将原始程序逻辑改写为有限域上的算术电路，并进一步转换为 R1CS (Rank-1 Constraint System) 等算术约束系统。
- **2. 参数生成阶段：** 系统执行 Setup 算法，生成证明键 PK (proving key) 与验证键 VK (verification key)。
- **3. 证明生成阶段：** 证明方根据公开输入与私有见证 (Witness)，调用证明算法生成大小固定的简洁证明 π 。
- **4. 证明验证阶段：** 验证方利用 VK 对公开输入与证明 π 执行验证算法，并输出接受或拒绝的判定结果。

秩一约束系统 (Rank-1 Constraint System, R1CS)。 为将待证明的算术逻辑转化为证明系统可处理的形式，通常需要先原始程序编译为约束系统。R1CS 是 ZK-SNARK 中最常见的电路表示形式之一^[55-56]。其基本思想是：将输入、见证和中间变量统一表示为赋值向量，再把程序中的算术关系写成两个线性形式的乘积等于另一个线性形式的约束。

形式上，设赋值向量为：

$$\mathbf{z} = (z_0, z_1, \dots, z_m),$$

其中通常取 $z_0 = 1$ 作为常数项，其余分量对应公开输入、私有见证以及中间变量。对于任意一条约束，R1CS 要求其满足：

$$\langle \mathbf{a}_i, \mathbf{z} \rangle \cdot \langle \mathbf{b}_i, \mathbf{z} \rangle = \langle \mathbf{c}_i, \mathbf{z} \rangle, \quad (1-10)$$

其中 $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i$ 为第 i 条约束的系数向量， $\langle \cdot, \cdot \rangle$ 表示向量内积。若一段程序包含 N 条约束，则只要存在一组赋值向量 $\mathbf{z}$ 使全部约束同时成立，就说明相应程序逻辑成立。

从表达能力上看，加法、减法和与常数的线性组合可以直接并入线性形式中，乘法关系则通过式 (1-10) 显式表示；更复杂的表达式可通过引入中间变量拆解为多条局部约束。由此，R1CS 为从算术程序到零知识证明系统之间提供了统一的约束表示接口。

下面以一个 ZK-SNARK 例子说明其具体实现过程。设证明者希望向验证者证明：自己知道两个私有整数 x, y ，使得它们满足公开关系

$$x \cdot y + x = p, \quad (1-11)$$

其中 p 为公开输入。为了便于举例，设公开输入为 $p = 6$ ，而证明者持有的私有见证为 $(x, y) = (2, 2)$ 。验证者希望确认证明者确实知道这样一组私有见证，但不希望知道 x 与 y 的具体取值。

为将该命题转化为可证明的约束系统，首先引入一个中间变量 $t = xy$ ，则原命题可分解为两个步骤：其一，验证 $x \cdot y = t$ ；其二，验证 $x + t = p$ 。因此，可构造赋值向量

$$\mathbf{z} = (1, x, y, t, p),$$

其中第一个分量 1 用于表示常数项。于是，上述关系可写成如下两条 R1CS 约束：

$$x \cdot y = t, \quad (1-12)$$

$$(x + t) \cdot 1 = p. \quad (1-13)$$

相应地，可得到如下 R1CS 矩阵表示。设变量顺序固定为 $(1, x, y, t, p)$ ，则两条

约束对应的系数向量分别为:

$$\mathbf{a}_1 = (0, 1, 0, 0, 0), \quad \mathbf{b}_1 = (0, 0, 1, 0, 0), \quad \mathbf{c}_1 = (0, 0, 0, 1, 0),$$

$$\mathbf{a}_2 = (0, 1, 0, 1, 0), \quad \mathbf{b}_2 = (1, 0, 0, 0, 0), \quad \mathbf{c}_2 = (0, 0, 0, 0, 1).$$

进一步写成矩阵形式为:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

对赋值 $\mathbf{z} = (1, 2, 2, 4, 6)$ 而言, 可以直接检查: 第一条约束对应 $2 \cdot 2 = 4$, 第二条约束对应 $(2 + 4) \cdot 1 = 6$, 因此该赋值满足全部约束。

接下来, 需要将 R1CS 转换为 QAP (Quadratic Arithmetic Program) 形式。设共有两条约束, 则选取两个约束点 $r_1 = 1, r_2 = 2$, 并为每个变量构造三个插值多项式族 $\{u_j(X)\}, \{v_j(X)\}, \{w_j(X)\}$, 使其在点 r_i 处分别等于矩阵 A, B, C 中对应列的值。按上述矩阵可得到:

$$\begin{aligned} u_0(X) &= 0, & u_1(X) &= 1, & u_2(X) &= 0, & u_3(X) &= X - 1, & u_4(X) &= 0, \\ v_0(X) &= X - 1, & v_1(X) &= 0, & v_2(X) &= 2 - X, & v_3(X) &= 0, & v_4(X) &= 0, \\ w_0(X) &= 0, & w_1(X) &= 0, & w_2(X) &= 0, & w_3(X) &= 2 - X, & w_4(X) &= X - 1. \end{aligned}$$

于是, 对赋值向量 $\mathbf{z} = (z_0, z_1, z_2, z_3, z_4)$, 可定义

$$U(X) = \sum_{j=0}^4 z_j u_j(X), \quad V(X) = \sum_{j=0}^4 z_j v_j(X), \quad W(X) = \sum_{j=0}^4 z_j w_j(X). \quad (1-14)$$

代入本例中的赋值 $\mathbf{z} = (1, 2, 2, 4, 6)$, 可得

$$U(X) = 2 + 4(X - 1) = 4X - 2,$$

$$V(X) = (X - 1) + 2(2 - X) = 3 - X,$$

$$W(X) = 4(2 - X) + 6(X - 1) = 2X + 2.$$

同时, 约束点对应的目标多项式为

$$Z(X) = (X - 1)(X - 2). \quad (1-15)$$

此时有

$$U(X)V(X) - W(X) = (4X - 2)(3 - X) - (2X + 2) = -4X^2 + 12X - 8,$$

进一步可化简为

$$U(X)V(X) - W(X) = H(X) \cdot Z(X), \quad (1-16)$$

其中

$$H(X) = -4.$$

这说明给定赋值确实满足该 QAP；反之，若赋值不能满足原始约束，则上式一般不会被 $Z(X)$ 整除。由此，原本的程序正确性问题就被转化为了一个多项式可整除性问题。

在此基础上，ZK-SNARK 的可信设置 (Setup) 阶段需要围绕该 QAP 执行一次参数生成。以 Groth16 为例，系统首先随机采样一组仅在设置阶段使用的陷门参数

$$(\tau, \alpha, \beta, \gamma, \delta),$$

并在相应双线性群中计算与 $u_j(\tau), v_j(\tau), w_j(\tau), Z(\tau)$ 相关的一系列编码值，进而生成证明键 **PK** 与验证键 **VK** [55, 57]。其中，**PK** 供证明者使用，用于在给定公开输入与私有见证后生成证明；**VK** 则供验证者使用，用于对公开输入与证明执行独立验证。可信设置的本质，是将原本关于 QAP 多项式的验证问题固定到一个秘密评估点 τ 上，并通过群元素编码将其转换为简洁证明的验证关系。

证明阶段中，证明者输入公开值 $p = 6$ 以及私有见证 $(x, y, t) = (2, 2, 4)$ ，利用 **PK** 生成证明 π 。验证阶段中，验证者输入公开值 $p = 6$ 、验证键 **VK** 以及证明 π ，检查其是否通过验证。若验证通过，则验证者可确认：证明者确实知道一组私有见证，使得式 (1-11) 成立；与此同时，验证者不会从证明中直接得知 x 与 y 的具体数值。

由上述示例可以看出，ZK-SNARK 的实现过程是先将待证明命题转化为约束系统，再通过可信设置、证明生成与证明验证，将存在一组满足约束的私有见证这一事实压缩为一个可独立验证的简洁证明。本文后续的查询期审计电路将沿用与上述示例相同的实现范式，即先将版本绑定与数据面不变量检查编码为约束，再利用 ZK-SNARK 机制生成证明，从而使外部审计者能够在不接触真实路径与配置细节的前提下独立验证审计结论。

1.3.4 MiMC 承诺

MiMC 是一类面向有限域算术电路设计的轻量级密码学置换/哈希原语，其设计初衷在于降低零知识证明与相关算术约束系统中的实现成本。该原语最早由 Albrecht 等人系统提出，核心特点是在有限域上仅使用加法、轮常数注入与低次幂映射迭代构造轮函数，从而避免传统位运算型哈希在算术电路中的高额代价^[60]。由于 MiMC 的计算主要由有限域乘法与加法组成，因此特别适合作为零知识证明、可验证计算以及隐私计算场景中的哈希与承诺构件。

从形式上看，设有限域为 $\mathbb{F}_q$ ，输入状态为 $x \in \mathbb{F}_q$ ，轮常数序列为 $\{c_i\}_{i=1}^r$ ，指数参数为 d ，则 MiMC 的典型迭代形式可写为：

$$x_{i+1} = (x_i + k + c_i)^d, \quad i = 0, 1, \dots, r-1,$$

其中 $x_0 = x$ ， k 为密钥或固定参数， r 为轮数。经过 r 轮迭代后，得到最终输出 x_r 。在哈希或承诺应用中，通常将待编码对象按固定顺序映射到有限域元素，对于单个域元素输入，MiMC 可以直接作为有限域上的迭代压缩原语使用；而对于由多个字段组成的对象，通常先按照预先约定的顺序将其序列化为一个有限域元素序列，再采用迭代吸收的方式逐个处理这些元素，最终得到单个摘要值。这样，MiMC 就可以从单输入情形自然扩展到对象级的多输入承诺计算，从而得到最终摘要值。这里的轮数 r 并不是任意给定的，而是依赖于安全参数、域大小和设计目标来选取。

从承诺功能角度看，MiMC 承诺是本文所采用的对象级承诺中的底层压缩原语。系统先对对象执行固定格式的编码，再利用 MiMC 计算该对象的路径见证的摘要值，并将该摘要作为对象的承诺表示。一旦对象内容发生变化，其对应摘要也会变化，从而能够为后续的集合绑定与成员性证明提供稳定、紧凑且适合算术电路的叶子表示。

1.3.5 Merkle 树

Merkle 树是一种典型的哈希树结构，其思想最早由 Merkle 提出，用于在公开一个短摘要的前提下，对大规模对象集合进行整体绑定与高效成员性验证^[61]。它的基本做法是：先对底层对象分别计算叶子哈希，再将相邻节点两两组合并继续哈希，逐层向上构造，直到得到唯一的根值。该根值可以被看作对整组对象的统一摘要，因此 Merkle 树本质上是一种基于哈希递归聚合得到的树形承诺结构。

形式上, 设底层对象集合为 $\{m_1, m_2, \dots, m_n\}$, 其叶子节点可写为 $\text{leaf}_i = H(m_i)$, $i = 1, 2, \dots, n$, 其中 $H(\cdot)$ 表示底层哈希函数。对任意内部节点, 若其左右子节点分别为 u 与 v , 则父节点按如下方式计算: $\text{node} = H(u \| v)$, 其中 $\|$ 表示串接。经过自底向上的递归构造后, 最终得到根节点 root 。若某个底层对象发生变化, 则对应叶子及其到根路径上的节点值都会变化, 从而导致根值改变。因此, Merkle 根能够对整组对象提供整体绑定。

Merkle 树的一个关键性质在于其成员性证明能力。对于某个给定叶子, 只需提供从该叶子到根路径上的兄弟节点值以及左右位置关系, 就可以自底向上重构根值, 并验证该叶子是否确实属于该树所表示的对象集合。与直接公开整组对象相比, 这种方式使验证者只需处理与目标对象相关的一条认证路径, 而无需接触其他对象内容, 因此在大规模对象集合上具有很好的效率优势。

本文采用 Merkle 树作为全局审计状态绑定与证据检索的核心机制。系统将每个覆盖对象的见证作为底层对象, 构建 Merkle 树以得到版本根, 并通过版本根来绑定与标识某一时刻的全局审计状态。

1.4 研究内容

尽管学术界已提出多种数据平面验证技术, 但在多管理域环境中, 问题的关注点不再只是如何完成一次验证计算, 而是如何围绕数据平面配置引发的异常, 为事后复核、责任界定与持续询证提供可独立验证的证据支撑。相较于单管理域场景, 多管理域环境下的验证对象、部署假设与工程约束均发生了变化: 一方面, 审计所面对的并不是抽象的控制面可见结果, 而是由各域内部转发表项、边界过滤与策略规则共同决定的有效数据面语义; 另一方面, 各自治系统通常不愿以明文形式披露配置细节, 使相关结论难以通过集中收集状态的方式直接获得。与此同时, 事故处置和复盘往往伴随多轮修复、重复核验与版本对比, 这进一步要求系统能够围绕特定时刻的状态形成可追溯、可复验的审计证据。

基于上述分析, 本文认为, 多管理域数据平面审计至少面临以下三个核心挑战。

(1) **数据面语义的隐私生成挑战**。多管理域场景中的真实转发语义由各域内部转发表项、边界过滤与策略规则共同决定, 而这些状态通常无法以明文方式集中收集。如何在不泄露各域配置细节的前提下, 围绕覆盖对象生成能够反映真实数据面行为的语义见证, 是支撑后续审计的首要前提。

(2) **审计证据的低延迟交互挑战**。若每一次审计查询都要求各参与方重新在线执行完整的隐私协作计算,则查询成本会随复查次数不断累积,难以支撑现实中的持续询证需求。如何将生成阶段得到的对象级语义见证转化为可绑定、可公开验证、可独立复验的证据结构,是多管理域审计接口设计中的关键问题。

(3) **运行期变更下的高效维护挑战**。现实网络中的配置与策略更新是常态,若每次状态变化都触发全量重算与全量重建,则难以支撑故障复盘、修复跟踪与跨版本比较。如何在状态变化后仅对受影响对象及其证据结构执行局部更新,并形成新的可验证版本,是持续审计场景中的核心工程问题。

围绕上述挑战,本文的研究内容聚焦于:在不泄露各自治系统数据面配置细节的前提下,面向覆盖对象生成由真实数据面状态决定的语义见证及其不变量结论。并以此为基础,构建一种可独立复验、可绑定特定版本状态、且能适应频繁变更的多管理域数据平面审计方法。具体而言,本文的研究内容包括以下三个方面。

(1) **多管理域隐私约束下的数据平面语义见证生成**。针对真实数据面语义难以集中获取的问题,本文研究在隐私计算框架下如何围绕覆盖对象生成可用于审计的数据平面语义见证。该部分内容以各域本地数据面配置与策略为输入,对转发表项、边界过滤规则与策略约束进行统一编码,在安全多方计算过程中按照覆盖对象逐一执行数据面语义推演,获得由真实配置共同决定的对象级数据面转发路径。对于每一个覆盖对象,系统输出其在当前版本下的实际转发结果,包括逐跳转发路径、路径终止状态以及与该对象关联的结构化语义字段,并将这些信息封装为后续承诺、证明与查询所使用的对象级语义见证。该部分研究内容为后续审计阶段提供了可直接绑定到特定对象和特定版本的数据面证据。

(2) **面向事后复核的版本化审计接口构建**。针对重复核验成本高、外部主体难以独立复验的问题,本文研究如何将生成阶段得到的语义见证组织为面向查询阶段的版本化证据结构。该部分内容对对象级见证进行密码学绑定,并通过 Merkle 聚合形成与特定版本状态对应的版本根;在查询阶段,由证明方围绕相应版本输出零知识证明,使外部主体在不接触真实转发路径与域内策略细节的前提下,能够独立核验某一数据平面不变量结论是否确实来源于该版本下的已生成见证。该研究内容的核心在于将重复核验从多方在线重算转化为面向外部的低交互验证过程。

(3) **面向持续审计的版本化维护与增量更新**。针对多管理域环境中配置与策略频繁变化、审计证据需要持续维护的问题,本文研究运行期变更下的局部更新

方法。该部分内容包括两个方面：其一，在生成阶段建立增量输入接口与驱动方式，并设计面向受影响覆盖对象的增量推演算法，使配置变化后仅需对相关对象重新生成语义见证；其二，在证据结构层面对受影响叶子的承诺进行更新，沿对应 Merkle 分支局部重算并发布新版本根，使后续查询验证能够显式绑定到新的版本状态。通过上述设计，系统能够在避免全量重算与全量重建的前提下支持运行期持续核验，并使不同时间点的审计结论能够被清晰地区分、追溯与复验。

通过上述研究内容，本文围绕多管理域数据平面事故后的审计与复核需求，形成了一套以数据平面不变量为审计对象、以隐私保护下的验证计算为技术支撑、并能够对外提供独立复验能力的审计方法。

1.5 论文章节安排

本文共分为七个章节，整体框架如图 1.7 所示。全文围绕在隐私约束下生成真实多管理域数据平面语义，并基于版本化证据对外提供可独立复验的审计能力这一主线展开，其中第 3 章 (§3) 至第 5 章 (§5) 为本文研究工作的核心内容。各章节安排如下。

第 1 章为绪论 (§1)。本章首先介绍多管理域数据平面审计的研究背景与现实意义，分析现有方法在隐私约束、数据面真实语义获取、多次核验以及运行期变更复核等方面存在的不足；随后综述集中式数据平面验证、面向大规模与动态更新的可扩展验证、多管理域隐私协作验证以及可验证审计与证据机制等相关研究以及他们的局限性；接下来介绍本文涉及的基本概念与相关技术，包括多管理域数据平面语义与验证对象、安全多方计算以及零知识证明；最后明确本文的研究内容、技术路线与全文组织结构。

第 2 章为系统概述 (§2)。本章首先随后给出系统整体架构，说明自治系统节点、生成期计算节点、证明与托管服务以及外部验证者等功能角色及其执行阶段；接着通过一个具体示例展示系统从隐私语义生成到查询期复验的完整工作流程；最后给出威胁模型与隐私风险分析。

第 3 章为基于门限秘密共享的多管理域数据平面语义生成 (§3)。本章围绕生成期展开，首先定义多管理域数据平面语义生成问题、覆盖对象与二元组级语义见证，并明确生成期的输入、输出与系统边界；随后介绍基于门限秘密共享的多管理域数据平面语义生成方法，包括异构配置的编码与门限分片、秘密分享域上的逐对象语义推演以及版本化见证的结构化封装；最后对该生成机制的安全性与正

确性进行分析。

第 4 章为基于零知识证明的版本化数据平面审计 (§4)。本章围绕查询期审计接口展开，首先形式化定义多管理域审计问题；随后说明版本化证据结构的构建逻辑，包括语义见证的哈希承诺生成、Merkle 聚合与版本根构造；在此基础上，进一步给出路径属性审计电路的实现细节；最后对系统审计接口的安全性与正确性进行分析。

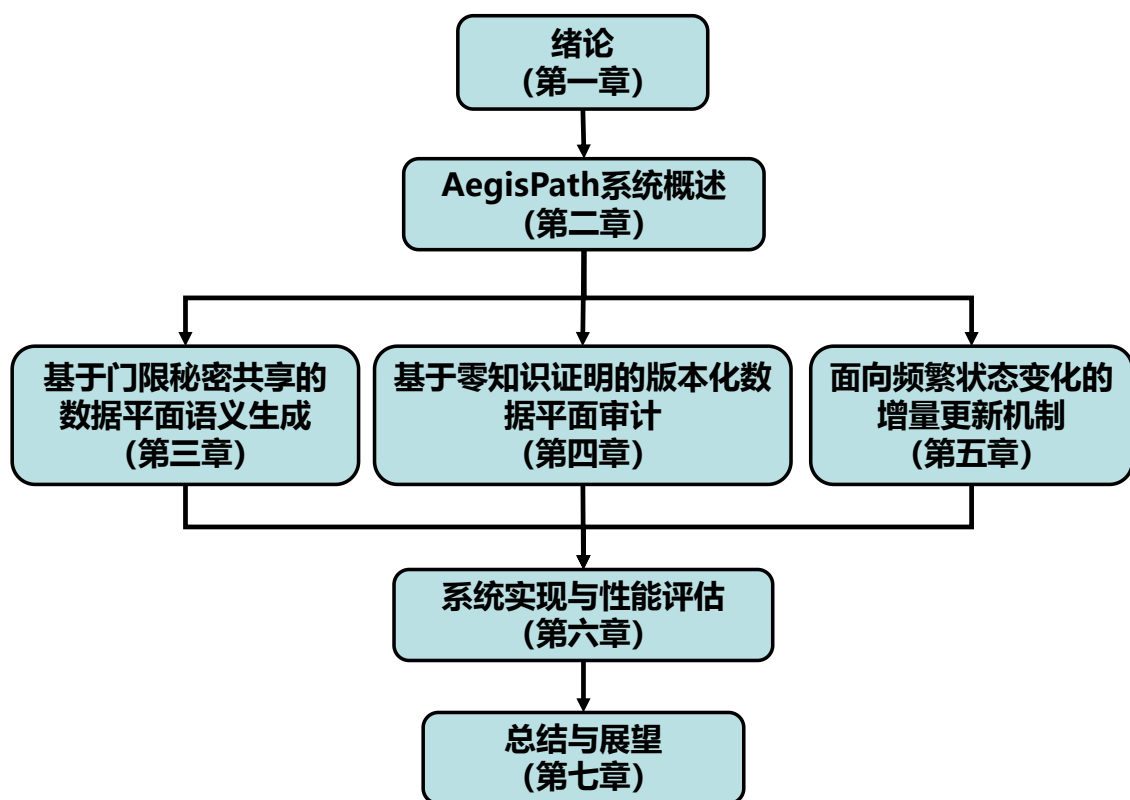


图 1.7 论文组织架构图

第 5 章为面向频繁状态变化的增量更新机制 (§5)。本章面向频繁配置变更与持续审计需求，首先介绍生成期增量更新机制，包括增量输入接口与驱动方式、增量推演算法以及复杂度与性能预期；随后讨论证据结构的增量维护方式，包括受影响叶子的承诺更新、Merkle 分支更新与新版本根发布；最后对本章提出的版本化维护与增量更新机制进行安全性与正确性分析。

第 6 章为系统实现与性能评估 (§6)。本章首先介绍系统实现、测试环境与实验数据集；随后从三个方面对 AegisPath 进行评估：生成期协作计算在不同规模多管理域网络中的开销与可扩展性，查询期零知识审计在证明与验证阶段的性能开

销，以及版本化与增量更新机制在局部变化场景下的性能表现；最后对实验结果进行总结。

第 7 章为总结与展望 (§7)。本章对全文工作进行总结，归纳本文在多管理域数据平面语义生成、版本化证据组织、零知识审计接口以及增量维护机制等方面的主要贡献，并对未来可进一步扩展的研究方向进行展望。

1.6 本章小结

本章围绕多管理域数据平面审计的研究背景、相关工作、基础技术与研究内容展开。首先，从数据面配置引发的异常处置、事后复核与持续询证需求出发，分析了多管理域场景下面临的隐私约束、证据复验以及运行期维护等关键挑战；随后，梳理了数据平面验证、多管理域协作验证与可验证审计等方向的研究进展，并总结了现有方法在配置隐私保护、外部独立复验与持续维护能力方面的不足。在此基础上，本章进一步介绍了本文所依赖的基本概念与相关技术，并概述了围绕数据平面语义生成、版本化审计接口以及增量更新机制展开的研究内容与技术路线。最后，本章给出了全文的章节安排与组织结构，为后续系统设计、协议实现与实验评估奠定了整体叙事基础。

第 2 章 AegisPath 系统概述

本文提出一套面向多管理域场景的数据平面审计系统 AegisPath。通过将隐私协作计算用于多管理域数据面语义生成，并在版本根绑定下借助零知识证明对外提供独立复验接口，在不泄露域内策略与路径细节的前提下，实现了可扩展的验证与持续核验能力。

本章首先介绍系统的整体架构 (§2.1)；接着通过一个示例说明系统从生成到核验的完整流程 (§2.2)；最后给出威胁模型与安全性讨论 (§2.3)，明确隐私边界、威胁模型以及可复验性的保证来源。

2.1 系统架构

在多管理域环境中，数据平面不变量所对应的真实转发语义取决于各自治系统内部的转发表项、边界过滤与策略配置，无法仅凭公开控制面观测直接推出；而若要求各域公开这些状态，又会突破实际运行中的隐私与商业边界。基于这一约束，AegisPath 首先在生成阶段采用隐私协作计算，在不集中暴露各自治系统明文配置的前提下，围绕覆盖对象生成由真实数据面状态决定的语义见证。

在此基础上，系统进一步面向外部复核场景构建低交互查询机制。若每次审计查询都重新组织一次在线多方计算，计算与通信开销会随复查次数不断累积，难以支撑事故复盘、责任界定与持续询证中的反复核验需求。为此，AegisPath 对生成阶段得到的对象级见证进行密码学绑定，并以 Merkle 树聚合形成公开版本根；在查询阶段，证明方围绕相应版本下的见证输出与查询结论对应的零知识证明，使外部主体能够在不接触真实转发路径与域内策略细节的前提下完成独立复验，并将审计结论明确绑定到特定版本状态。

考虑到多管理域网络中的策略调整、过滤规则变更与修复操作会持续发生，AegisPath 还进一步引入版本化维护与增量更新设计。当局部状态发生变化时，系统仅对受影响覆盖对象重新生成见证，并更新相应承诺叶子及其到根分支，随后发布新的版本根。这样，不同时间点的审计结论都能够对应到各自版本下的状态，同时也能够在持续运行环境中以较低代价支持反复复核。

如图2.1所示，基于上述设计考虑，AegisPath 形成了一个由生成期与查询期组成的分层架构。前者负责在隐私约束下生成并绑定覆盖对象上的数据平面语义见

证，后者负责围绕已发布版本根输出可独立验证的审计证明。下面分别从功能角色与执行阶段两个角度对系统进行说明。

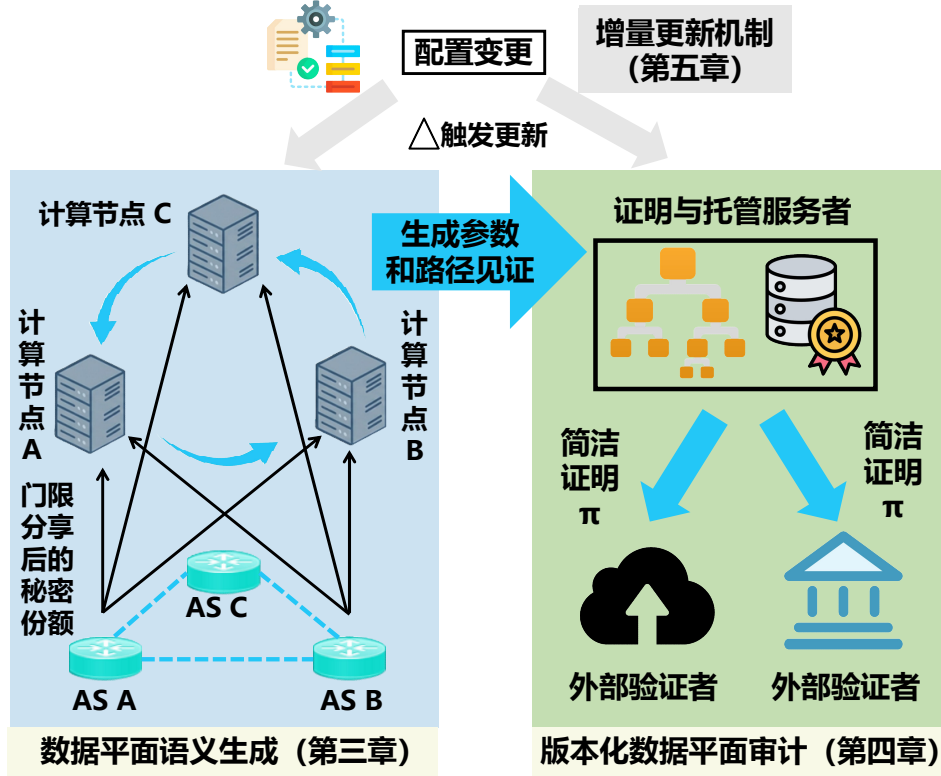


图 2.1 AegisPath 的系统结构

2.1.1 功能角色划分

按功能角色划分，AegisPath 包含以下四类实体。

(1) 自治系统节点。每个参与验证的自治系统作为配置提供方，维护本域与多管理域转发相关的数据面配置，包括前缀级转发表信息以及边界过滤、黑洞、策略路由等会影响多管理域数据平面语义的策略对象。AS 节点在本地完成配置抽取、编码与预处理，并向生成期计算节点提供秘密共享形式的输入。

(2) 生成期计算节点。该类节点在系统中承担两项相关但阶段不同的功能。在系统初始化阶段，它们可作为公共参数生成仪式的参与方，联合执行零知识电路所需的多方参数生成过程，并产出后续证明所需的证明参数与验证参数；在生成期阶段，它们继续作为门限秘密共享与数据无关执行框架下的联合计算节点，接收各自自治系统节点提交的共享份额，并完成覆盖对象上的数据平面语义推演。该阶段不恢复全局明文配置，而是直接在共享域中完成计算。在实际部署中，这类

节点可以由信誉良好或作为服务方运行的外部实体承担。

(3) 证明与托管服务者。在生成期结束后，该服务负责接收见证、计算叶子承诺、构造 Merkle 树并发布版本根。在查询期，该服务根据审计请求选取相应版本下的见证与认证路径，生成与版本根绑定的零知识证明。该服务同时承担版本化见证托管与查询证明生成两项功能。就部署方式而言，本文采用类似资源公钥基础设施（Resource Public Key Infrastructure, RPKI）^[62-63] 中集中发布，外部独立验证的模式：由一个集中服务统一托管和发布与版本状态相关的公开对象，外部各方无需接触内部状态，只需获取公开材料即可完成独立核验。在 RPKI 中，资源证书、吊销列表以及相关签名对象通过分布式资源仓库系统发布，供外部依赖方获取并验证；本文中的证明与托管服务与此类似，负责统一发布版本根、认证路径及相关证明材料，而不要求验证者接触底层见证明文。

(4) 外部验证者。外部验证者可以是监管机构、CERT/CSIRT、受影响网络、平台运营方或其他审计主体。验证者在查询期不接触见证明文，而只持有公开版本根、查询参数与证明。其职责是独立验证：某一关于覆盖对象的审计结论是否确由某个特定版本状态下的见证导出。

2.1.2 系统执行阶段

按任务阶段划分，AegisPath 的执行流程包括以下四个环节。**(1) 公共参数生成与发布阶段。**在零知识电路固定之后，生成期计算节点首先以多方参数生成仪式参与方的身份，联合执行公共参数生成过程，得到后续证明所需的证明参数与验证参数。其中，验证参数作为后续公开验证所需的公共材料发布，证明参数则由证明与托管服务持有并在查询期用于证明生成。该阶段仅在电路版本固定或参数需要更新时执行，不参与日常语义生成与审计查询流程。

(2) 预处理与秘密共享阶段。各 AS 节点首先将本地数据面配置转换为统一的中间表示，并将前缀、下一跳、动作标记以及边界策略对象编码为有限域上的定长整数向量或矩阵。随后，各 AS 节点对这些结果执行门限秘密共享，并将得到的份额发送给生成期计算节点。

(3) 隐私语义生成阶段。生成期计算节点接收共享份额后，围绕覆盖对象集合执行数据无关的数据平面语义推演。对每个覆盖对象 (p, s) ，系统从起始自治系统 s 出发，在共享域中执行前缀匹配、最长前缀匹配优先级消解、边界策略处置与逐跳路径推进，得到对应的数据平面语义见证。

(4) 承诺聚合与版本发布阶段。证明与托管服务接收生成期输出的见证集合后, 对每个见证计算叶子承诺, 并按照覆盖对象的固定顺序构造 Merkle 证据树, 发布版本根 R_v 。版本根作为某一时刻全局见证状态的公开摘要, 也是查询期审计的公共锚点。若网络状态在后续发生局部变化, 系统可仅对受影响对象重算见证并更新相应叶子与分支, 发布新的版本根 R_{v+1} 。

(5) 查询期零知识审计阶段。当外部验证者针对某个覆盖对象 (p, s) 及某一不变量 Φ 发起查询时, 证明与托管服务从对应版本的见证库中取出该对象的见证及 Merkle 认证路径, 并生成零知识证明。验证者基于公开版本根、查询描述与证明执行验证, 从而在不接触见证明文的前提下完成对结论的独立复验。

2.2 工作流程

本节通过一个具体的多管理域网络示例说明 AegisPath 的完整工作流程。通过选取一个固定的前缀与一个固定的起始自治系统, 展示系统如何从各 AS 的本地数据面配置出发, 依次完成隐私语义生成、MiMC 承诺、Merkle 版本根发布, 以及查询期零知识审计。图 2.2 给出了 AegisPath 的整体工作流程示意。

图 2.2 中, 目标前缀由 AS C 对外宣告, 其前缀为 134.34.0.0/24。起始自治系统为 AS S 。公共控制面观测系统观测到的候选路径为 $S \rightarrow A \rightarrow B \rightarrow C$ 。也就是说, 从控制面上看, 前缀 134.34.0.0/24 是经由 AS A 、AS B 到达 AS C 的。但是 AS B 在本地部署了一条更细粒度的数据面规则: 对于目的地址落在 134.34.0.128/25 范围内的流量执行 deny 处置。这一局部边界策略的存在使得同一控制面前缀在数据平面上不再对应单一一致的转发语义, 而需要在生成阶段围绕覆盖对象进一步细化并分别推演; 随后, 得到的对象级见证还需要进一步承诺、聚合并绑定到公开版本根上, 以支持查询期的零知识审计以及状态变化后的增量维护。以下将结合图 2.2 分步骤介绍这一过程: 步骤 1 和步骤 2 对应第 3 章中的多管理域数据平面语义生成, 重点说明各 AS 如何在隐私约束下完成配置预处理、秘密共享与对象级语义见证生成; 步骤 3 和步骤 4 对应第 4 章中的版本化数据平面审计, 说明对象级见证如何经过 MiMC 承诺、Merkle 聚合形成版本根, 并在查询阶段围绕特定不变量生成零知识证明; 步骤 5 对应第 5 章中的版本化维护与增量更新, 说明局部配置变化后受影响对象如何被重新生成、更新承诺并发布新版本根。

步骤 1: 各 AS 本地配置预处理与秘密共享。首先, 各自治系统在本地提取与数据平面语义相关的配置子集。以本示例为例, 至少包括两类关键内容: 其一,

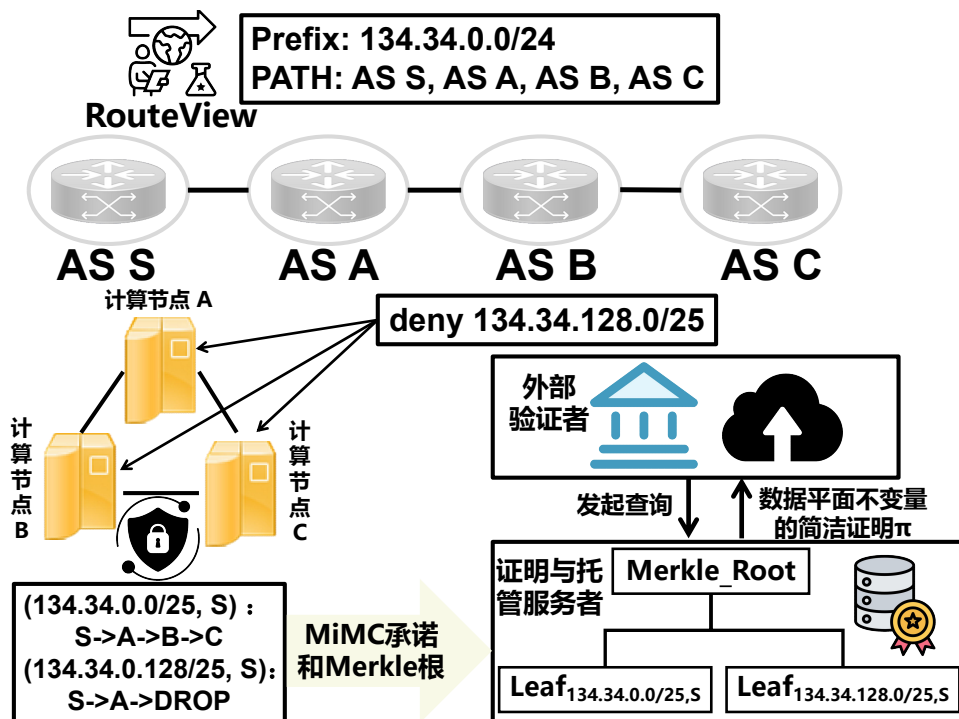


图 2.2 工作流程示例图

AS C 对前缀 134.34.0.0/24 的对外宣告所诱导出转发表项；其二，AS B 对该前缀内部地址空间施加的更细粒度边界规则，即对 134.34.0.128/25 执行 deny，而对 134.34.0.0/25 保持正常转发。随后，各 AS 将这些配置编码为统一的整数化表示，并执行门限秘密共享。此时上传给计算节点的不是明文配置，而只是共享后的份额。

步骤 2：生成期 SMPC 联合生成两个覆盖对象的见证。计算节点接收各 AS 的共享份额后，在共享域中执行数据平面语义推演。首先围绕 (134.34.0.0/24, S) 开始计算，从起始自治系统 S 出发，逐跳执行前缀匹配、最长前缀匹配优先级消解、边界策略处置与路径推进。当推演到 AS B 时，由于其本地规则对 134.34.0.0/24 内部的两个地址子区间给出了不同处置，系统需要将原始对象进一步细化为两个不相交的覆盖对象，并分别继续推演。

对于第一个对象 (134.34.0.0/25, S)，系统从 AS S 出发继续执行路径推进，得到有效转发路径 $S \rightarrow A \rightarrow B \rightarrow C$ 。因此，该对象对应的见证可以记为 $W_{v,134.34.0.0/25,S}$ ，其语义结论是可达。

对于第二个对象 (134.34.0.128/25, S)，系统同样从 AS S 出发进行推演，但在到达 AS B 时命中了针对该子前缀的 deny 规则，因此路径在 AS B 终止，无法继续到达 AS C。该对象对应的见证可记为 $W_{v,134.34.0.128/25,S}$ ，其语义结论是不可达。

也就是说, 在这个例子里, SMPC 的输出不是一个前缀对应一个统一结论, 而是原始基础对象 $(134.34.0.0/24, S)$ 在 AS B 的局部规则作用下被细化为两个不相交覆盖对象, 各自产生一个见证: 其中 $(134.34.0.0/25, S)$ 是可达对象, 而 $(134.34.0.128/25, S)$ 是不可达对象。

步骤 3: MiMC 承诺、Merkle 聚合与版本根发布。在生成期得到这两个对象的见证后, 系统分别计算它们的 MiMC 叶子承诺。例如, 类别 p_1 与 p_2 对应的叶子可分别记为 $\text{leaf}_{v,134.34.0.0/25,S} = \text{MiMC}(\text{Encode}(W_{v,134.34.0.0/25,S}))$ 和 $\text{leaf}_{v,134.34.0.128/25,S} = \text{MiMC}(\text{Encode}(W_{v,134.34.0.128/25,S}))$ 。随后, 系统按照公开约定的对象顺序排列这些叶子承诺, 并进一步构造 Merkle 树, 得到版本 v 下的公开根 R_v 。从此时起, 对外发布的不是见证明文, 而是版本根 R_v 。它绑定了该版本下整个见证集合的状态, 是后续查询期证明的公共锚点。

步骤 4: 查询期零知识审计。在该版本发布之后, 不同外部主体可以针对不同覆盖对象提出查询。

例如, Cloudflare 关心的是: 从 AS S 出发, 到 $134.34.0.128/25$ 这一部分流量是否可达。于是它发起查询 $(R_v, 134.34.0.128/25, S, \text{Reachability})$ 。证明服务从内部见证库中取出 $W_{v,134.34.0.128/25,S}$ 及其 Merkle 认证路径, 并在零知识电路中证明两件事: 第一, 该见证确实属于版本根 R_v 所绑定的见证集合; 第二, 对该见证执行可达性判定所得结果为不可达。最终, 系统返回结论 $y_{\text{CF}} = 0$ 以及对应的零知识证明 π_{CF} 。

再例如, APNIC 关心的是: 从 AS S 出发, 到 $134.34.0.0/25$ 这一部分流量时, 路径是否经过 AS B 。于是它发起查询 $(R_v, 134.34.0.0/25, S, \text{Waypoint}(B))$ 。证明服务取出 $W_{v,134.34.0.0/25,S}$ 及其认证路径, 在零知识电路中证明该见证属于版本根 R_v , 并且对该见证执行必经点判定的结果为经过 AS B 。最终, 系统返回结论 $y_{\text{APNIC}} = 1$ 及其证明 π_{APNIC} 。在这两个查询中, Cloudflare 和 APNIC 都不需要接触真实路径本身, 只需验证对应的零知识证明是否成立, 即可独立确认结论。

步骤 5: 局部更新后的版本维护与复核。最后, 如果经过一段时间后, AS B 调整了本地边界策略, 例如取消了对 $134.34.0.128/25$ 的 deny 规则。此时, Aegis-Path 并不需要重新对全部覆盖对象执行一次完整生成, 而只需识别出受影响的对象。在本示例中, 真正受影响的是 $(134.34.0.128/25, S)$, 因为 $(134.34.0.0/25, S)$ 原本就正常可达, 并未受到这条策略变化影响。于是, 系统只需重新生成新的见证 $W_{v+1,134.34.0.128/25,S}$, 更新其对应的 MiMC 叶子承诺以及相关的 Merkle 分支, 并发布

新的版本根 R_{v+1} 。此后, 审计者即可分别围绕 $(R_v, 134.34.0.128/25, S, \text{Reachability})$ 与 $(R_{v+1}, 134.34.0.128/25, S, \text{Reachability})$ 发起查询, 从而对更新前后的结果进行独立比较。

通过这个例子可以看出, AegisPath 的核心工作方式是: 先将一个公开观测到的基础前缀对象利用安全多方计算技术, 在生成期细化为若干个真正具有一致数据面语义的不相交覆盖对象, 再为这些对象生成见证; 随后对见证执行 MiMC 承诺和 Merkle 聚合, 发布版本根; 查询期则围绕特定对象和版本根生成零知识证明, 而不再重新组织多管理域联合计算; 若局部配置发生变化, 则系统只对受影响对象执行局部重算与版本维护。

2.3 安全性分析

本节给出 AegisPath 的整体安全性分析。围绕这一问题, 本文在系统设计中将结果可验证性与配置隐私性分别交由不同机制处理: 生成期通过安全多方计算避免各 AS 的明文配置被集中恢复, 查询期通过承诺绑定与零知识证明避免外部验证者在核验过程中获得路径与策略细节。因此, 本节从两个层面展开分析: 首先, 在威胁模型中明确系统所防御的攻击者能力与信任边界; 其次, 在隐私风险分析中讨论不同阶段可能出现的信息泄露面, 并说明本文架构如何对这些风险进行约束。

2.3.1 威胁模型

本文按生成期与查询期分别刻画 AegisPath 的威胁模型, 并将系统中的对手能力限定在两个阶段各自的交互边界内。

2.3.1.0.1 生成期威胁模型. 设参与隐私计算的实体集合为 $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$, 其中每个 P_i 表示一个参与安全多方计算的执行实体; 设秘密共享的重构阈值为 t 。对任意自治系统 AS_i , 记其本地私有配置为 C_i , 其秘密共享后的输入记为 $\llbracket C_i \rrbracket$, 本文在生成期采用半诚实模型: 各参与实体按照协议执行计算, 但可能试图从本地持有的份额、协议中间状态与执行轨迹中推断其他参与方的私有输入。

设攻击者控制的合谋实体集合为 $\mathcal{A} \subseteq \mathcal{P}$, 并满足 $|\mathcal{A}| < t$, 记攻击者在生成期的可见视图为:

$$\text{View}_{\mathcal{A}}^{\text{gen}} = \left(\{\llbracket C_i \rrbracket\}_{\mathcal{A}}, \text{param}_{\text{gen}}, \text{trace}_{\text{gen}}, \mathcal{W}_v \right),$$

其中 $\llbracket \mathcal{C}_i \rrbracket_{\mathcal{A}}$ 表示攻击者所控制实体持有的配置份额, $\text{param}_{\text{gen}}$ 表示生成期公开参数, $\text{trace}_{\text{gen}}$ 表示攻击者可观测的执行轨迹, $\mathcal{W}_v$ 表示协议输出的见证集合。本文要求: 对任意未被攻击者控制的自治系统 AS_j , 攻击者不能由 $\text{View}_{\mathcal{A}}^{\text{gen}}$ 恢复其明文配置 $\mathcal{C}_j$, 也不能获得除输出所必然泄露信息之外的额外知识。

2.3.1.0.2 查询期威胁模型. 设查询期存在一个负责见证托管、承诺聚合与证明生成的中心服务, 记为 **Prover**。对任意版本 v 、覆盖对象 (p, s) 及查询不变量 Φ , 查询期的公开输入记为:

$$\text{st} = (R_v, p, s, \Phi, y),$$

其中 R_v 为版本根, $y \in \{0, 1\}$ 为对外声称的结论; 私有见证记为:

$$\mathbf{w} = (W_{v,p,s}, \text{auth}_{p,s}),$$

其中 $W_{v,p,s}$ 为对象 (p, s) 在版本 v 下的语义见证, $\text{auth}_{p,s}$ 为对应的 Merkle 认证路径。

在该阶段, 本文不假设中心服务在结论生成上是可信的。也就是说, 中心服务可以输出任意候选结论 y , 并尝试围绕任意公开输入 st 生成证明 π 。但是, 外部验证者并不直接接受中心服务的结论陈述, 而仅依据公开输入与证明执行验证函数

$$\text{Verify}(\pi; \text{st}) \in \{0, 1\}.$$

因此, 查询期的安全目标可表述为: 若某一结论未能对应到某个满足关系 $\mathcal{R}(\text{st}, \mathbf{w}) = 1$ 的私有见证, 则攻击者不能使验证者以非可忽略概率接受该结论。即系统要求所有可被接受的查询期结论都必须与某个公开版本根所绑定的见证状态一致。

2.3.1.0.3 安全目标.

- **生成期输入隐私:** 对任意满足 $|\mathcal{A}| < t$ 的合谋实体集合, 攻击者不应从生成期视图中恢复任何未授权自治系统的明文配置。形式上, 对任意未被控制的自治系统 AS_j , 攻击者均不应由其可见视图推出对应的私有输入, 即:

$$\text{View}_{\mathcal{A}}^{\text{gen}} \not\Rightarrow \mathcal{C}_j, \quad AS_j \notin \mathcal{A}.$$

- **查询期结果完整性:** 对任意公开输入 st , 若不存在私有见证 $\mathbf{w}$ 使关系 $\mathcal{R}(\text{st}, \mathbf{w}) = 1$

成立，则攻击者至多只能以可忽略概率构造一个证明 π 使验证者接受，即

$$\Pr [\text{Verify}(\pi; \text{st}) = 1 \wedge \nexists w : \mathcal{R}(\text{st}, w) = 1] \leq \text{negl}(\lambda),$$

其中 $\text{negl}(\lambda)$ 表示关于安全参数 λ 的可忽略函数。

2.3.2 隐私风险分析

本小节讨论 AegisPath 在上述威胁模型下可能出现的隐私泄露面，并分析系统如何分别在生成期与查询期对这些风险进行约束。

2.3.2.0.1 生成期的配置恢复风险 若各自治系统将本地数据面配置直接提交给单一验证器，则验证器能够恢复完整的多管理域配置状态，从而获得前缀级转发表项、边界处置规则与策略逻辑。为避免这一风险，AegisPath 在生成期采用门限秘密共享与数据无关执行，仅允许参与方在共享域中联合生成见证，而不集中恢复底层明文配置。

引理 2-1. 在生成期，若任意合谋实体集合的规模严格小于重构阈值 t ，则攻击者从秘密共享份额与协议执行过程中不能恢复任一自治系统的明文配置；其可见信息被限制为自身输入、协议公开参数以及输出所必然泄露的信息。

证明. 该结论直接继承第三章关于门限秘密共享与数据无关执行的安全性分析。少于阈值的份额集合不足以恢复底层秘密，而固定的控制流与访问模式不引入额外的执行轨迹泄露，因此攻击者在生成期不能从协作计算过程中获得额外的明文配置知识。 $\square$

2.3.2.0.2 查询期的路径语义泄露风险 即便生成期不泄露底层配置，若系统在查询期直接公开见证明文，则外部验证者仍可从路径槽位、终止位置与边界处置结果中恢复多管理域真实转发路径及其语义细节。为此，AegisPath 在查询期不返回见证明文，而是仅发布版本根并围绕查询对象生成零知识证明，使外部验证者只能获得结论是否成立的结果，而不能恢复支撑该结论的完整路径语义。

引理 2-2. 在查询期，若审计证明系统满足零知识性，则攻击者从公开版本根、查询参数及证明中不能恢复与私有见证相关的额外路径语义；其可见信息被限制为公开输入与证明系统必然泄露的信息。

证明. 该结论直接继承第四章关于零知识审计接口的安全性分析。对任意查询，证明系统保证公开可见的证明分布可由模拟器生成，因此不会额外泄露私有见证中的路径槽位、边界处置与策略细节。□

2.3.2.0.3 跨阶段组合下的隐私泄露面 从整体上看，AegisPath 的工作流由生成期与查询期两部分组成，二者对应不同的泄露面：前者面向协作计算输入隐私，后者面向审计接口输出隐私。系统整体的隐私性取决于这两个阶段是否分别将泄露面限制在设计允许的范围之内。

定理 2.3.1. 在本节威胁模型下，AegisPath 的隐私泄露面可被分解为生成期输出所必然泄露的信息与查询期公开输入所必然泄露的信息；除这些信息外，攻击者不能从系统执行过程中额外恢复各自治系统的明文配置或查询对象对应的私有路径语义。

证明. 由引理 2-1，生成期不会额外泄露明文配置；由引理 2-2，查询期不会额外泄露私有见证。由于 AegisPath 的工作流由生成期与查询期两部分组成，且二者的公开接口分别受上述两条结论约束，故系统整体的隐私泄露面被限制在输出与公开输入所必然泄露的信息之内。□

2.3.2.0.4 生成期输入一致性假设的现实性讨论 除隐私泄露面之外，生成期还涉及各自治系统输入配置的一致性问题的讨论。对此，本文采用半诚实模型下常见的输入一致性假设：各参与方按照协议流程提交能够反映其当前有效数据面状态的输入，并在此基础上完成联合计算。这样的建模方式与已有多管理域隐私验证系统的常见处理保持一致。例如，IVeri 在多管理域协议验证中明确采用半诚实安全模型^[5]，Seagull 在隐私保护网络验证中同样建立在受限对手与按协议执行的前提之上^[6]。

这一假设在本文场景中具有现实合理性。生成期输入虽然由各自治系统在本地产地提供，但其导出的结果会在后续网络运行中持续体现为前缀可达性异常、异常绕路或持续不可达等对外症状。已有工作表明，多管理域网络中存在面向这类外部症状的主动观测手段。Hubble 持续监测覆盖互联网边缘地址空间约 89% 的前缀数据路径，并估计其能够发现全面探测本应发现的可达性问题中的 85%^[37]。在本文中，这一主动探测能力被用作周期性抽样核验的外部手段：系统每隔一个计费月，对当前对象集合进行一次随机抽样检查，并对被抽中的对象调用 Hubble 式主动探测，对其当前窗口内的可达性症状进行核验。由此，生成期输入失真并非零

风险、可持续隐藏的策略，其短期逃避收益与后续暴露代价之间存在可分析的长期权衡关系。

为刻画这一长期暴露过程，本文采用离散时间 hazard rate 模型^[64]。设 T 表示某一失真输入在第几轮月度分析窗口后被揭示，则第 t 轮的条件暴露风险定义为

$$h_t = \Pr(T = t \mid T \geq t),$$

该写法对应离散时间事件历史分析中常用的 hazard 定义，可用于描述在此前尚未暴露的条件下，本轮被揭示的条件概率。在本文场景中，设每轮随机抽样检查比例为

$$q = 0.02.$$

同时，记 Hubble 的覆盖因子为

$$\alpha = 0.89,$$

记其在覆盖范围内对可达性问题的发现能力为

$$\beta = 0.85.$$

则在齐次近似下，可将单轮条件暴露风险写为

$$h_t \equiv h = q\alpha\beta = 0.02 \times 0.89 \times 0.85 = 0.01513.$$

于是，未暴露至第 t 轮末的生存概率为

$$S_t = (1 - h)^t.$$

从长期激励角度看，生成期输入失真可视为一种在连续月度窗口中重复尝试、直到被揭示为止的行为。设单轮失真输入所暂时逃避的收益为 B_{evas} ，一旦暴露后至少需要承担的代价为 C_{after} ，则重复尝试直到被揭示为止的长期期望净收益可写为

$$\mathbb{E}[G_{\text{mis}}^{\text{rep}}] = \sum_{t \geq 0} S_t B_{\text{evas}} - C_{\text{after}} = \sum_{t \geq 0} (1 - h)^t B_{\text{evas}} - C_{\text{after}} = \frac{B_{\text{evas}}}{h} - C_{\text{after}}.$$

对于 B_{evas} ，本文采用 Google Cloud 专用与合作伙伴互联服务的信用上限作为保守代理。Google 官方服务等级协议（SLA）规定，在单个计费月内发放的财务信用额度总额，不超过当月受保服务（Covered Service）费用的 50%^[65]；同时，Google

官方定价页面显示，跨站互联的 100 Gbps 电路价格为 \$23.28/小时^[66]。按每月 730 小时折算，单条 100 Gbps 电路的月费用约为

$$M = 23.28 \times 730 = \$16,994.4,$$

故单月最多可逃避的服务信用为

$$B_{\text{evas}} \leq 0.5 \times 16,994.4 = \$8,497.2.$$

沿用前文统一法律成本口径，取

$$C_{\text{after}} = \$1,000,000.$$

代入可得

$$\mathbb{E}[G_{\text{mis}}^{\text{rep}}] \leq \frac{8,497.2}{0.01513} - 1,000,000 \approx 561,612.7 - 1,000,000 = -438,387.3 < 0.$$

这一结果表明，在采用月度 2% 抽样检查，并以 Hubble 的覆盖比例与问题发现能力作为核验手段时，即便使用对作假方更有利的收益代理与法律成本口径，长期重复提交失真输入仍然缺乏正的期望收益。这里的分析对象是持续性或重复性失真输入的长期激励，其约束效果来自周期性抽样核验与外部可达性检查的组合；对短时间内出现且在核验窗口前已消失的一次性伪造，仍需要依赖本文机制所提供的版本绑定语义证据来支撑事后复验。因而，在本文所采用的半诚实模型下，将生成期输入一致性视为一项工程前提具有现实合理性；若要进一步覆盖更强的恶意输入伪造，则仍需要引入远程度量、可信硬件、外部配置认证或更强的治理约束。

2.3.2.0.5 路径泄露后的可观测性分析 本小节讨论假如托管服务将路径泄露给某个自治系统后的攻击场景。路径见证一旦被第三方非法利用，便会显著降低敌手在多管理域路由攻击中的信息不对称，使其能够更高效地识别高价值目标、更准确地选择引流位置，并更有把握地预估攻击的可观测性与持续窗口。路径泄露后的风险并不止于信息暴露本身，还会进一步取决于相关行为发生后是否容易暴露，以及由此引致的现实代价。类似地，已有研究在主密码学机制之外，也会进一步引入额外约束来压制残余作恶激励：例如，Dziembowski 等提出的告密式秘密分享机制在秘密分享保护之外，为非法重构行为生成可验证证明，并进一步配合经

济惩罚压缩越界收益空间^[67]；Faust 等提出的金融支持型隐蔽安全机制则将作弊检测与金融惩罚直接绑定，用以提高作恶代价并削弱剩余激励^[68]。沿用既有路由攻击研究对前缀劫持（prefix hijacking）后果的分类框架^[69-70]，本文将路径泄露可能支撑的现实利用方式归纳为黑洞化（blackholing）、冒充响应（imposture）与中间拦截（interception）三类。

对任一攻击面 a ，记其在成功利用路径信息后可私有化的经济收益为 $B^{(a)}$ ，被发现并归责后承担的总代价为

$$C^{(a)} = C_{\text{legal}}^{(a)} + C_{\text{business}}^{(a)}.$$

于是，对应攻击面的期望净收益可写为

$$\mathbb{E}[G^{(a)}] = B^{(a)} - p_{\text{det}}^{(a)} C^{(a)}.$$

黑洞化（blackholing）攻击面。在黑洞化场景下，路径见证泄露主要增强了敌手在目标选择与引流位置选择上的确定性：敌手能够更快识别哪些前缀或哪些多管理域路径具有更高的流量汇聚性，并优先挑选更容易形成大范围不可达的目标；同时，敌手也更容易定位更接近关键汇聚节点的注入位置，从而减少试错成本并提高一次攻击即触发业务中断的概率^[70]。本文对该攻击面的收益刻画限定为攻击者可私有化的直接经济收益，并采用公开网络勒索金额作为收益代理。新加坡网络安全局关于网络勒索的公开说明^[71]给出的公开金额大致落在 \$200 至 \$25,000 区间，Cloudflare 2024 年第二季度的拒绝服务威胁报告^[72]则给出，在其统计到的受攻击组织中，有 12.3% 表示攻击伴随威胁或勒索。因此，若将黑洞化后进一步被转化为勒索支付的情形作为保守收益代理，则有

$$B^{(\text{blk})} \leq 0.123 \times 25,000 = \$3,075.$$

在检测侧，Ares^[73]对隐蔽精确前缀路径劫持与子前缀路径劫持的平均检测率分别达到 97.2% 与 99.3%。为保持量化口径对攻击者更有利，本文取其中较低的 97.2% 作为黑洞化攻击面的检测率代理，即

$$p_{\text{det}}^{(\text{blk})} = 0.972.$$

在法律成本上，本文统一采用中国《中华人民共和国网络安全法》^[74]、美国《美

国法典》第 18 编^[75]与欧盟《通用数据保护条例》所对应的代表性高罚款口径作为参照^[76]。其中，中国侧对造成特别严重影响或者特别严重后果的网络安全违法行为最高罚款可达 1000 万元，美国侧在涉及金融机构的线缆欺诈场景下罚金上限可达 \$1,000,000\$，欧洲侧罚款上限可达 2000 万欧元或全球营业额的 4%。为保持量化口径对攻击者更有利，本文在三者中取最低的 \$1,000,000\$ 作为统一法律成本代理，即

$$C_{\text{legal}}^{(\text{blk})} = \$1,000,000.$$

即便暂不计入业务代价，也有

$$\mathbb{E}[G^{(\text{blk})}] \leq 3,075 - 0.972 \times 1,000,000 = -968,925 < 0.$$

冒充响应(imposture)攻击面。在冒充响应场景下，路径见证泄露的价值体现为敌手能够更高效地识别高价值伪装目标。路径信息一旦被还原，敌手便更容易判断哪些前缀后方承载的是钱包、桥接、托管、认证网关等对交互真实性高度敏感的服务，并优先挑选那些更适合维持短时伪装且更可能直接形成资产转移的对象。对这一类攻击，本文采用公开披露的数字资产劫持事件作为收益代理。MyEtherWallet 事件公开报道损失约为 \$150,000\$^[77]，Celer Bridge 事件公开报道损失约为 \$235,000\$^[78]，因此可将

$$B^{(\text{imp})} \in [\$150,000, \$235,000].$$

在检测侧，DFOH^[79]对伪造起源攻击的检测率达到 90.9%，因此本文对冒充响应攻击面取

$$p_{\text{det}}^{(\text{imp})} = 0.909.$$

在法律成本上，本文沿用上述法律成本口径，取

$$C_{\text{legal}}^{(\text{imp})} = \$1,000,000.$$

即便暂不计入业务代价，也有

$$\mathbb{E}[G^{(\text{imp})}] \leq 235,000 - 0.909 \times 1,000,000 = -674,000 < 0.$$

中间拦截(interception)攻击面。在中间拦截场景下，路径见证泄露的价值主要体现在可持续性判断上。敌手若希望在不直接造成服务中断的情况下持续观察、记录甚至局部篡改流量，就必须挑选那些既能把流量吸引到自己，又仍然保有

可行回程路径的注入位置。路径见证一旦泄露，敌手便更容易识别哪些多管理域路径更适合维持引流后再转发回目的端这一闭环，从而降低试探成本并提高潜伏成功率^[70]。公开事件也说明该类攻击具有现实收益：2014 年针对加密货币矿池的劫持攻击被公开报道获利约 \$83,000^[80]，而 Celer Bridge 事件则表明一旦中间拦截或相近的中间人利用成功，收益可达 \$235,000 量级^[78]。因此，可将

$$B^{(\text{int})} \in [\$83,000, \$235,000].$$

在检测侧，BEAM^[81] 在真实 RouteViews 数据集与真实 ISP 部署上给出了稳定的异常检测结果。为避免将已确认异常集合上的全部检出直接等同于现实环境中的统一检测率，本文在量化时采用更保守的折减口径，并取

$$p_{\text{det}}^{(\text{int})} = 0.97.$$

在法律成本上，本文沿用上述法律成本口径，取

$$C_{\text{legal}}^{(\text{int})} = \$1,000,000.$$

即便暂不计入业务代价，也有

$$\mathbb{E}[G^{(\text{int})}] \leq 235,000 - 0.97 \times 1,000,000 = -735,000 < 0.$$

敌手在获得路径信息后，既可能只选择其中一种利用方式，也可能在不同时段或不同目标上尝试多种利用方式。因此，总体净收益不宜再写成权重和为 1 的凸组合，而更适合采用上界形式进行估计。记路径泄露所诱发的总体净收益为 G_{leak} ，则有

$$G_{\text{leak}} \leq G^{(\text{blk})} + G^{(\text{imp})} + G^{(\text{int})}.$$

对两侧取期望可得

$$\mathbb{E}[G_{\text{leak}}] \leq \mathbb{E}[G^{(\text{blk})}] + \mathbb{E}[G^{(\text{imp})}] + \mathbb{E}[G^{(\text{int})}].$$

代入前述三项上界，可得

$$\mathbb{E}[G_{\text{leak}}] \leq -968,925 - 674,000 - 735,000 = -2,377,925 < 0.$$

由此可见，在仅采用公开可核查的收益代理、检测率代理，并统一取对攻击者更有

利的 \$1,000,000 法律成本口径，且暂不计入额外业务代价的保守前提下，路径泄露所支撑的总体攻击收益上界已经严格为负。这表明，路径泄露虽然能够降低敌手对目标、注入位置与潜伏窗口的搜索成本，从而为黑洞化、冒充响应与中间拦截提供战术价值，但在现实检测与归责体系下，其经济吸引力仍然受到显著抑制。相比之下，单次审计结果造假更多体现为结论层面的真实性破坏，其外部可见性通常弱于大规模引流、伪装或截取行为。因此，本文仍将密码学防御重点放在查询结果的真实性与不可抵赖性上，而将路径泄露视为一种可通过敌手能力分解、检测能力代理与统一法律成本口径进行量化分析的隐私风险。

2.4 本章小结

本章围绕 AegisPath 的系统概述展开。首先给出了 AegisPath 的系统架构，从功能角色划分与系统执行阶段两个方面说明了各参与实体在生成期与查询期中的职责分工。随后，通过一个具体示例介绍了 AegisPath 的工作流程，展示了系统如何完成从本地配置输入、秘密共享、见证生成、承诺聚合、版本根发布到查询期审计与版本更新的全过程。最后，从威胁模型与隐私风险两个层面对系统的安全性进行了分析，明确了生成期与查询期分别面向的攻击者能力以及系统在不同阶段需要满足的安全目标。

第3章 基于门限秘密共享的数据平面语义生成

在传统的平面验证研究中，验证者通常需要集中获取网络设备的转发表、过滤规则与策略配置，并在此基础上统一建模与分析。该范式在单管理域场景下具有较好的可操作性，但在多管理域环境中往往难以成立：一方面，多管理域数据平面语义的形成依赖多个自治系统的本地配置共同作用；另一方面，这些配置与策略通常包含运营细节与安全策略，自治系统一般不愿以明文形式提交给外部集中式验证器。为解决这一矛盾，本文在本章引入基于门限秘密共享的隐私协作计算方法，将原本集中收集并计算的数据平面语义生成过程改造为分布式输入并联合计算的生成过程，使各参与方在不暴露本域数据面配置细节的前提下，共同得到覆盖对象对应的数据平面语义见证，为后续章节中的承诺构造与零知识审计提供输入基础。

本章聚焦生成期的离线语义计算问题，即在隐私约束下如何为覆盖对象集合中的目标对象生成可用于后续审计的数据平面语义见证。具体而言，（§3.1）给出多管理域数据平面语义生成的问题定义、覆盖对象与二元组级见证的形式化描述，并明确生成期的输入输出与系统边界；（§3.2）介绍基于门限秘密共享与数据无关计算的整体生成流程，说明参与方分工以及逐对象语义生成的核心过程；最后，（§3.3）对本章方法的安全性进行分析。

3.1 问题定义与生成目标

本节首先明确本文在生成期所处理的对象、输入输出以及语义边界。与集中式数据平面验证器直接读取明文配置并在本地完成计算不同，本文关注的是多管理域场景下的隐私协作生成问题：多个自治系统共同参与计算，但任何一方都不应获得其他自治系统的明文数据面配置。为此，本文将生成期的目标表述为：在门限秘密共享与数据无关执行约束下，面向给定覆盖对象集合，联合生成可用于后续审计的数据平面语义见证。

3.1.1 多管理域数据平面语义生成问题定义

设某一生成版本对应的参与自治系统集合为 $AS_set(v) = \{AS_1, \dots, AS_n\}$ ，其中 v 表示版本标识。每个自治系统 AS_i 持有本域与多管理域转发相关的数据面配

置子集, 记为 C_i 。在本文的语义范围内, C_i 至少包含前缀级转发信息以及边界处会影响转发结果的策略信息; 这些配置共同决定了多管理域数据平面的实际转发行为, 但其明文内容不对其他自治系统或外部验证者公开。

在给定覆盖对象集合 $\mathcal{Q}(v)$ 的条件下, 本文希望对每个目标对象 $(p, s) \in \mathcal{Q}(v)$ 生成一个数据平面语义见证, 记为 $W_{v,p,s}$ 。该见证用于刻画“从起始自治系统 s 出发、目的地址属于前缀 p 的流量集合”在版本 v 下的有效多管理域转发语义, 并作为后续承诺与零知识证明阶段的私有输入。据此, 本文将本章研究的问题形式化为如下映射关系: 在隐私约束下, 联合计算

$$(\mathcal{C}_1, \dots, \mathcal{C}_n, \mathcal{Q}(v)) \mapsto \{W_{v,p,s} \mid (p, s) \in \mathcal{Q}(v)\}, \quad (3-1)$$

其中计算过程满足两项基本要求: 其一, 输出见证在语义上与明文数据面配置所定义的二元组级有效转发行为一致; 其二, 除输出所必然泄露的信息外, 任一参与方不应从协议执行过程中获得其他参与方的明文配置细节。

3.1.2 覆盖对象与生成期输入输出

本文以起始自治系统与目的前缀构成的二元组作为生成期处理的基础对象。形式上, 对任一前缀 $p = (a_p, \ell_p)$, 其中 a_p 表示前缀地址编码, ℓ_p 表示前缀长度, 且给定起始自治系统 s , 其对应的基础流量集合记为:

$$\mathcal{F}(p, s) = \{x \mid x \text{ 的起始自治系统为 } s, \text{ 且目的地址属于前缀 } p\}. \quad (3-2)$$

因此, 二元组 (p, s) 可视为生成期的原始覆盖对象, 用于刻画从自治系统 s 出发、目的地址属于前缀 p 的流量范围。

在未考虑更具体规则、边界过滤或策略改写之前, 原始对象 (p, s) 可由单一前缀集合表示。然而, 在真实数据面中, 同一基础流量集合 $\mathcal{F}(p, s)$ 内部未必总能由单一条转发语义完整刻画。特别是当更具体前缀、边界过滤、黑洞处置或策略改写等规则生效时, $\mathcal{F}(p, s)$ 内部可能出现不同子集在实际数据面上经历不同转发路径或处置结果的情况。为使见证对象与实际语义保持一致, 生成期允许对原始覆盖对象 (p, s) 对应的流量空间进行逐步细化。

设原始前缀 p 在生成期经过规则驱动流量空间切割后, 被细化为一组更小的前缀片段 $\{p^{(1)}, p^{(2)}, \dots, p^{(m)}\}$ 。则这些细化前缀在同一起始自治系统 s 下对应的

流量集合满足：

$$\mathcal{F}(p^{(k)}, s) \subseteq \mathcal{F}(p, s), \quad k = 1, \dots, m, \quad (3-3)$$

并且各细化流量集合两两不交，其并集仍等于原始基础流量集合。于是，生成期不再为原始对象 (p, s) 直接生成单一语义结果，而是先将其细化为若干语义一致的细化二元组对象 $(p^{(k)}, s)$ ，再分别计算这些细化对象的有效多管理域转发语义。

覆盖对象集合 $\mathcal{Q}(v)$ 用于限定版本 v 中需要执行语义生成的原始对象范围。其构造方式来源于 RouteViews 组织提供的控制面路径宣告^[17]。在此基础上，生成期的输入可分为两类。第一类是公共输入，包括版本标识 v 、参与自治系统集合 $AS_set(v)$ 、原始覆盖对象集合 $\mathcal{Q}(v)$ 以及与编码和执行相关的公共参数。第二类是私有输入，即各自治系统持有的本域数据面配置子集 $\mathcal{C}_i$ ；这些配置在协议执行前经过整数化与秘密共享处理，以便进入后续的隐私协作计算流程。

生成期的输出是由原始对象细化得到的细化二元组对象集合及其对应见证集合。对任意原始对象 $(p, s) \in \mathcal{Q}(v)$ ，生成期输出其对应的细化对象集合 $\tilde{\mathcal{Q}}_{v,p,s}$ 及见证集合 $\mathcal{W}_{v,p,s}$ 。于是，版本 v 下的整体输出可写为：

$$\tilde{\mathcal{Q}}(v) = \bigcup_{(p,s) \in \mathcal{Q}(v)} \tilde{\mathcal{Q}}_{v,p,s}, \quad \mathcal{W}_v = \{\mathcal{W}_{v,p',s} \mid (p', s) \in \tilde{\mathcal{Q}}(v)\}. \quad (3-4)$$

其中， $\tilde{\mathcal{Q}}(v)$ 表示版本 v 下经由流量空间切割后得到的细化二元组对象全集， $\mathcal{W}_v$ 表示与这些细化对象一一对应的语义见证集合。该输出集合在逻辑上属于版本 v 的离线语义结果，不直接对外公开明文，而是在后续章节中进一步组织为承诺叶子并聚合为版本根。后续章节中的承诺、版本化与审计查询，均以这些细化二元组对象及其对应见证为直接输入。

3.2 基于秘密共享的多管理域数据平面语义生成

本节讨论 AegisPath 如何在秘密分享框架与数据无关执行约束下，实现多管理域数据平面语义见证的联合生成。离线生成期以覆盖对象为基本处理单元，将各自治系统的本域异构配置统一编码为有限域上的定长标量，并在秘密分享域中协同生成版本 v 下的细化对象集合及其语义见证。对任意原始覆盖对象 $(p, s) \in \mathcal{Q}(v)$ ，系统依据数据面规则所诱导的语义分化对其进行逐步细化，并在多管理域状态推进过程中生成与细化对象 $(p^{(k)}, s)$ 对应的见证 $\mathcal{W}_{v,p^{(k)},s}$ 。本节以下内容依次介绍生成期的输入编码与秘密分片、生成期全局调度流程、逐对象语义推演过程，以及

最终的版本化见证封装方式。

3.2.1 异构配置的标量化编码与门限分片

多管理域数据面配置具有异构性，直接在隐私计算中处理原始配置文本既不利于统一语义，也不利于满足数据无关执行对定长结构的要求。本文因此在本地预处理阶段引入整数化中间表示，将参与计算的关键字段映射为有限域 $\mathbb{F}_p$ 上的标量，并组织为定长数组后再进行门限分片。

IPv4 前缀与地址编码。本文将 IPv4 地址视为无符号 32 位整数并嵌入到 $\mathbb{F}_p$ (取 $p > 2^{32}$ 的素数域)。对前缀 $p = (a, \ell)$ ，其中 $a \in [0, 2^{32})$ 为前缀地址的整数表示， $\ell \in \{0, \dots, 32\}$ 为前缀长度。为保证匹配语义一致性，前缀地址在输入阶段执行预掩码化

$$a \leftarrow a \wedge M[\ell],$$

其中 $M[\ell]$ 为按长度索引的公开掩码常量表。目标地址或对象当前对应的前缀代表地址同样以 32 位整数编码进入协议，用于后续匹配算子中的按掩码取位与等值比较。

动作与策略字段编码。转发动作与边界处置以常量编码进入规则条目，例如用常量区分 **FORWARD** 与 **DROP**，并以额外标记位表示可实现子集中的策略覆盖、黑洞或重定向等效果。对转发表条目而言，除 (a, ℓ) 外，至少包含下一跳自治系统标识（或索引）、动作标记以及必要的策略字段。对过滤、黑洞、策略路由等边界策略，本文统一抽象为对给定前缀返回 **DROP** 或覆盖下一跳的谓词式接口，并同样以整数化字段进入协议计算。完成编码后，各自治系统的本地规则集合按自治系统粒度组织，并在输入规范化阶段整理为固定长度的规则块，为后续逐对象语义推演提供统一的规则输入格式。

覆盖对象与起始点编码。由于本文的基本验证对象是二元组 (p, s) ，因此除目前的前缀相关字段外，起始自治系统标识 s 也需要以整数形式进入生成流程。实现上，可将 s 统一编码为公开索引或有限域中的整数标量，并作为路径推进阶段的初始状态输入，用以确定对应原始见证的起始位置。

完成上述整数化与组织后，各自治系统对本地数组执行 (t, n) -Shamir 秘密分享，将每个标量拆分为份额并分发给参与计算的执行实体。对任一字段值 x ，记其秘密共享表示为 $\llbracket x \rrbracket$ 。分发完成后，联合计算仅在 $\llbracket \cdot \rrbracket$ 上进行，计算节点无法通过本地持有的碎片化数据还原任何有效的原始配置信息。至此，各 AS 的私有配置已

被重构为分布式的秘密份额。这些份额在消除原始数据敏感特征的同时，也为后续在秘密分享域上执行统一的代数运算提供了标准化的输入格式。

3.2.2 生成期全局调度流程

生成期以原始覆盖对象集合 $\mathcal{Q}(v)$ 为输入，在秘密分享域中逐步生成版本 v 下的细化对象集合及其语义见证。整体执行上，生成期采用规则预处理、对象装表、统一调度与槽位回写相结合的组织方式。原始对象与细化对象在全局对象槽表中具有统一的状态表示，因此对象细化、状态继承与逐 AS 推进都可在同一套固定内存布局上完成。算法 1 给出了生成期全局调度流程：系统首先按自治系统粒度完成规则排序并生成固定长度规则块，随后将原始覆盖对象写入全局对象槽表，再在统一调度循环中依次处理所有活跃对象槽位；若对象在当前规则作用下触发细化，则父对象槽位失活，并在空闲槽位中依次写入新生成的细化对象。整个调度循环的总轮数由公开参数 $MaxSteps$ 给定，其中 $MaxSteps$ 可依据控制面观测路径的最大长度并结合一个固定缓冲跳数预先确定；对于较早终止的对象，后续轮次仅执行状态保持而不再产生新的有效语义更新。

表 3.1 全局对象槽表中的主要状态字段

字段	含义
$pref_addr[o]$	槽位 o 当前对象对应的前缀地址编码
$pref_len[o]$	槽位 o 当前对象对应的前缀长度
$src_as[o]$	对象的起始自治系统标识
$cur_as[o]$	对象当前推进到的自治系统标识
$scan_ptr[o]$	对象在当前自治系统规则块中的扫描前沿位置
$path_ptr[o]$	对象级路径见证当前已写入的位置
$active[o]$	槽位 o 是否为当前有效对象，取值为 1 或 0
$done[o]$	对象是否已经终止，取值为 1 或 0

设全局对象槽表记为 $\mathbf{T}[1..O_{\max}]$ 。对任一槽位 o ，系统维护的主要状态字段见表 3.1。这些字段共同刻画了对象当前对应的前缀片段、推进位置、规则扫描进度、路径写入进度以及活跃状态。原始对象与后续生成的细化对象在槽表中的表示保持一致，区别仅体现在字段取值上。

生成期开始前,系统先按自治系统粒度对本地规则集合执行一次统一排序。设自治系统 u 的本地规则集合编码后记为 $\mathcal{C}_u^{IR}$, 则系统依据统一优先级语义对其执行一次数据无关排序, 并填充为固定长度规则块

$$\mathbf{R}_u = \langle \mathcal{C}_{u,1}^{IR}, \mathcal{C}_{u,2}^{IR}, \dots, \mathcal{C}_{u,\text{FIB_MAX}}^{IR} \rangle.$$

其中, 规则块内部按前缀长度降序排列; 若前缀长度相同, 则再按预先约定的稳定次序打破并列。排序仅在生成期开始前执行一次, 用于确保后续访问到的规则顺序由协议统一定义。

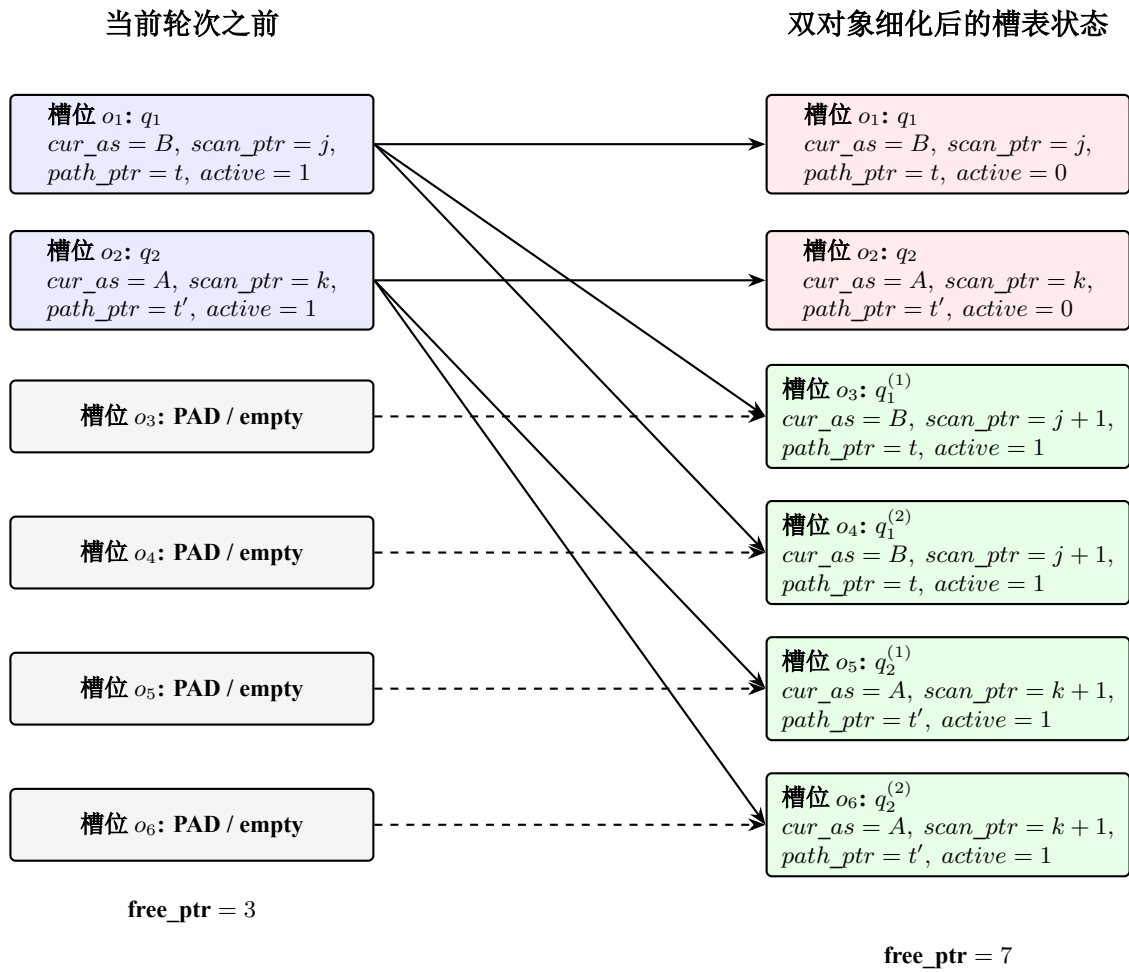


图 3.1 全局对象槽表中多个覆盖对象并发细化的示意图

随后, 系统将原始覆盖对象集合 $\mathcal{Q}(v)$ 依次写入全局对象槽表的前若干活跃槽位, 并将其余槽位填充为 PAD。对于每个新写入的原始对象 (p, s) , 系统将其起始自治系统与当前自治系统均初始化为 s , 规则扫描前沿初始化为当前自治系统规则块的起始位置, 路径写入位置初始化为 0, 活跃位置为 1, 终止位置为 0。与此同

算法 1: 生成期全局调度流程

输入: 原始覆盖对象集合 $\mathcal{Q}(v)$; 各自治系统本地规则集合 $\{\mathcal{C}_u^{IR}\}_{u \in AS_set(v)}$; 公开参数 $(O_{\max}, MaxSteps)$

输出: 细化对象集合 $\tilde{\mathcal{Q}}(v)$; 见证集合 $\mathcal{W}_v$

```

1 foreach  $u \in AS\_set(v)$  do
2    $\mathbf{R}_u \leftarrow \text{SortRules}(\mathcal{C}_u^{IR});$ 
3 end
4  $\mathbf{T}[1..O_{\max}] \leftarrow \text{PAD};$ 
5 for  $o \leftarrow 1$  to  $|\mathcal{Q}(v)|$  do
6    $\mathbf{T}[o] \leftarrow \text{InitObject}(\mathcal{Q}(v)[o]);$ 
7 end
8  $\text{free\_ptr} \leftarrow |\mathcal{Q}(v)| + 1;$ 
9 for  $t \leftarrow 0$  to  $MaxSteps - 1$  do
10   for  $o \leftarrow 1$  to  $O_{\max}$  do
11     if  $\mathbf{T}[o].active = 1 \wedge \mathbf{T}[o].done = 0$  then
12        $u \leftarrow \mathbf{T}[o].cur\_as;$ 
13        $(\mathcal{S}, \mathbf{st}) \leftarrow \text{PerObjectDeduce}(\mathbf{T}[o], \mathbf{R}_u);$ 
14       if  $|\mathcal{S}| = 0$  then
15          $\mathbf{T}[o] \leftarrow \mathbf{st};$ 
16       end
17       else
18          $\mathbf{T}[o].active \leftarrow 0;$ 
19         for  $i \leftarrow 1$  to  $|\mathcal{S}|$  do
20            $\mathbf{T}[\text{free\_ptr}] \leftarrow \text{SpawnChild}(\mathbf{st}, \mathcal{S}[i]);$ 
21            $\mathbf{T}[\text{free\_ptr}].active \leftarrow 1;$ 
22            $\text{free\_ptr} \leftarrow \text{free\_ptr} + 1;$ 
23         end
24       end
25     end
26   end
27 end
28  $(\tilde{\mathcal{Q}}(v), \mathcal{W}_v) \leftarrow \text{ExtractActive}(\mathbf{T});$ 
29 return  $(\tilde{\mathcal{Q}}(v), \mathcal{W}_v);$ 

```

时，系统维护一个空闲槽位指针 free_ptr ，用于指示当前可写入新细化对象的最小空闲槽位索引。

在全局调度循环中，系统依次扫描所有满足 $\text{active} = 1$ 且 $\text{done} = 0$ 的对象槽位。对任一对象 o ，系统首先依据其 $\text{cur_as}[o]$ 读取对应自治系统的已排序规则块 $\mathbf{R}_u$ ，再调用逐对象语义推演内核完成当前跳的局部流量空间切割、规则命中消解、边界动作作用与路径状态更新。若对象 o 在当前规则作用下触发细化，父对象槽位保持原索引不变并将 $\text{active}[o]$ 置为 0；随后，系统从 free_ptr 开始，依次为每个新生成的细化对象分配空闲槽位。新对象继承父对象的当前自治系统、路径写入位置与终止状态，并将规则扫描前沿更新为当前规则之后的位置。分配完成后， free_ptr 向后移动到下一处空闲槽位。这样，细化对象即可像普通对象一样继续参与当前自治系统剩余规则的扫描，并在本跳处理完成后进入下一跳推进。生成结束后，所有仍保持活跃状态的对象槽位共同构成版本 v 下的细化对象集合及其语义见证集

合。

图 3.1 给出了算法 1 在同一轮调度中维护多个对象细化结果的示意。设初始时两个原始覆盖对象 $q_1 = (p_1, s_1)$ 与 $q_2 = (p_2, s_2)$ 分别占据槽位 o_1 与 o_2 。当前轮调度开始时，空闲槽位指针满足 $\text{free_ptr} = 3$ 。若对象 q_1 与 q_2 在各自当前自治系统的规则作用下都触发细化，则系统先将父对象槽位 o_1, o_2 的 **active** 位置为 0，再从 free_ptr 指向的位置开始，依次写入新生成的细化对象。新对象继承父对象的 **cur_as**、**path_ptr** 以及终止状态，并将 **scan_ptr** 更新为当前规则之后的位置。随着新对象完成写入，空闲槽位指针继续后移，从而保证后续细化对象能够写入尚未占用的槽位。

在实现上，本文基于 MP-SPDZ 框架^[47] 对全局对象槽表进行组织。考虑到对象状态需要在调度循环与并行处理单元之间共享，适合采用 MP-SPDZ 提供的内存型容器对槽表字段进行统一存储，以 Array 这个适合存储密文对象的定长内存数组分别保存 **cur_as**、**scan_ptr**、**path_ptr**、**active** 与 **done** 等状态字段。这样，每个槽位索引 o 都对应全局数组中的一组固定位置；父对象失活时仅需将对应位置的 **active**[o] 置为 0，而新对象则写入由 free_ptr 指定的后续空闲位置。由于这些状态字段在内存中按槽位索引统一维护，原始对象与细化对象能够在同一张全局对象槽表中持续演化，并继续参与后续调度与推进过程。

3.2.3 逐对象语义推演

本节给出覆盖对象在全局调度循环中的逐对象语义更新方法。本节给出覆盖对象在全局调度循环中的逐对象语义更新方法。对任一对象槽位 o ，其状态由前缀片段、起始自治系统、当前自治系统、规则扫描前沿、路径写入位置以及活跃位和终止位共同刻画。算法 2 展开的是全局调度循环中单个对象在当前轮次、当前自治系统上的局部语义更新过程。给定当前对象状态 st_o 与当前自治系统对应的已排序规则块 $\mathbf{R}_u$ ，系统首先在该自治系统上对当前活跃对象执行局部流量空间切割，再对切割后保留下来的对象执行规则命中消解、边界动作作用与状态推进，并将更新后的对象状态返回给全局对象槽表。其中 $B_{\max}$ 参数表示单个对象在当前自治系统上执行局部细化时允许使用的临时子槽位上界，用于容纳当前轮次中由规则切割产生的多个细化结果；该上界仅作用于单对象内部的局部语义更新过程，与全局对象槽表的总槽位上界 $O_{\max}$ 相互独立。若局部切割产生多个新的细化对象，则这些对象先在 $B_{\max}$ 个临时子槽位中完成整理，再在当前轮次结束后写回全局对象

算法 2: 当前自治系统上的逐对象语义更新过程

输入: 当前对象状态 $\mathbf{st}_o$; 当前自治系统的已排序规则块 $\llbracket \mathbf{R}_u \rrbracket$; 公开参数 $(L, B_{\max}, \{M[\ell]\})$
输出: 新生成的细化对象集合 $\mathcal{S}$; 更新后的对象状态 $\mathbf{st}'_o$

- 1 从 $\mathbf{st}_o$ 中读取 $\llbracket \Pi \rrbracket$, $\llbracket act \rrbracket$, $\llbracket cur \rrbracket$, $\llbracket done \rrbracket$, $\llbracket path_ptr \rrbracket$ 及当前路径缓冲区状态;
- 2 **for** $b \leftarrow 1$ **to** $B_{\max}$ **do**
- 3 $\llbracket u^{(b)} \rrbracket \leftarrow \llbracket cur^{(b)} \rrbracket$;
- 4 $(\llbracket \Pi^*[b] \rrbracket, \llbracket act^*[b] \rrbracket) \leftarrow \mathbf{RefineAtAS}(\llbracket \Pi[b] \rrbracket, \llbracket act[b] \rrbracket, \llbracket u^{(b)} \rrbracket, \llbracket \mathbf{R}_u \rrbracket)$;
- 5 **end**
- 6 $(\llbracket \Pi \rrbracket, \llbracket act \rrbracket) \leftarrow \mathbf{CompactAndPad}(\llbracket \Pi^* \rrbracket, \llbracket act^* \rrbracket, B_{\max})$;
- 7 **for** $b \leftarrow 1$ **to $B_{\max}$ **do****
- 8 $\llbracket u^{(b)} \rrbracket \leftarrow \llbracket cur^{(b)} \rrbracket$;
- 9 $(\llbracket nh^{(b)} \rrbracket, \llbracket found^{(b)} \rrbracket) \leftarrow \mathbf{ResolveLPM}(\llbracket \Pi[b] \rrbracket, \llbracket u^{(b)} \rrbracket, \llbracket \mathbf{R}_u \rrbracket)$;
- 10 $(\llbracket drop^{(b)} \rrbracket, \llbracket nh_{\text{eff}}^{(b)} \rrbracket) \leftarrow \mathbf{ApplyBoundaryPolicy}(\llbracket u^{(b)} \rrbracket, \llbracket \Pi[b] \rrbracket, \llbracket nh^{(b)} \rrbracket)$;
- 11 $\llbracket \mathbf{s}^{(b)}[\llbracket path_ptr^{(b)} \rrbracket] \rrbracket \leftarrow \mathbf{Sel}(\llbracket act[b] \rrbracket \cdot (1 - \llbracket done^{(b)} \rrbracket), \llbracket u^{(b)} \rrbracket, \llbracket \mathbf{s}^{(b)}[\llbracket path_ptr^{(b)} \rrbracket] \rrbracket)$;
- 12 $\llbracket done^{(b)} \rrbracket \leftarrow \llbracket done^{(b)} \rrbracket \vee \llbracket drop^{(b)} \rrbracket \vee \mathbf{IsTerminal}(\llbracket u^{(b)} \rrbracket, \llbracket \Pi[b] \rrbracket)$;
- 13 $\llbracket cur^{(b)} \rrbracket \leftarrow \mathbf{Sel}(1 - \llbracket done^{(b)} \rrbracket, \llbracket nh_{\text{eff}}^{(b)} \rrbracket, \llbracket u^{(b)} \rrbracket)$;
- 14 $\llbracket path_ptr^{(b)} \rrbracket \leftarrow \mathbf{Sel}(\llbracket act[b] \rrbracket \cdot (1 - \llbracket done^{(b)} \rrbracket), \llbracket path_ptr^{(b)} \rrbracket + 1, \llbracket path_ptr^{(b)} \rrbracket)$;
- 15 **end**
- 16 $\mathcal{S} \leftarrow \mathbf{CollectChildren}(\llbracket \Pi \rrbracket, \llbracket act \rrbracket, \llbracket cur \rrbracket, \llbracket done \rrbracket, \llbracket \mathbf{s} \rrbracket, \llbracket path_ptr \rrbracket)$;
- 17 $\mathbf{st}'_o \leftarrow \mathbf{UpdateState}(\llbracket \Pi \rrbracket, \llbracket act \rrbracket, \llbracket cur \rrbracket, \llbracket done \rrbracket, \llbracket \mathbf{s} \rrbracket, \llbracket path_ptr \rrbracket)$;
- 18 **return** $(\mathcal{S}, \mathbf{st}'_o)$;

槽表，并在后续调度中继续参与剩余自治系统上的语义推演。

整个过程中，协议始终保持固定的局部槽位上界、固定的规则扫描顺序与固定的全局推进轮数，从而满足数据无关执行的要求。对任一活跃对象而言，当前轮次的局部细化只针对其当前自治系统对应的规则块进行；若细化产生新的子对象，这些子对象继承父对象在当前轮次开始时的公共执行上下文，并在后续调度中继续参与剩余自治系统上的状态推进。

前缀关系判定与命中判定函数。为了支持局部流量空间切割与后续规则命中消解，本文区分两类判定原语。第一类用于 **RefineAtAS** 函数中的前缀空间关系判断，负责识别当前子空间是否需要进一步细化；第二类用于 **ResolveLPM** 函数中的规则命中判断，负责在细化完成后确定对象在当前 AS 的生效规则。

设当前子空间表示为 $\pi = (a_\pi, \ell_\pi)$ ，候选规则前缀表示为 $r = (a_r, \ell_r)$ ，其中地址编码均为固定宽度整数， $M[\ell]$ 表示长度为 ℓ 的公开前缀掩码。对于局部细化阶段，首先定义前缀包含关系为

$$\mathbf{sub}(\pi, r) = (\ell_\pi \geq \ell_r) \cdot ((a_\pi \wedge M[\ell_r]) = a_r). \quad (3-5)$$

该谓词表示当前子空间 π 是否整体落在规则前缀 r 的覆盖范围内。交换参数次序即可得到 $\mathbf{sub}(r, \pi)$ 。

进一步, 记 $\ell_{\min} = \min(\ell_{\pi}, \ell_r)$, 则二者是否存在交集可表示为

$$\text{overlap}(\pi, r) = ((a_{\pi} \wedge M[\ell_{\min}]) = (a_r \wedge M[\ell_{\min}])). \quad (3-6)$$

在此基础上, 定义局部切割触发谓词为

$$\text{split}(\pi, r) = \text{overlap}(\pi, r) \cdot (1 - \text{sub}(\pi, r)). \quad (3-7)$$

当 $\text{split}(\pi, r) = 1$ 时, 说明规则 r 仅覆盖当前子空间 π 的一部分, **RefineAtAS** 函数需要触发进一步细化; 若 $\text{overlap}(\pi, r) = 0$, 则规则与当前子空间无交集; 若 $\text{sub}(\pi, r) = 1$, 则当前子空间整体落在规则范围内, 当前轮无需继续切割。

对于规则命中消解阶段, 本文在 **ResolveLPM** 函数中使用基础命中谓词

$$\text{match}(\pi, r) = ((a_{\pi} \wedge M[\ell_r]) = a_r). \quad (3-8)$$

该谓词用于判断某一已经完成局部细化的对象是否命中当前候选规则, 并在已排序规则块上配合首个命中保留策略实现最长前缀匹配。随后, **ApplyBoundaryPolicy** 函数再对 **ResolveLPM** 函数输出的生效动作施加边界策略作用。上述计算仅涉及按公开掩码的位与运算、秘密比较与秘密相等判断, 因此可以在固定轮数下实现, 不引入数据相关分支。

局部细化函数。函数 **RefineAtAS** 对应每一跳开始时的局部流量空间切割。给定当前活跃对象、当前自治系统 u 以及 u 对应的已排序规则块 $\mathbf{R}_u$, 系统按照规则块中的既定顺序逐项扫描规则, 并对当前活跃对象执行前缀关系判定。若某条规则与当前对象无交集, 则保持该对象不变; 若当前对象整体落在该规则覆盖范围内, 则该对象在当前规则下保持语义一致, 也不触发切割; 若规则仅覆盖其内部一部分, 则系统调用 **SplitSpace** 函数将该对象细化为若干更小的语义一致子对象, 并以这些新生成的子对象替换原对象继续参与后续规则扫描。

局部细化是在同一自治系统的规则块内部逐轮推进的。某一轮由当前规则生成的子对象不会退出本轮处理流程, 而是直接作为新的活跃对象继续接受该自治系统剩余规则的扫描; 若后续规则仍只覆盖其中某个子对象的一部分, 则该子对象会再次被细化, 并继续进入再下一轮规则判定。随着规则块扫描逐步推进, 活跃对象集合会不断演化, 但每个新子对象都来源于其父对象的前缀子集, 因此自动继承父对象对已扫描规则的一致性, 只需继续接受剩余规则的处理即可。经过当前自治系统全部规则扫描后, 最终保留下来的所有活跃对象都已对规则块 $\mathbf{R}_u$ 保

持语义一致，随后再进入 **ResolveLPM** 与 **ApplyBoundaryPolicy** 函数所对应的后续处理阶段。

如图 3.2 所示，局部细化会随着规则扫描持续发生。以对象 $(134.34.0.0/24, s)$ 为例，设当前自治系统的规则块中包含两条相关规则：一条对前缀 $134.34.0.0/27$ 生效的 **blackhole** 规则，以及一条对前缀 $134.34.0.0/25$ 生效的 **ACL** 规则。按照前缀长度降序的扫描顺序，对象 $(134.34.0.0/24, s)$ 进入该自治系统后，先在 **blackhole** 规则 $134.34.0.0/27$ 的作用下被细化为 $(134.34.0.0/27, s)$ 、 $(134.34.0.32/27, s)$ 、 $(134.34.0.64/26, s)$ 与 $(134.34.0.128/25, s)$ 。其中， $(134.34.0.0/27, s)$ 完全命中 **blackhole** 规则， $(134.34.0.32/27, s)$ 、 $(134.34.0.64/26, s)$ 与 $(134.34.0.128/25, s)$ 均不命中该规则。

随后，这些新生成的对象继续参与后续 **ACL** 规则 $134.34.0.0/25$ 的扫描。对象 $(134.34.0.0/27, s)$ 由于整体落在前缀 $134.34.0.0/25$ 的覆盖范围内，因此命中 **ACL** 规则；对象 $(134.34.0.32/27, s)$ 与 $(134.34.0.64/26, s)$ 也均落在该前缀范围内，因此同样命中 **ACL** 规则；对象 $(134.34.0.128/25, s)$ 与该 **ACL** 规则无交集，因此保持不变。经过当前自治系统全部规则扫描后，最终活跃对象集合为 $(134.34.0.0/27, s)$ 、 $(134.34.0.32/27, s)$ 、 $(134.34.0.64/26, s)$ 与 $(134.34.0.128/25, s)$ ，其中前三个对象命中 **ACL** 规则，首个对象同时命中更具体的 **blackhole** 规则，最后一个对象不命中这两条规则。它们共同进入后续处理阶段的规则命中消解与路径推进过程。

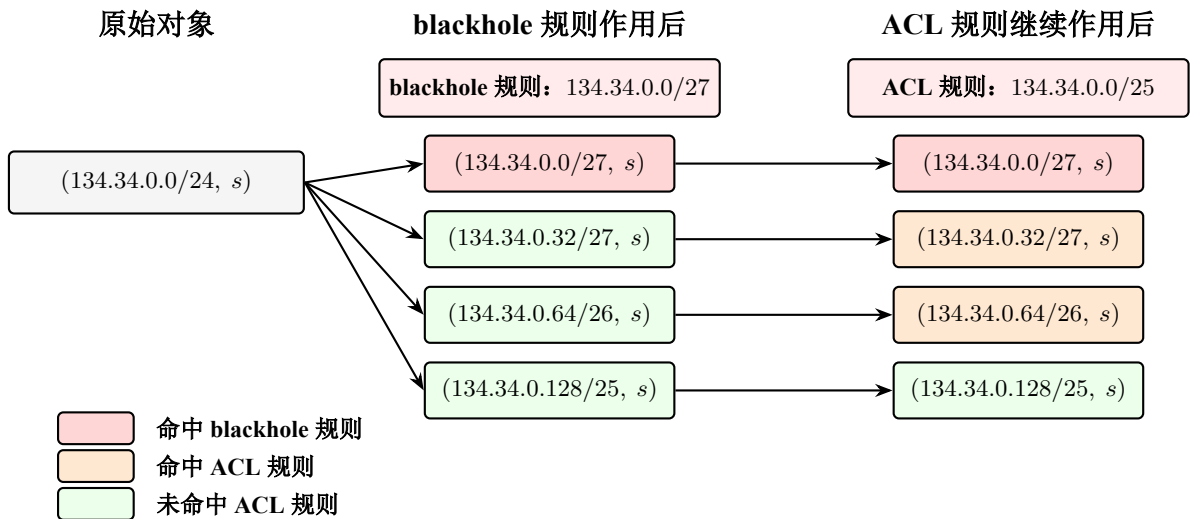


图 3.2 局部细化函数中规则扫描示例图

优先级消解函数。函数 **ResolveLPM** 在当前自治系统 u 的已排序规则块 $\mathbf{R}_u$ 上执行最长前缀匹配。由于 $\mathbf{R}_u$ 已按优先级统一排序，系统只需扫描完整个规则块，

并通过秘密比特 $\llbracket found \rrbracket$ 锁定首个命中条目。若第 j 条规则命中且此前尚未锁定其他命中规则，则置

$$\llbracket take_j \rrbracket = \llbracket match_j \rrbracket \cdot (1 - \llbracket found \rrbracket). \quad (3-9)$$

由此即可在不泄露命中位置的前提下实现与明文最长前缀匹配一致的优先级消解。

路径推进函数。在完成当前自治系统上的局部细化后，系统对每个当前活跃对象分别执行一次新的规则命中消解与动作计算。设某活跃对象在当前时刻的执行上下文包括当前自治系统 $\llbracket cur^{(b)} \rrbracket$ 、起始自治系统 s 、前缀片段 $\llbracket \Pi[b] \rrbracket$ 、已写入的路径前缀 $\llbracket s^{(b)} \rrbracket$ 、路径写入位置 $\llbracket path_ptr^{(b)} \rrbracket$ 以及终止位 $\llbracket done^{(b)} \rrbracket$ 。若该对象在局部细化阶段被拆分为多个子对象，则这些子对象继承上述执行上下文中的公共部分，包括起始自治系统、当前自治系统、路径前缀、路径写入位置与终止状态；与此同时，它们以各自的细化前缀片段作为新的对象状态，在当前自治系统上重新调用 **ResolveLPM** 函数与 **ApplyBoundaryPolicy** 函数，独立计算该自治系统上的有效动作。

对任一当前活跃对象，系统先通过 **ResolveLPM** 函数在当前自治系统的已排序规则块上确定生效条目，再通过 **ApplyBoundaryPolicy** 函数将该条目的动作语义作用到状态上，例如将下一跳修正为有效出口、黑洞或丢弃；随后，将当前自治系统写入见证序列，并依据终止位与生效动作更新对象的当前状态。对于任一细化对象 $(p^{(k)}, s)$ ，路径推进都从其继承得到的当前自治系统状态继续执行：若当前规则或边界动作导出 **DROP**，则函数置终止标记；若当前自治系统已达到目标语义终点，则函数保持终止状态；否则，将新计算得到的有效下一跳写入当前状态，以供后续轮次继续调度。

为了保持数据无关执行，本文引入秘密共享终止位 $\llbracket done \rrbracket$ 控制后续更新。当前自治系统状态通过选择器更新为

$$\llbracket cur_{t+1} \rrbracket = \mathbf{Sel}(1 - \llbracket done_t \rrbracket, \llbracket nh_t \rrbracket, \llbracket cur_t \rrbracket). \quad (3-10)$$

相应地，路径写入位置也通过选择器进行更新：当对象在当前轮次仍处于活跃且未终止状态时， $\llbracket path_ptr \rrbracket$ 前移一位；否则保持原值不变。由此，即便某一对象在较早轮次已经到达终止状态，后续轮次中其状态也仅执行保持型更新，而不会改变已经确定的语义结果。

当前自治系统上的状态更新完成后，算法进一步调用 **UpdateState** 函数，将当

前轮次得到的前缀片段、活跃位、当前自治系统、终止位、路径缓冲区与路径写入位置整理为统一的对象状态表示，并写回全局对象槽表中对应的父对象槽位。若局部细化阶段产生了多个新的有效对象，则算法调用 **CollectChildren** 函数，从当前轮次保留下来的细化对象中提取需要继续参与后续调度的子对象集合 $\mathcal{S}$ 。这些子对象在逻辑上继承父对象的公共执行上下文，在实现上则被组织为统一的状态单元，供全局调度算法分配空闲槽位并继续推进。因此，**UpdateState** 负责维护父对象在当前轮次结束后的状态一致性，**CollectChildren** 负责生成需要写入全局对象槽表的新对象集合；二者共同构成逐对象语义更新过程与全局调度流程之间的接口。

3.2.4 版本化见证的结构化封装

在完成逐对象的隐私推演后，**AegisPath** 会将生成结果整理为统一格式的见证集合 $\mathcal{W}_v$ 。本小节的目的，是说明生成模块如何将离线计算得到的对象级语义结果进一步组织为后续审计阶段可直接引用的版本化证据。对 **AegisPath** 而言，离线生成期的最后阶段主要完成以下三个动作：

1. **数据格式对齐与公开索引确定**：将原本按计算过程逐步产生的对象级结果，从内部对象槽表中提取出来，并按照覆盖对象的公开规范顺序重新排列，再附加版本号、时间戳等版本元数据，从而形成版本 v 下的结构化见证集合 $\mathcal{W}_v$ 。这里的覆盖对象索引为 $(SourceAS, prefix)$ 二元组；该二元组属于后续查询中可公开引用的对象标识，而本文需要保护的是其对应的真实路径见证与域内策略细节，因此最终发布阶段可按该公开二元组顺序对结果进行统一整理。由此，生成期内部对象被处理和细化的先后次序，不会直接决定最终证据集合中的排列位置。
2. **生成对象级 MiMC 叶子承诺**：为每个对象级见证确定统一的序列化方式与字段顺序，并在离线生成阶段计算对应的 MiMC 叶子承诺。对象级承诺在计算完成后，首先在生成模块内部与其公开索引完成一一对应，并按统一顺序进行对齐；中间未排序结果不作为公开接口暴露。这样，离线生成期输出的不仅包括结构化见证集合，也包括与之逐一对应的对象级承诺值。
3. **构造 Merkle 树并发布版本根**：在叶子承诺已经按公开索引完成统一排序后，系统进一步对全部叶子承诺执行 Merkle 聚合，构造对应版本的证据树，并发

布公开版本根 R_v 作为该版本状态的全局锚点。若后续需要公开叶子级承诺信息，也应以排序完成后的叶子序列为准，而不公开中间未排序结果。

这种结构化封装的意义在于：生成期输出的不再只是若干彼此独立的路径结果，而是一个可按版本整体引用、整体绑定的见证集合及其对应的承诺与版本根。若后续查询期仅依赖托管服务本地持有的见证明文，而不存在一个公开固定的版本根作为全局锚点，则证明服务在查询时仍可能替换、混用或选择性返回不同见证。为避免这一问题，AegisPath 在离线生成阶段就完成对象级 MiMC 承诺与版本根发布，使后续查询期所使用的任一对象级见证都必须先对应到该版本下已经固定的叶子承诺，再证明该叶子确实属于版本根 R_v 所绑定的集合；因此，证明服务不能在保持版本根不变的前提下任意伪造、替换或混用见证。

在本文的威胁模型下，真正需要保护的是对象所对应的路径语义与域内策略影响，而不是覆盖对象本身的公开索引。因此，叶子排序由公开的 $(SourceAS, prefix)$ 二元组确定，而中间计算阶段产生的未排序承诺结果不作为对外发布对象。离线生成阶段的主要密码学工作集中在对象级 MiMC 承诺计算、按公开索引完成对齐以及后续 Merkle 根构造上。由此，版本根 R_v 可被理解为对按公开顺序排列的对象级承诺序列的确定性聚合结果。

因此，本小节完成的是从对象级语义结果到结构化见证集合、对象级叶子承诺以及公开版本根输出的离线生成闭环；而围绕这些叶子承诺如何在查询期通过零知识证明实现成员性验证与性质验证，将在第 4 章中进一步展开。

3.3 安全性分析

本文生成期协议的安全性建立在两个层面之上：其一是门限秘密分享所提供的输入隐私，其二是数据无关执行所提供的执行轨迹隐私。前者保证任意未达到重构阈值的参与方集合无法从份额中恢复其他自治系统的明文配置；后者保证协议的控制流、循环轮次与对象槽表访问方式仅由公开参数和公开调度策略决定，而不随秘密输入变化。

引理 3-1. 设各自治系统的本地输入经 (t, n) -Shamir 秘密分享后进入生成期协议，则任意少于 t 个计算实体所观察到的份额分布与底层明文配置无关。

证明. 该结论直接来自 Shamir 秘密分享的标准安全性质。对任意秘密 $x \in \mathbb{F}_p$ ，其份额由一个常数项为 x 的随机 $(t - 1)$ 次多项式在不同点上的取值给出。若观察到

的份额数严格小于 t ，则不足以唯一确定该多项式的常数项。于是，对任意候选秘密值，总存在与已观察份额一致的多项式，因此少于阈值的参与方无法区分不同明文输入对应的份额分布。□

引理 3-2. 生成期全局调度流程与逐对象语义推演过程的控制流和访问模式仅由公开参数决定。对象槽位上界、规则块扫描顺序、路径推进轮数、空闲槽位分配顺序以及槽位索引访问方式均不依赖于秘密输入取值。

证明. 由算法 1 可知，协议首先按自治系统粒度对规则集合执行固定排序，再在固定大小的全局对象槽表上执行统一调度。调度循环的轮数由公开参数 $MaxSteps$ 与对象槽位上界 O_{max} 决定；逐对象推演中每一跳的规则扫描顺序由已排序规则块 R_u 决定；对象细化后的子对象写入位置由公开槽位索引与空闲槽位指针确定。尽管对象是否触发细化、哪些槽位对应真实有效对象、哪些对象较早终止均与秘密输入有关，但这些差异仅体现在秘密比特和选择器输入取值上，不会改变循环轮数、数组访问位置更新规则或槽位分配方式。因此，协议的执行轨迹与访问模式不泄露额外秘密信息。□

定理 3.3.1. 在半诚实模型下，若计算实体的合谋规模严格小于重构阈值 t ，则本文的生成期协议除最终输出见证集合 $\mathcal{W}_o$ 所必然泄露的信息外，不会额外泄露任一自治系统的本地明文配置。

证明. 根据引理 3-1，协议输入在门限秘密分享层面满足标准输入隐私，未达到阈值的参与方无法从份额中恢复明文配置。根据引理 3-2，生成期的全局对象槽表调度、规则扫描、槽位分配与路径推进过程都遵循公开确定的执行轨迹，因此观察协议执行过程本身不会产生额外的控制流泄露。协议中的局部细化、规则命中消解和路径状态更新均通过秘密分享上的局部运算与选择器完成，不暴露明文中间状态。故在半诚实模型下，攻击者所能获得的信息被限制为其自身输入、公开参数及最终输出所包含的信息，协议满足本章所需的生成期隐私要求。□

3.4 本章小结

本章围绕多管理域数据平面审计的生成期需求展开：定义二元组级验证对象和见证格式，提出隐私约束下的生成流程，并详细设计了数据编码与核心算法。通过语义等价性分析和复杂度评估，验证了方法的正确性与效率，为后续可查询期提供了必要的结构化输入和理论支持。

第4章 基于零知识证明的版本化数据平面审计

第三章解决了如何在隐私保护约束下生成多管理域数据平面语义见证的问题，本章进一步讨论如何在不泄露真实转发路径与域内策略细节的前提下，对外提供可独立复验的审计接口。为了实现这一目标，系统需要在生成期见证与查询期审计之间建立稳定的证据组织与发布机制，从而确保查询时能够提供一致且可验证的审计回答。

一个朴素的想法是，将生成的见证交由中心化服务进行托管，并通过该服务向外提供查询接口。然而，单纯的中心化托管并不足以直接构成可信的审计接口。首先，托管服务可能为减少事故责任带来的社会与经济影响，与相关自治系统串通，通过替换、删改或选择性返回见证来误导外部验证者，进而伪造审计结果；在缺少额外密码学绑定机制时，查询方难以判断服务返回的对象是否真实对应于目标时刻的网络状态。其次，已有的 RPKI 研究表明，中心服务自身也可能因状态管理错误、版本混用、同步异常或内部故障而返回与目标时刻不一致的对象^[82-84]。

基于这些考虑，本文不将托管服务本身视为可信真实性来源，而是采用生成期承诺、查询期证明的设计思路：在生成期结束后，对二元组级语义见证计算哈希承诺并聚合为版本根，使后续查询使用的对象与公开版本根一一对应，不能被任意替换或混用；在查询期，证明方基于特定版本的见证构造零知识证明，使验证者能够在不接触明文路径的前提下，独立核验目标覆盖对象是否满足预定的数据平面不变量。通过这种方式，系统实现了多管理域联合计算与查询期审计的解耦，在保持隐私约束的同时降低重复查询的在线成本。

本章首先给出多管理域数据平面审计问题的形式化定义，明确见证、证据与审计结论之间的关系 (§4.1)；随后介绍版本化证据结构的构建逻辑，重点说明二元组级见证如何经由哈希承诺与 Merkle 聚合形成可绑定版本的证据根 (§4.2)；在此基础上，说明数据平面不变量是如何编码为可验证的算术约束 (§4.3)；最后，对本章方案的安全性进行分析 (§4.4)。

4.1 问题定义与生成目标

本节明确审计机制的核心逻辑，即如何将生成期产出的语义见证集合转化为外部可验证的证据。在特定版本下，虽然推演得到的见证已经包含了覆盖对象对

应的完整转发语义，但这些数据直接关联自治系统的路径与策略隐私，因而不能以明文形式对外公开。本章所研究的审计问题可以概括为：在隐藏见证明文的前提下，如何构造一份简洁的密码学凭证，使验证者能够独立确认某一覆盖对象在特定版本状态下是否满足预设的数据平面不变量。

审计接口面向查询期工作：对外不返回路径详情，而是返回与查询相关的逻辑判定及其可复验凭证。验证者仅需持有系统发布的版本根以及查询描述与结论，即可通过零知识证明完成核验。换言之，审计的信任基础从对明文数据的查验转移为对数学约束的校验。为精确描述该过程，下面给出形式化定义。

定义 4.1 (路径属性审计问题). 给定版本 v 下的私有见证集合

$$\mathcal{W}_v = \{W_{v,p,s} \mid (p, s) \in \mathcal{Q}(v)\},$$

以及与之对应的公开版本根 R_v 。

对于审计者发起的针对覆盖对象 (p, s) 的不变量 Φ 查询，审计接口输出判定结论 $y \in \{0, 1\}$ 与零知识证明 π 。该审计陈述的正确性由一个算术关系 $\mathcal{R}$ 定义：当且仅当存在私有见证 $W_{v,p,s}$ 使得下式成立时，陈述有效：

$$\mathcal{R}((R_v, p, s, \Phi, y), W_{v,p,s}) = 1 \iff \mathcal{F}_{\text{member}}(W_{v,p,s}, R_v, p, s) \wedge \mathcal{F}_{\text{check}}(W_{v,p,s}, \Phi, y).$$

其中，算术关系由以下两部分约束构成：

1. **成员性约束 $\mathcal{F}_{\text{member}}$** ：证明由 $W_{v,p,s}$ 导出的叶子承诺确实属于版本根 R_v 所绑定的见证集合。该约束用于将审计结论锚定在声称的版本中，并防止证明方通过注入伪造见证来欺骗验证者。
2. **属性匹配约束 $\mathcal{F}_{\text{check}}$** ：证明在私有见证 $W_{v,p,s}$ 上执行不变量判定逻辑得到的结果与公开结论 y 严格一致。该约束确保证明方不能在成员性成立的前提下，将不变量判定结果伪装为与 y 不一致的结论。

验证者持有公开参数 (R_v, p, s, Φ, y) ，执行验证函数：

$$\text{Verify}(\pi; R_v, p, s, \Phi, y) \rightarrow \{1, 0\}. \quad (4-1)$$

当且仅当验证函数输出为 1 时，验证者接受该审计陈述成立，即在不接触见证明文的条件下完成对数据平面不变量结论的核验。

4.2 版本化证据结构

上一节将多管理域数据平面审计问题的形式化定义为算术关系 $\mathcal{R}$ ，其中成员性约束 $\mathcal{F}_{\text{member}}$ 用于将私有见证 $W_{v,p}$ 锚定到公开版本根 R_v ，从而固定审计结论所对应的版本语义状态。本节进一步给出版本化证据结构的构建逻辑：在生成期结束后，对覆盖集上的前缀级语义见证进行哈希承诺，并按固定索引规则聚合为 Merkle 树，最终发布版本根 R_v 作为该版本的公开证据锚点。随后，查询期证明方可针对任意前缀提取对应的成员性路径，并与不变量判定逻辑一起构成零知识审计的输入。

4.2.1 语义见证的哈希承诺生成

对版本 v 下每个覆盖对象 $(p, s) \in \mathcal{Q}(v)$ ，生成期得到私有语义见证 $W_{v,p,s}$ 。为在不公开见证明文的条件下提供可核验的绑定关系，本文对 $W_{v,p,s}$ 计算一个叶子承诺 $\text{leaf}_{v,p,s}$ ，并将其作为 Merkle 树叶子输入。叶子承诺的作用是将见证内容压缩为短摘要，使得任何对见证的篡改都会导致叶子承诺变化，从而在后续成员性验证中被检测出来。

为保证电路可实现性与接口稳定性，本文采用固定格式的见证编码。具体地，令前缀 $p = (a_p, \ell_p)$ ，其中 a_p 与 ℓ_p 分别表示前缀地址与前缀长度的整数编码；令 s 表示起始自治系统标识；令 $\mathbf{s}_{v,p,s} = (s_0, \dots, s_{L-1})$ 表示固定长度路径槽位向量。则见证可写为

$$W_{v,p,s} = (a_p, \ell_p, s, \mathbf{s}_{v,p,s}). \quad (4-2)$$

在此基础上，叶子承诺定义为

$$\text{leaf}_{v,p,s} = \text{MiMC}(\text{Encode}(W_{v,p,s})), \quad (4-3)$$

其中 $\text{Encode}(\cdot)$ 为将见证字段串联并映射到电路有限域的编码函数， $\text{MiMC}(\cdot)$ 为适合零知识电路实现的哈希承诺原语。该承诺既可在电路外计算用于构建 Merkle 树，也可在电路内复算以约束私有见证 $\leftrightarrow$ 公共叶子承诺的一致性，从而为后续的成员性证明与不变量证明提供统一接口。

4.2.2 Merkle 聚合与版本根生成

为了将版本 v 下的全量见证统一绑定为单个公开锚点, 本文将所有叶子承诺 $\{\text{leaf}_{v,p,s} \mid (p,s) \in \mathcal{Q}(v)\}$ 构造为 Merkle 证据树, 并将其根节点发布为版本根 R_v 。在该结构中, 叶子节点的排列顺序必须遵循公开且确定的索引规则, 否则验证者无法判断某个查询对象对应的叶子位置, 也无法排除证明方通过交换叶子位置来操纵成员性结果。因此, 本文在每个版本 v 上固定覆盖对象集合 $\mathcal{Q}(v)$ 的一个规范序, 并据此定义唯一的叶子索引。

设覆盖对象为二元组 $q = (p, s) \in \mathcal{Q}(v)$, 其中前缀 $p = (a_p, \ell_p)$ 由前缀地址 a_p 与前缀长度 ℓ_p 构成。本文采用如下排序规则: 首先按前缀地址的整数编码从小到大排序; 若前缀地址相同, 则按前缀长度从小到大排序; 若前缀本身完全相同, 则再按起始自治系统标识从小到大排序。形式上, 对任意 $q_1 = (p_1, s_1), q_2 = (p_2, s_2)$, 定义排序关系 $\prec$ 如下:

$$\begin{aligned} q_1 \prec q_2 \iff & \text{addr}(p_1) < \text{addr}(p_2) \\ & \vee (\text{addr}(p_1) = \text{addr}(p_2) \wedge \ell_{p_1} < \ell_{p_2}) \\ & \vee (\text{addr}(p_1) = \text{addr}(p_2) \wedge \ell_{p_1} = \ell_{p_2} \wedge s_1 < s_2). \end{aligned} \quad (4-4)$$

图 4.1 以一组具体的覆盖对象为例展示了上述排序逻辑。在该示例中, 对象 $q_1(134.34.0.0/24, S_1)$ 与 $q_2(134.34.0.0/25, S_1)$ 虽然起始地址相同, 但由于前缀长度 $\ell_1 < \ell_2$, 故 q_1 排在 q_2 之前; 而对于前缀完全相同的 q_2 与 $q_3(134.34.0.0/25, S_2)$, 则通过起始自治系统标识 $S_1 < S_2$ 确定先后顺序。

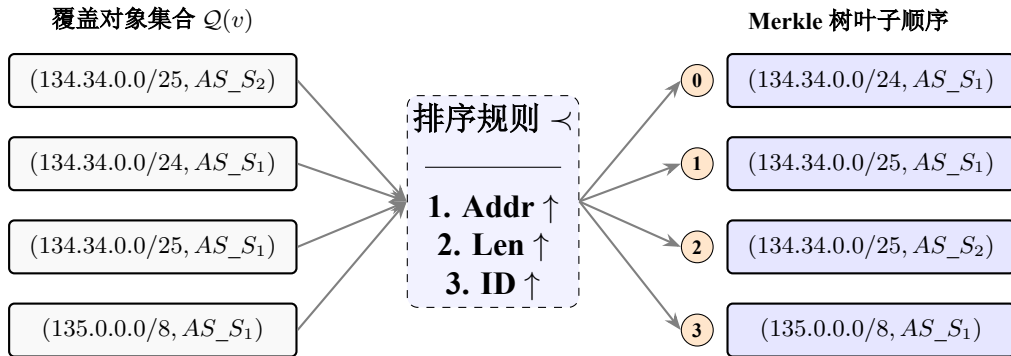


图 4.1 覆盖对象在 Merkle 树内的排序规则示意图

由此, $\prec$ 在 $\mathcal{Q}(v)$ 上推导出一个全序关系, 使每个覆盖对象都具有唯一确定的叶子位置。定义版本 v 的叶子索引函数为 $\text{idx}_v: \mathcal{Q}(v) \rightarrow \{0, 1, \dots, |\mathcal{Q}(v)| - 1\}$, 其

中 $\text{idx}_v(p, s)$ 表示对象 (p, s) 在上述规范序中的排名。于是, Merkle 树的第 $\text{idx}_v(p, s)$ 个叶子被定义为 $\text{leaf}_{v,p,s}$, 相应地有

$$R_v = \text{MerkleRoot}\left(\{\text{leaf}_{v,p,s}\}_{(p,s) \in Q(v)}; \text{idx}_v\right). \quad (4-5)$$

在得到上述索引之后, 系统即可按照叶子编号顺序组织版本 v 的底层承诺层。具体而言, 对每个覆盖对象 $(p, s) \in Q(v)$, 系统先计算其对应的数据面语义见证承诺 $\text{leaf}_{v,p,s}$, 再按照 $\text{idx}_v(p, s)$ 的顺序将这些叶子放置到 Merkle 树的底层。随后, 系统按照固定的左右子节点组合方式, 自底向上递归计算各层内部节点哈希, 直至生成唯一的根节点 R_v 。

图 4.2 给出了版本 v 下 Merkle 聚合过程的结构示意。叶子层按照前述索引顺序放置各覆盖对象对应的叶子承诺, 其中每个叶子均由相应对象的数据面语义见证经 MiMC 承诺得到。设排序后第 i 个覆盖对象对应的叶子为 $\text{leaf}_{v,i}$, 则系统首先以叶子层为第 0 层节点, 即令 $h_i^{(0)} = \text{leaf}_{v,i}$ 。随后, 对任意第 t 层的相邻两个子节点 $h_{2j}^{(t)}$ 与 $h_{2j+1}^{(t)}$, 系统按照固定的左右顺序进行哈希聚合, 得到上一层节点:

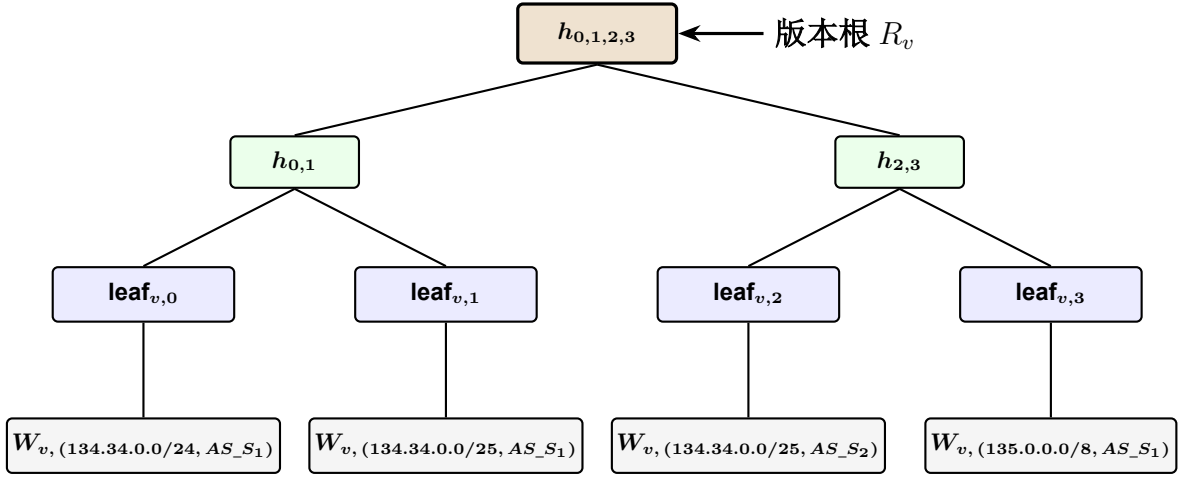
$$h_j^{(t+1)} = \text{MiMC}(h_{2j}^{(t)}, h_{2j+1}^{(t)}). \quad (4-6)$$

系统重复上述过程, 直至得到最顶层的唯一根节点。若记 Merkle 树总高度为 H_v , 则版本 v 的根节点可写为

$$R_v = h_0^{(H_v)}. \quad (4-7)$$

以图 4.2 中的四个叶子为例, 第一层聚合结果可分别记为 $h_{0,1} = \text{MiMC}(\text{leaf}_{v,0}, \text{leaf}_{v,1})$ 与 $h_{2,3} = \text{MiMC}(\text{leaf}_{v,2}, \text{leaf}_{v,3})$, 最终根节点为 $R_v = h_{0,1,2,3} = \text{MiMC}(h_{0,1}, h_{2,3})$ 。因此, R_v 用于对版本 v 下全部覆盖对象的叶子承诺及其位置关系进行统一绑定, 为后续查询期的成员性证明提供公开锚点。

通过以上设计, Merkle 树中审计对象的叶子位置与树形聚合过程均被固定下来。给定任意查询对象 (p, s) , 证明方可依据 $\text{idx}_v(p, s)$ 确定其对应叶子 $\text{leaf}_{v,p,s}$ 在树中的位置, 并进一步提取从该叶子到版本根 R_v 的认证路径。验证方则可基于公开的版本根检查该叶子是否属于版本 v 所承诺的全量审计状态, 而无须获知其他覆盖对象的见证内容。


 图 4.2 版本 v 下的 Merkle 树结构示例图

4.3 数据平面不变量验证电路

上一节给出了版本化证据结构的构建逻辑：对每个覆盖对象见证 $W_{v,p,s}$ 计算叶子承诺 $\text{leaf}_{v,p,s}$ ，并按固定索引规则聚合为 Merkle 根 R_v 。本节进一步给出查询期零知识审计电路的实现细节，用以落实 4.1 中的算术关系 $\mathcal{R}$ ：证明方在不公开见证明文的前提下，证明（1）其持有的见证确实属于版本根 R_v 所绑定的见证集合，以及（2）该见证在不变量判定逻辑下导出的结果与公开结论 y 一致。本节重点说明电路输入规格、成员性约束的电路化方式，以及本文实现的可达性、必经点与无环性三类不变量的算术化编码。

4.3.1 电路输入规格与约束框架设计

审计电路以版本根 R_v 为核心公共锚点，并将查询描述与结论显式作为公开输入，使证明与验证在语义上绑定到“哪个版本、哪个覆盖对象、核验哪个不变量、声称的结论是什么”。在本文实现中，电路输入分为三类：

公开输入包括：版本根 R_v ；查询对象 (p, s) 的编码（或其索引 $\text{idx}(p, s)$ ）；不变量类型 Φ 及其公开参数（例如必经点查询中的目标自治系统标识 w ）；以及系统对外声称的结论 $y \in \{0, 1\}$ 。为便于实现，本文将不同不变量的结论统一编码为若干布尔公开输入（如 **Reach**、**WaypointHit**、**LoopFree**），并在对应查询中启用相关字段。

私有输入包括：二元组级语义见证 $W_{v,p,s} = (a_p, \ell_p, s, \mathbf{s}_{v,p,s})$ ，其中 $\mathbf{s}_{v,p,s}$ 为固定长度路径槽位向量；以及该叶子对应的 Merkle 认证路径信息（兄弟节点列表与方向位）。这些私有输入在电路内用于复算叶子承诺、重构根值，并对不变量判定逻辑

辑提供见证。

约束框架由两部分组成：成员性约束用于保证见证属于版本根 R_v ；属性匹配约束用于保证不变量判定结果等于公开结论。两类约束在同一电路内组合，使证明方无法在“成员性成立但属性不成立”或“属性成立但成员性不成立”的情况下通过验证，从而实现对审计陈述的整体绑定。

4.3.2 见证合法性与版本一致性约束

成员性约束的目标是将私有见证 $W_{v,p,s}$ 锚定到公开版本根 R_v 。电路内首先根据私有见证计算叶子承诺，再沿 Merkle 认证路径迭代计算根值，并约束该根值等于公开输入 R_v 。该过程可形式化描述为：

叶子复算。电路内对见证执行固定格式编码，并计算

$$\text{leaf}_{v,p,s} = \text{MiMC}(\text{Encode}(W_{v,p,s})). \quad (4-8)$$

根值重构。令 $\text{sib}[i]$ 表示第 i 层的兄弟节点， $\text{dir}[i] \in \{0, 1\}$ 表示方向位（例如 0 表示当前哈希在左，1 表示在右）。从 $\text{leaf}_{v,p,s}$ 出发，电路按层迭代计算

$$h_0 = \text{leaf}_{v,p,s}, \quad h_{i+1} = \text{Hash}(\text{Sel}(\text{dir}[i], \text{sib}[i], h_i), \text{Sel}(\text{dir}[i], h_i, \text{sib}[i])), \quad (4-9)$$

最终得到 h_d 并施加约束 $h_d = R_v$ ，其中 d 为树高， $\text{Hash}(\cdot)$ 与叶子承诺保持一致。方向选择通过无泄露选择器 Sel 实现，从而避免在电路内引入数据相关分支。

上述约束保证了：证明方所使用的见证确实对应某个 Merkle 叶子，并且该叶子属于公开版本根 R_v 所绑定的见证集合。由于叶子由见证内容唯一确定，证明方无法在不改变见证的情况下伪造叶子，也无法在不改变根的情况下替换叶子，从而实现成员性约束 $\mathcal{F}_{\text{member}}$ 的电路化落地。

4.3.3 网络语义不变量的算术化编码

在成员性约束成立的基础上，电路进一步对私有见证中的路径槽位向量 $\mathbf{s}_{v,p,s}$ 执行不变量判定，并将判定结果与公开结论对齐。本文当前实现的三类不变量均以路径槽位向量为输入，并在固定长度 L 上以纯算术约束编码。

4.3.3.0.1 可达性 (Reachability)。可达性对应“路径未以 DROP 终止且到达目标语义终点”。在实现中，电路对路径槽位逐位检查是否出现 DROP 终止符，并结

合终点标记得得到 $\text{ReachCalc} \in \{0, 1\}$ 。最后施加约束

$$\text{ReachCalc} = \text{Reach},$$

其中 **Reach** 为公开输入的声称结论。若你将“终点”抽象为某个目标自治系统标识，则可达性可写为“存在某一槽位等于目标且此前未出现 **DROP**”。

4.3.3.0.2 必经点 (Waypointing)。 必经点性质对应“路径中存在某指定自治系统 w ”。电路对每个槽位 s_i 执行等值比较得到命中比特 $b_i = [s_i = w]$ ，再对所有 b_i 做或聚合得到

$$\text{WaypointCalc} = b_0 \vee b_1 \vee \dots \vee b_{L-1},$$

并施加约束 $\text{WaypointCalc} = \text{WaypointHit}$ ，其中 **WaypointHit** 为公开输入结论。等值比较与或聚合均可用算术化布尔约束实现。

4.3.3.0.3 无环性 (Loop-Freeness)。 无环性对应路径槽位向量中不出现重复自治系统标识。电路可对所有成对槽位 (s_i, s_j) ($i < j$) 执行不等约束，并用屏蔽比特忽略填充项，从而得到 $\text{LoopFreeCalc} \in \{0, 1\}$ ，最后施加约束 $\text{LoopFreeCalc} = \text{LoopFree}$ （公开输入）。该编码的计算量与 L^2 成正比，但在本文的二元组级路径槽位上界设置下可接受。

以上三类不变量的编码均在固定长度槽位向量上执行，电路形状不随实际路径长度变化；若路径较短，则由 **PAD** 填充并通过屏蔽约束避免影响判定结果。该设计保持了电路的可复用性，也与生成期见证的固定格式输出一致。

4.3.4 审计证明的生成与验证流程

如算法 3 所示，查询期审计流程由证明生成与证明验证两个阶段组成。其目标是：在不公开见证明文的前提下，使验证者能够确认某个覆盖对象 (p, s) 在版本 v 下是否满足给定不变量 Φ ，且该结论确实来源于版本根 R_v 所绑定的见证集合。

在证明生成阶段，证明方首先根据审计者提交的查询请求 (v, p, s, Φ) 从本地私有存储中检索对象 (p, s) 在版本 v 下对应的语义见证 $W_{v,p,s}$ ，并同步获取该叶子在版本证据树中的 **Merkle** 认证路径 $\text{auth}_{p,s}$ 。其中， $\text{auth}_{p,s}$ 由两部分组成：其一是从叶子节点到根节点路径上各层的兄弟节点哈希值；其二是对应的方向标记，用于指示当前节点在该层位于左子树还是右子树。这些信息与语义见证一起构成证明

方在查询期所使用的私有见证 (private witness)。

随后, 证明方在零知识证明电路内构造算术约束系统。电路首先对私有见证 $W_{v,p,s}$ 按固定字段顺序进行编码, 并在电路内重新计算其叶子承诺

$$\text{leaf}_{v,p,s} = \text{MiMC}(\text{Encode}(W_{v,p,s})). \quad (4-10)$$

在此基础上, 电路将 $\text{leaf}_{v,p,s}$ 作为路径重构的起点, 并利用 $\text{auth}_{p,s}$ 中给出的兄弟节点哈希值与方向标记, 自底向上逐层重构根值。若记第 i 层的兄弟节点哈希为 sib_i , 方向位为 $\text{dir}_i \in \{0, 1\}$, 则电路从 $h_0 = \text{leaf}_{v,p,s}$ 出发, 迭代计算

$$h_{i+1} = \text{Hash}(\text{Sel}(\text{dir}_i, \text{sib}_i, h_i), \text{Sel}(\text{dir}_i, h_i, \text{sib}_i)), \quad (4-11)$$

最终得到重构根值 $R'_v = h_d$, 其中 d 为 Merkle 树高度。最后, 电路施加相等约束 $R'_v = R_v$, 从而落实成员性校验 $\mathcal{F}_{\text{member}}$ 。该约束的含义是: 证明方当前用于回答查询的私有见证, 确实属于公开版本根 R_v 所绑定的叶子集合, 而不是在查询期临时伪造、替换或从其他版本中混入的对象。

在确保见证归属真实性之后, 电路进一步执行属性匹配校验 $\mathcal{F}_{\text{check}}$ 。证明方调用预设的不变量判定算子 $\text{InvEval}(\Phi, W_{v,p,s})$, 在私有见证上计算得到布尔结果 y' 。随后, 电路施加约束 $y' = y$, 其中 y 为对外公开声称的审计结论。该约束要求: 一方面, 证明方必须持有属于版本 R_v 的合法见证; 另一方面, 电路在该见证上实际计算出的属性结果必须与对外声称的结论严格一致。由此, 证明方不能在持有合法见证的前提下篡改查询结果, 也不能在声称正确结果的同时使用不属于版本 R_v 的对象。

当上述成员性约束与属性匹配约束在电路内同时满足后, 证明方运行证明算法生成零知识证明 π 。在验证阶段, 验证者持有公开输入 (R_v, p, s, Φ, y) 以及证明 π , 并调用验证函数检查其数学有效性。若验证通过, 则说明验证者在不接触见证明文的前提下, 确认了如下命题: 对象 (p, s) 在版本 v 所对应的见证状态下满足不变量 Φ , 且该结论与公开版本根 R_v 保持一致。

4.4 安全性分析

本章审计接口建立在通用零知识证明系统之上。为表述方便, 令公开输入

$$\text{st} = (R_v, p, s, \Phi, y),$$

算法 3: 查询期零知识审计：证明生成与验证

输入: 公开输入 $\text{st} = (R_v, p, s, \Phi, y)$; 验证键 vk ; 证明键 pk

输入: 私有数据: 见证库 $\mathcal{W}_v$; 认证路径库 $\mathcal{A}_v$

输出: 证明 π ; 验证结果 b

```

1  Prover:
2       $(W_{v,p,s}, \text{auth}_{p,s}) \leftarrow \text{Fetch}(v, p, s);$  // 读取见证与认证路径
3       $\text{leaf}_{v,p,s} \leftarrow \text{MiMC}(\text{Encode}(W_{v,p,s}));$  // 复算叶子承诺
4       $h_0 \leftarrow \text{leaf}_{v,p,s};$ 
5      for  $i \leftarrow 0$  to  $d - 1$  do
6          读取  $(\text{sib}_i, \text{dir}_i) \in \text{auth}_{p,s};$ 
7           $h_{i+1} \leftarrow \text{Hash}(\text{Sel}(\text{dir}_i, \text{sib}_i, h_i), \text{Sel}(\text{dir}_i, h_i, \text{sib}_i));$ 
8      end
9       $R'_v \leftarrow h_d$ , 约束  $R'_v = R_v$ ; // 成员性校验
10      $y' \leftarrow \text{InvEval}(\Phi, W_{v,p,s})$ , 约束  $y' = y$ ; // 属性匹配
11      $\pi \leftarrow \text{ZK.Prove}(\text{pk}, \text{st}, \{W_{v,p,s}, \text{auth}_{p,s}\});$ 
12     return  $\pi$ ;

13 Verifier:
14      $b \leftarrow \text{ZK.Verify}(\text{vk}, \text{st}, \pi);$ 
15     return  $b$ ;
    
```

私有见证为

$$\mathbf{w} = (W_{v,p,s}, \text{auth}_{p,s}),$$

其中 $\text{auth}_{p,s}$ 为对应的 Merkle 认证路径。令 $\mathcal{R}$ 为 4.1 节定义的算术关系，即

$$\mathcal{R}(\text{st}, \mathbf{w}) = 1 \iff \mathcal{F}_{\text{member}}(W_{v,p,s}, R_v, p, s) \wedge \mathcal{F}_{\text{check}}(W_{v,p,s}, \Phi, y).$$

本章采用 Groth16 作为查询期的证明系统，其标准性质由底层协议直接保证。

引理 4-1 (Groth16 的完备性、零知识性与健全性). 设关系 $\mathcal{R}$ 被编译为 Groth16 支持的约束系统（电路），证明系统由 $(\text{Prove}, \text{Verify})$ 给出。则 Groth16 满足以下标准性质：(1) **完备性 (Completeness)**: 若存在 $\mathbf{w}$ 使 $\mathcal{R}(\text{st}, \mathbf{w}) = 1$ ，则诚实证明方生成的证明会被验证者接受；(2) **零知识性 (Zero Knowledge)**: 对任意多项式时间验证者，存在模拟器使其看到的证明分布可被模拟，从而不泄露关于 $\mathbf{w}$ 的额外信息；(3) **健全性 (soundness)**: 任何多项式时间的证明方若能产生被接受的证明，则必然对应某个使关系成立的见证（除可忽略概率外）。

证明. 以上结论为 Groth16 协议的标准性质，详见 Groth16 原论文^[57]。 $\square$

定理 4.4.1 (本章审计接口的零知识隐私性). 若本章电路正确实现关系 $\mathcal{R}$ ，且公开输入/私有见证的划分与 4.1 节定义一致，则验证者除公开输入 (R_v, p, s, Φ, y) 与证明系统必然泄露外，不会从证明中获得关于私有见证 $W_{v,p,s}$ 的额外信息。

证明. 由引理 4-1 的零知识性可知, 给定公开输入 st , 验证者可见到的证明分布可被多项式时间模拟器生成, 从而不依赖私有见证 w 的具体取值。因此, 只要本章将路径槽位、动作处置与认证路径等敏感信息仅作为私有见证输入电路, 并且电路不额外输出明文 $witness$, 则证明不会泄露额外信息。该结论直接归约到 Groth16 的零知识性。□

4.5 本章小结

本章将生成期的私有语义见证转化为对外可复验的审计证据。通过构造版本根 R_v 并在电路内同时约束成员性与不变量判定, 本章实现了对外的零知识审计接口; 其安全性分别由零知识性、完备性与健全性三项性质保证, 为下一章的版本维护与增量更新奠定基础。

第5章 面向频繁状态变化的增量更新机制

前文已经给出了本文系统在生成期与查询期的具体设计。然而，多管理域网络的运行状态并非静态不变。已有测量研究表明，互联网控制平面中的路由更新是持续存在的，公开路由观测系统长期记录前缀宣告、路径变化与路由表快照的演化过程^[36, 85]。因此，在不同时间点，被外部观测系统持续关注的前缀及其控制面路径会发生变化。相应地，审计覆盖对象也不是一个与时间无关的静态集合，而需要依据特定时间点上公开可观测到的前缀及其控制面输入加以确定。基于这一原因，一旦公开控制面数据源中相关前缀的状态发生更新，系统就需要围绕新的时间点调整覆盖对象集合，并同步维护这些覆盖对象在相应版本下对应的数据平面语义见证。

另一方面，即便审计覆盖对象集合保持不变，网络内部配置与边界策略的局部调整仍可能改变既有覆盖对象所对应的真实数据平面语义。已有研究表明，网络更新过程中即便初始状态与最终状态本身满足预期，中间状态仍可能引入黑洞、环路或策略违例^[86-87]。因此，在长期运行的多管理域环境中，需要被维护的并不只是哪些对象应纳入当前版本的审计覆盖范围，还包括既有覆盖对象在新状态下是否对应新的数据平面语义见证。

在本文中，需要进行版本化维护的变化来源主要包括两类：一类是公开控制面输入的变化，例如前缀宣告状态、可见路径或关注范围发生改变；另一类是网络配置的变化，例如边界过滤、黑洞策略、访问控制、策略路由的局部调整。Zhao等人的研究指出，数据面配置的变更通常具有显著的增量特征，单次更新往往仅涉及少量的规则变动，且其影响范围被局限于网络流量的极小部分^[88]。

为此，本文引入版本化维护与增量更新机制：当网络仅发生局部变化时，只对受影响的二元组对象重算语义见证、更新相应叶子承诺及其到根的相关分支，并发布新的版本根。这样既能够保持全局审计证据的完整性与版本绑定关系，又能够将维护成本限制在真实受影响的对象范围内，从而为后续的跨版本对比、修复复核与重复审计提供支撑。

本章围绕版本化维护与增量更新机制的具体实现展开。首先，介绍生成期的增量推演接口与算法 (§5.1)；其次，说明如何在 Merkle 证据结构上完成叶子更新、分支维护与新版本根发布，并进一步支持查询期的版本绑定与跨版本复核 (§5.2)；最后，对本章机制的安全性进行分析 (§5.3)。

5.1 生成期增量更新

5.1.1 增量输入接口与驱动方式

为使生成期能够在保持固定计算框架的同时仅对局部对象重算，本文将版本 $v \rightarrow v + 1$ 的更新请求抽象为一个增量输入接口

$$\Delta_v = (mode, delta_objs, prev_witness, prev_result),$$

其中 $mode$ 用于标识本轮更新的触发类型， $delta_objs$ 用于给出候选受影响对象集合， $prev_witness$ 表示上一版本已经生成的路径见证集合， $prev_result$ 则表示上一版本中可直接复用的派生结果。该接口的目标是提供哪些对象需要重新计算的驱动信息，使系统能够在上一版本基础上执行局部重算。

在据此识别本轮需更新的对象之前，系统会对上一版本中拟复用的结果执行一次有效性校验。对于 $prev_witness$ 中待复用的路径见证及 $prev_result$ 中可直接继承的派生结果，系统重新计算其对应的 MiMC 叶子承诺，并结合版本 v 已公开发表的版本根 R_v 检查这些对象是否仍与上一版本的已承诺审计状态一致。只有当相应对象能够通过该承诺一致性校验时，系统才将其视为可复用的有效输入；否则，该对象将被纳入本轮重算范围。通过这一步骤，增量更新过程的起点被约束在版本 v 已绑定的旧状态之上，从而避免后续局部重算建立在未验证或已失效的历史结果之上。

本节所支持的将增量触发分为以下两种来源。

(1) 配置变更触发。当某一自治系统 AS_k 的本地配置发生变化时，系统并不直接按前缀集合做粗粒度更新，而是以上一版本的路径见证集合作为输入，对其中的路径见证进行扫描。若旧版路径见证的路径槽位中包含 AS_k ，则对应的覆盖对象 (p, s) 被判定为候选受影响对象。形式上，若上一版本路径见证 $W_{v,p,s}$ 所编码的路径记为 $\pi_{v,p,s}$ ，则可将该触发条件写为

$$(p, s) \in S_v \leftarrow AS_k \in \pi_{v,p,s}.$$

首先通过旧路径见证是否经过受影响 AS 进行传播，而不是简单地由运维方手工列举所有前缀对象。这样既更符合真实网络中局部配置变更对路径语义的影响方式，也能够自然对接本文以 (p, s) 为单位的路径见证结构。

(2) 前缀输入变更触发。当公开控制面数据源中某个前缀的宣告状态发生变化

时，系统首先依据新的控制面输入重新识别需要关注的目的前缀集合；随后，再结合起始自治系统范围，将这些变化映射为需要重新生成路径见证的覆盖对象集合。若某一前缀 p 被判定为控制面输入已发生变化，则所有与该前缀相关、且在当前版本覆盖范围内需要继续维护的对象 (p, s) ，都可被纳入候选受影响集合。

无论增量触发源为何种类型，其作用粒度均精确到覆盖对象级别。这一特性保证了覆盖对象间推演逻辑的正交性，即单个对象的更新状态不依赖于集合内其他对象，从而支持对覆盖对象集合进行分治处理。系统都会将最终的候选对象集合规范化为

$$S_v \subseteq \mathcal{Q}(v),$$

并在覆盖对象集合 $\mathcal{Q}(v)$ 上定义一个更新指示函数：

$$\delta_v(q) \in \{0, 1\}, \quad q \in \mathcal{Q}(v),$$

其中：

$$\delta_v(q) = \begin{cases} 1, & q \in S_v, \\ 0, & q \notin S_v. \end{cases}$$

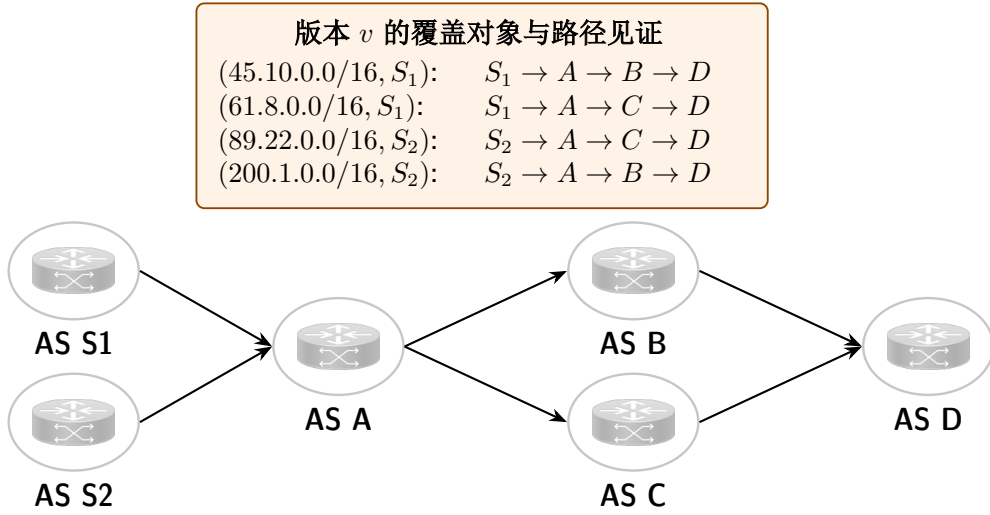
该指示函数直接表示对象 q 在本轮中是否需要重算：若 $\delta_v(q) = 1$ ，则系统对对象 $q = (p, s)$ 重新执行完整的语义推演；若 $\delta_v(q) = 0$ ，则直接复用上一版本结果。因此，生成期增量更新虽然仍在全体覆盖对象的统一框架中进行，但真正需要重新计算的仅是其中受影响的局部对象。

增量推演算法在保持全量语义规则的前提下，仅针对新增和受影响的对象执行增量更新，通过复用不非受影响对象的历史路径见证，减少计算和通信量从而达到降低时间开销的目的。

5.1.2 增量推演算法

增量推演算法在保持全量语义规则不变的前提下，仅针对新增对象和受影响对象执行增量更新，并通过复用未受影响对象的历史路径见证来减少计算与通信量，从而达到降低时间开销的目的。

本节结合具体示例说明生成期增量推演算法如何处理对象复用、局部重算与新增对象。设在版本 v 下，系统已经生成四个覆盖对象的数据平面语义见证，分别为 $(45.10.0.0/16, S_1)$ ， $(61.8.0.0/16, S_1)$ ， $(89.22.0.0/16, S_2)$ 和 $(200.1.0.0/16, S_2)$ 。


 图 5.1 版本 v 下的网络拓扑与覆盖对象示例图

如图 5.1 所示，这些对象的流量分别按不同路径转发：其中 $(45.10.0.0/16, S_1)$ 与 $(61.8.0.0/16, S_1)$ 从 S_1 出发， $(89.22.0.0/16, S_2)$ 与 $(200.1.0.0/16, S_2)$ 从 S_2 出发；对象的流量在经过 A 后分别沿分支 $A \rightarrow B \rightarrow D$ 或 $A \rightarrow C \rightarrow D$ 转发。

在版本 $v+1$ 中，设 $AS B$ 上的边界策略发生调整，如图 5.2 所示。具体而言， $AS B$ 新增了一条针对前缀 $200.1.0.0/16$ 的边界访问控制规则。围绕这一配置变更，增量推演算法的执行过程可分为如下四个步骤。

步骤 1：构造受影响对象集合与复用结果集。系统接收本轮配置变更后，首先依据变化规则的作用位置与作用范围识别受影响对象。设发生配置变更的自治系统为 u ，变化规则所作用的前缀范围记为 Δ_p ，上一版本中对象 $(p, s) \in \mathcal{Q}(v)$ 的对象级结果路径记为 $\text{Path}_v(p, s)$ 。对任意对象 (p, s) ，定义其在版本 $v+1$ 下的受影响判定为：

$$q_{v+1}(p, s) = \begin{cases} 1, & \text{若 } u \in \text{Path}_v(p, s) \text{ 且 } \text{overlap}(p, \Delta_p) = 1, \\ 0, & \text{否则.} \end{cases} \quad (5-1)$$

其中， $\text{overlap}(p, \Delta_p)$ 表示对象前缀 p 与变化规则作用范围 Δ_p 是否存在交集。只有当对象旧路径经过发生配置变化的自治系统，且其对应前缀空间与变化范围存在交集时，系统才将该对象判定为受影响对象。于是，受影响对象集合与复用结果集分别定义为：

$$S_v = \{(p, s) \in \mathcal{Q}(v) \mid q_{v+1}(p, s) = 1\}, \quad (5-2)$$

$$R_v = \mathcal{Q}(v) \setminus S_v. \quad (5-3)$$

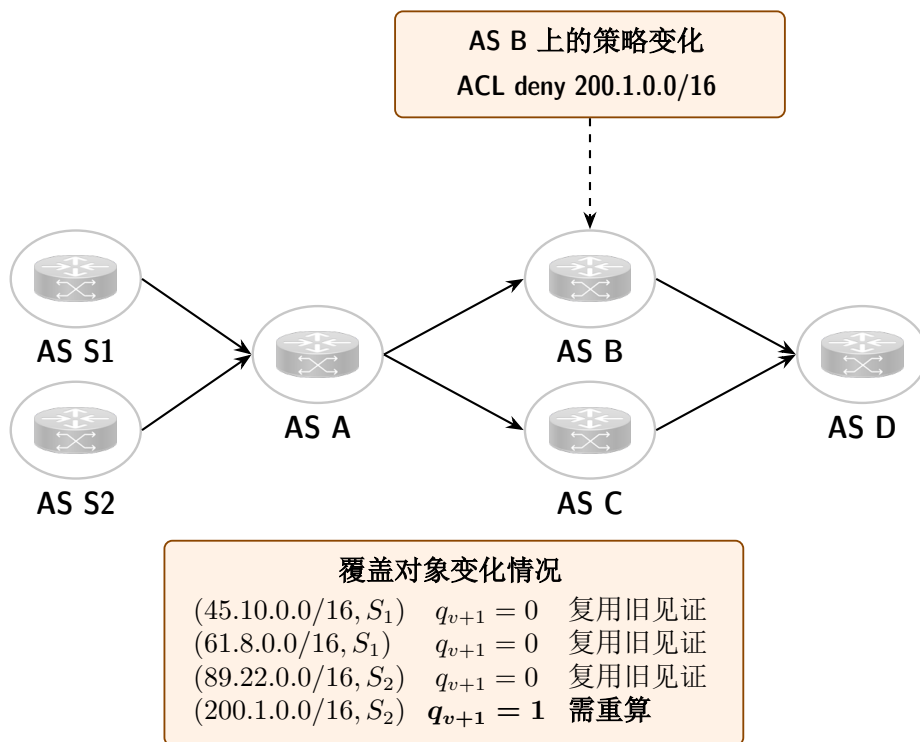


图 5.2 配置变更后的确认覆盖对象是否需要重算

相应的遍历识别过程如算法 4 所示。系统逐一检查上一版本中的覆盖对象：若对象旧路径经过受影响自治系统，且其前缀空间命中本轮变化范围，则将其加入受影响对象集合；否则，将其加入复用结果集。经过该步骤后，后续重算仅需作用于集合 S_v 中的对象，而集合 R_v 中的对象可直接继承上一版本结果。

在本例中，由于新增规则直接作用于前缀 200.1.0.0/16，且该前缀对应的对象 $(200.1.0.0/16, S_2)$ 的旧版本路径经过 AS B，因此图 5.2 中将其标记为 $q_{v+1} = 1$ ，表示该对象需要在版本 $v+1$ 中重新计算。其余对象 $(45.10.0.0/16, S_1)$ 、 $(61.8.0.0/16, S_1)$ 与 $(89.22.0.0/16, S_2)$ 的旧路径不经过受影响位置，或其前缀空间与本轮变化范围无交集，因此被标记为 $q_{v+1} = 0$ ，可直接复用上一版本的见证结果。

步骤 2：遍历对象并区分重算与复用。在受影响对象集合确定之后，系统对当前版本中的对象逐个判定，并按照图 5.2 下方给出的对象状态执行分流处理。对于满足 $q_{v+1} = 1$ 的对象，系统将其送入秘密分享域中的完整语义生成流程；对于满足 $q_{v+1} = 0$ 的对象，则直接复用旧版本结果并写入新版本输出。在本例中，仅对象 $(200.1.0.0/16, S_2)$ 进入重算路径，其余三个对象均沿复用路径直接写入版本 $v+1$ 的结果集合。

步骤 3：对受影响对象执行完整语义重算。对每个需要重算的对象，系统仍按

算法 4: 配置变更下的受影响对象识别算法

输入: 上一版本覆盖对象集合 $Q(v)$; 对象级旧结果路径集合 $\{\text{Path}_v(p, s)\}$;
 配置变更位置 u ; 变化规则作用前缀范围 Δ_p

输出: 受影响对象集合 S_v ; 复用结果集 R_v

```

1  初始化  $S_v \leftarrow \emptyset, R_v \leftarrow \emptyset$ ;
2  foreach  $(p, s) \in Q(v)$  do
3      if  $u \in \text{Path}_v(p, s)$  and  $\text{overlap}(p, \Delta_p) = 1$  then
4           $q_{v+1}(p, s) \leftarrow 1$ ;
5           $S_v \leftarrow S_v \cup \{(p, s)\}$ ;
6      else
7           $q_{v+1}(p, s) \leftarrow 0$ ;
8           $R_v \leftarrow R_v \cup \{(p, s)\}$ ;
9      end
10 end
11 return  $(S_v, R_v)$ ;
    
```

照第三章给出的逐对象流量切割与语义推演过程，在安全多方计算域中重新生成其数据面语义结果。具体而言，系统首先以受影响覆盖对象 (p, s) 为输入，在固定规则扫描轮数内对其对应的前缀空间执行逐步细化：若某条候选规则仅覆盖当前子空间的一部分，则触发流量空间切割，将该对象拆分为若干语义一致的细化子空间；若当前子空间与候选规则无交集，或当前子空间整体落在该规则覆盖范围内，则保持该子空间不变。在此基础上，系统再对每个细化对象分别执行规则命中消解、边界动作作用与逐 hop 路径推进，从而得到该对象在版本 $v + 1$ 下的新语义结果及其对应见证。

以本例中的 $(200.1.0.0/16, S_2)$ 为例，若该对象在重算后未发生进一步细化，则其对象级路径可直接由逐 hop 推演得到。旧版本下，其对应结果为 $S_2 \rightarrow A \rightarrow B \rightarrow D$ ；在版本 $v + 1$ 中，该对象到达 AS B 后受到新的边界动作影响而被拦截，因此更新后的对象级结果变为 $S_2 \rightarrow A \rightarrow B \rightarrow \text{DROP}$ 。图 5.3 展示了这一对象在版本演化前后的结果变化。

步骤 4: 写回新版本输出并形成版本 $v + 1$ 。在受影响对象完成重算之后，系统将重算结果与复用结果统一写入新版本输出集合 $\mathcal{W}_{v+1}$ 。对于未受影响对象，直接复用旧版本见证并按原有对象索引写回；对于受影响对象，则以步骤 3 中得到的新结果替换旧版本对应条目。由此，版本 $v + 1$ 的输出仍然是一个覆盖全部对象的完整语义结果集合。图 5.3 下方给出了本例中版本 $v + 1$ 的覆盖对象与路径见证： $(45.10.0.0/16, S_1)$ 、 $(61.8.0.0/16, S_1)$ 与 $(89.22.0.0/16, S_2)$ 保持不变，而 $(200.1.0.0/16, S_2)$ 则由原有路径更新为 $S_2 \rightarrow A \rightarrow B \rightarrow \text{DROP}$ 。这样，增量算法虽然只对局部对象执行完整重算，但对外形成的仍然是索引稳定、结构完整的新版本输出，从而可以直接作为后续承诺更新与 Merkle 树维护的输入。

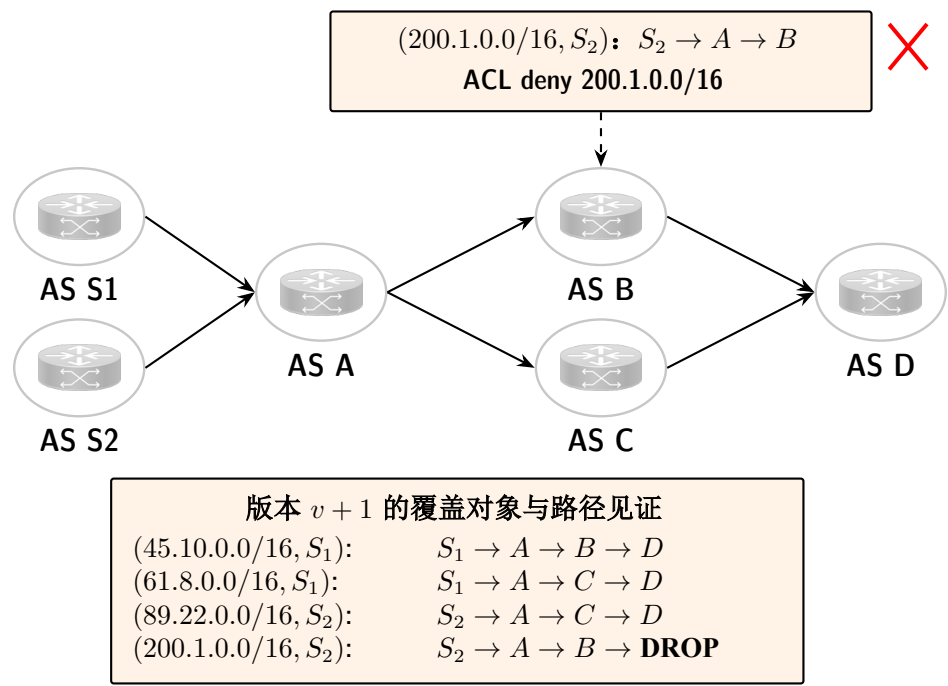


图 5.3 配置变更后的版本演化与新版本输出

此时，假设网络中有个 AS S_1 宣告了一个新的前缀 134.34.0.0/24，如图 5.4 所示。由于该对象并不存在于旧版本输出中，系统不涉及旧见证复用，而是将 (134.34.0.0/24, S_1) 直接纳入本轮待处理集合，并对其执行一次完整的秘密分享域语义推演。对于新增对象，系统首先依据控制面输入确定其在当前拓扑中的初始传播骨架，并据此生成对应的基础转发项。在本例中，该对象从起始自治系统 S_1 出发，其控制面可观测候选路径为 $S_1 \rightarrow A \rightarrow C \rightarrow D$ ，因此系统将这一路径所反映的传播方向作为新增对象进入生成期时的初始转发上下文。随后，系统在安全多方计算域中继续调用第三章定义的逐对象语义生成过程，并在每一跳结合当前自治系统的本地数据面规则块，对该对象执行局部细化、规则命中消解、边界动作作用与路径状态推进，从而得到版本更新后的对象级语义结果。

对象细化发生在推演过程中，而不是在整条路径结束后再统一处理。具体到本例，当对象 (134.34.0.0/24, S_1) 推演到 AS C 时，AS C 上存在更细粒度的前缀规则：其中，对子前缀 134.34.0.0/25 命中边界访问控制规则并被丢弃，而对子前缀 134.34.128.0/25 则不触发丢弃，仍可继续沿原有方向传播。于是，原始对象 (134.34.0.0/24, S_1) 在 AS C 处分化为两个互不重叠的子对象：(134.34.0.0/25, S_1) 与 (134.34.128.0/25, S_1)。其中，前者的结果为 $S_1 \rightarrow A \rightarrow C \rightarrow \mathbf{DROP}$ ，后者的结果为 $S_1 \rightarrow A \rightarrow C \rightarrow D$ 。

在最终输出阶段，系统不再将原始对象 (134.34.0.0/24, S_1) 作为单一条目写回，而是将其在 AS C 处分化得到的全部细化子对象及其对应见证统一写入新版本输出集合。与此同时，旧版本中未受影响的对象仍保持原有结果不变。由此，新增前缀对象的首次生成与既有对象的结果保留最终都被纳入统一的对象索引与输出组织框架之中，从而为后续承诺更新与 Merkle 树维护提供一致接口。

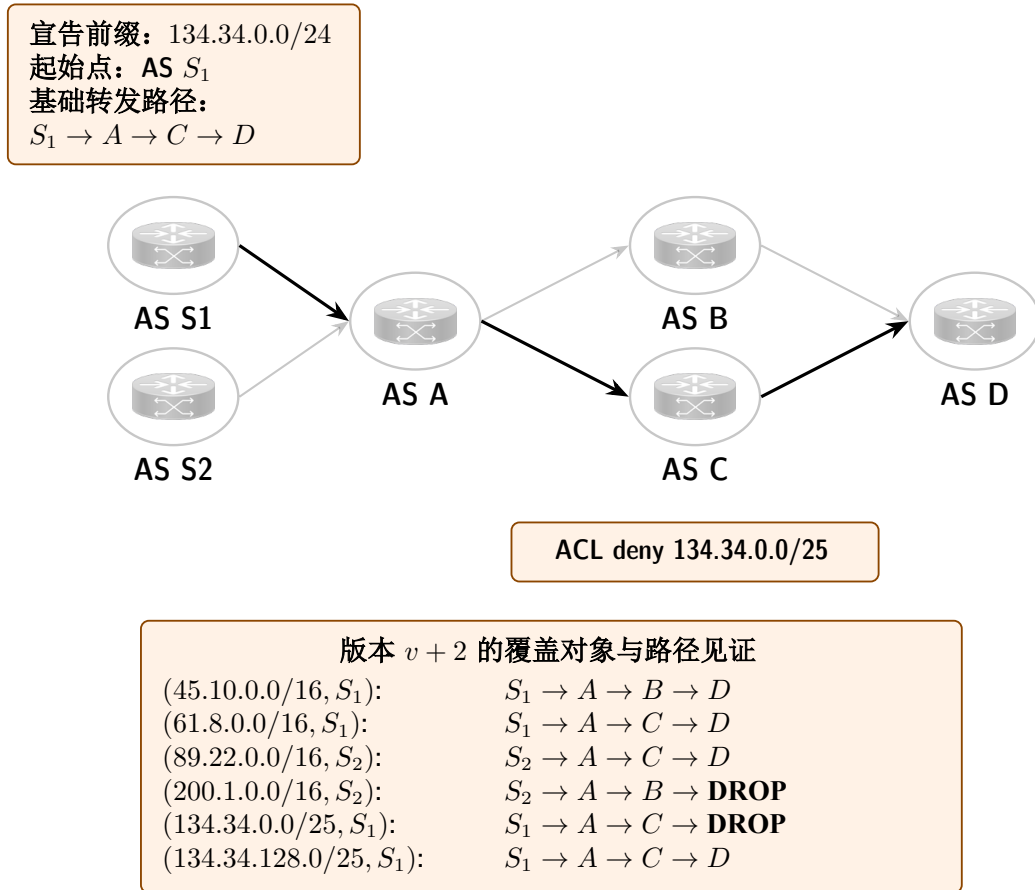


图 5.4 新增前缀对象场景下增量更新流程示例图

5.2 证据结构增量维护

上一节已经说明，生成期只对受影响覆盖对象重新执行语义推演，从而得到版本 $v + 1$ 下更新后的见证集合。然而，新的见证结果本身并不能直接作为查询期的可验证证据，因为外部验证者并不直接信任见证明文，而是依赖由叶子承诺、Merkle 聚合以及版本根共同构成的证据结构完成成员性验证与版本绑定。因此，在生成期完成局部重算之后，系统还需要对证据结构执行相应的增量维护：对受影响对象更新叶子承诺，并且沿相应路径重算内部节点，并发布新的版本根 R_{v+1} 。以

下将分为两个小节分别介绍如何对受影响对象更新叶子承诺 (§5.2.1) 和如何重算 Merkle 树内部节点并发布新的版本根 (§5.2.2)。

版本更新完成后, 系统还应将版本号与对应根值一并记录, 形成 $(v, R_v), (v + 1, R_{v+1}), \dots$ 这样的版本序列, 从而使后续查询能够显式绑定到某一具体版本, 并支持对修复前后结果的独立复核。

5.2.1 受影响叶子的承诺更新

本小节旨在探讨在确定受影响对象集合 S_v 后, Merkle 树叶子层级的更新策略。具体策略为针对属于集合 S_v 的受影响对象, 须在其预定义的索引位置执行状态更新, 以确保版本演进的一致性。对于不属于 S_v 的未受影响对象, 其对应的叶子节点保持现有状态, 本轮重算过程不涉及对该部分内容的冗余操作。

具体而言, 设版本 v 下的覆盖对象集合为 $Q(v)$ 。对任意对象 $(p, s) \in Q(v)$, 系统在版本 v 中已经生成见证 $W_{v,p,s}$, 并计算对应的叶子承诺 $C_{v,p,s} = \text{Commit}(W_{v,p,s})$ 。这些叶子承诺共同构成版本 v 的叶子层, 并进一步聚合为版本根 R_v 。

当系统从版本 v 过渡到版本 $v + 1$ 时, 上一节已经给出了受影响对象集合 $S_v \subseteq Q(v)$ 。对任意 $(p, s) \in S_v$, 生成期已经重新计算新的见证 $W_{v+1,p,s}$, 因此证据结构层只需进一步将该对象对应的叶子承诺更新为 $C_{v+1,p,s} = \text{Commit}(W_{v+1,p,s})$ 。相应地, 对任意 $(p, s) \in Q(v) \setminus S_v$, 由于见证未发生变化, 其叶子承诺保持不变, 即直接复用版本 v 下的结果。

从实现角度看, 系统并不重新排列叶子数组, 也不重新分配对象索引, 而是沿用第四章中已经固定的对象索引函数 $\text{idx}(p, s)$ 。因此, 若某个受影响对象 (p, s) 对应的索引为 $i = \text{idx}(p, s)$, 则系统仅在第 i 个位置写入新的叶子承诺值, 其余位置保持原样。

表 5.1 总结了叶子层在版本更新时的处理方式。因此, 受影响叶子的承诺更新本质上是一个局部替换过程: 系统只修改属于 S_v 的叶子位置, 而未受影响对象对应的叶子完全保留。这一步完成之后, 版本 $v + 1$ 下的叶子层已经确定; 接下来需要解决的问题, 就是这些局部叶子变化如何进一步传播到 Merkle 树内部节点, 并最终导出新的版本根。

表 5.1 版本更新时叶子层的处理规则

处理项	处理方式	叶子承诺结果
受影响对象 $(p, s) \in S_v$	在原索引位置写入 新承诺值	$C_{v+1,p,s} = \text{Commit}(W_{v+1,p,s})$
未受影响对象 $(p, s) \in Q(v) \setminus S_v$	保持原位置并复用旧值	$C_{v+1,p,s} = C_{v,p,s}$

5.2.2 Merkle 分支更新与新版本根计算

在叶子层完成局部替换之后，系统需要进一步将这些变化传播到 Merkle 树内部节点，并由此得到新的版本根 R_{v+1} 。这里的实现关键在于 Merkle 树的局部可组合性：任一内部节点仅由其左右子节点唯一确定，因此当某个叶子发生变化时，真正受到影响的只会是该叶子到根路径上的节点，而路径之外的其他子树保持不变。基于这一性质，系统无需基于全部叶子重新构造整棵树，而只需围绕受影响叶子所在的认证路径执行局部重算。

设某个受影响对象 $(p, s) \in S_v$ 的叶子索引为 $i = \text{idx}(p, s)$ ，其旧叶值为 $C_{v,p,s}$ ，新叶值为 $C_{v+1,p,s}$ 。系统首先读取该索引位置在版本 v 下对应的认证路径 $\text{auth}_{p,s}^{\text{old}}$ 。该认证路径由两部分组成：其一是从该叶子到根路径上各层的兄弟节点哈希值；其二是每一层的方向标记，用于指示当前节点位于左子树还是右子树。

在此基础上，系统按如下步骤执行局部更新：

步骤 1：定位受影响叶子并读取旧认证路径。系统根据对象索引函数 $\text{idx}(p, s)$ 定位受影响对象在叶子层中的位置，并从旧版本证据结构中读取该位置对应的认证路径 $\text{auth}_{p,s}^{\text{old}}$ 。由于对象到叶子位置的映射在版本演化过程中保持不变，因此这里不需要重新排列叶子数组，也不需要重新生成其他对象的认证路径。

步骤 2：验证旧位置属于旧树。为证明被修改的位置确实属于版本 v 所绑定的旧树，而不是任意伪造出的叶子位置。系统以旧叶值 $C_{v,p,s}$ 为起点，沿旧认证路径自底向上逐层计算父节点，直至得到旧根重构值。若记第 j 层的兄弟节点哈希为 $\text{sib}_j^{\text{old}}$ ，方向位为 dir_j ，则该重构过程可写为：

$$\begin{cases} h_0^{\text{old}} = C_{v,p,s} \\ h_{j+1}^{\text{old}} = \text{Hash}(\text{Sel}(\text{dir}_j, \text{sib}_j^{\text{old}}, h_j^{\text{old}}), \text{Sel}(\text{dir}_j, h_j^{\text{old}}, \text{sib}_j^{\text{old}})) \end{cases} \quad (5-4)$$

当递推结束后，系统检查最终结果是否等于旧版本根 R_v 。

步骤 3: 在同一位置替换为新叶值。在确认该位置属于旧树之后, 系统将同一索引位置上的旧叶值 $C_{v,p,s}$ 替换为新叶值 $C_{v+1,p,s}$ 。此时, 除该叶子到根路径上的节点可能发生变化外, 树中其余子树均保持不变。因此, 后续更新只需沿相同路径向上重算, 而无需处理路径之外的内部节点。

步骤 4: 沿相同路径逐层重算新根。为保证新根确实由该次局部叶子替换导出, 而不是与更新过程无关的任意根值。系统以新叶值 $C_{v+1,p,s}$ 为起点, 结合旧认证路径中的兄弟节点哈希与方向位, 再次自底向上逐层计算父节点, 得到更新后的新根重构值。若记该过程中的中间节点为 h_j^{new} , 则有:

$$\begin{cases} h_0^{\text{new}} = C_{v+1,p,s} \\ h_{j+1}^{\text{new}} = \text{Hash}(\text{Sel}(\text{dir}_j, \text{sib}_j^{\text{old}}, h_j^{\text{new}}), \text{Sel}(\text{dir}_j, h_j^{\text{new}}, \text{sib}_j^{\text{old}})) \end{cases} \quad (5-5)$$

递推结束后得到新根重构值 $R'_{v+1} = h_d^{\text{new}}$, 其中 d 为树高。系统进一步检查 R'_{v+1} 是否等于对外声明的新版本根 R_{v+1} 。

步骤 5: 对多个受影响叶子重复执行并合并结果。若本轮存在多个受影响对象, 则系统对每个对象分别执行上述旧树定位, 新叶替换和路径重算过程。对于共享内部节点的受影响路径, 系统只需保留最终一次重算后的节点值即可; 对于不相交路径, 则分别更新对应子树。全部受影响路径处理完成后, 根节点的最终值即记为版本 $v+1$ 的公开根 R_{v+1} 。

表 5.2 总结了这一局部更新过程在树结构不同层次上的处理方式。由此, 版本 $v+1$ 的根值不是通过整棵树重建得到的, 而是通过受影响叶子原位替换和认证路径逐层重算得到的。

通过增量更新 Merkle 树可以有效更新设版本 v 下的覆盖对象全集为 $Q(v)$, 其对应的 Merkle 树叶子数为 $|Q(v)|$; 若该树高度为 $h = O(\log |Q(v)|)$, 且本轮受影响对象数量为 $|S_v|$, 则对每个受影响叶子执行一次从叶子到根的路径重算, 其代价为 $O(h)$; 因此, 证据结构增量维护的总代价为:

$$O(|S_v| \cdot h) = O(|S_v| \log |Q(v)|).$$

相比之下, 若采用全量更新方式, 则系统需要基于全部叶子重新构造版本 $v+1$ 的整棵 Merkle 树, 其代价与叶子总数线性相关, 可记为:

$$O(|Q(v)|).$$

因此，当受影响对象数量满足 $|S_v| \ll |Q(v)|$ 时，局部更新方式能够显著降低证据结构维护开销。

表 5.2 Merkle 树局部更新时的处理规则

节点类别	处理方式	是否重算
受影响叶子对应的节点	用新叶子承诺替换旧值	是
受影响叶子到根路径上的内部节点	根据更新后的子节点值逐层重算	是
路径之外的其他子树节点	保持旧版本中的节点值不变	否

5.3 安全性分析

第五章的增量维护机制未引入新的密码学原语，而是在第三章生成期隐私协作计算与第四章查询期零知识审计接口的基础上，对版本演化过程进行了局部化组织。因此，本章的安全性结论可归约为前两章既有结论：生成期局部重算延续第三章的输入隐私与执行轨迹隐私，查询期围绕新版本根生成的审计证明延续第四章的零知识隐私性。

引理 5-1. 设版本 $v \rightarrow v+1$ 的生成期增量更新仅对受影响对象集合 $S_v \subseteq Q(v)$ 执行局部重算，且对每个 $(p, s) \in S_v$ 仍调用第三章定义的秘密分享域语义生成过程。若计算实体的合谋规模严格小于重构阈值 t ，则该局局部重算过程除更新后见证集合所必然泄露的信息外，不会额外泄露任一自治系统的本地明文配置。

证明. 局部重算仅将执行对象范围由 $Q(v)$ 缩小为 S_v ，并未改变第三章生成期协议的输入共享方式与数据无关执行过程。因此，第三章引理 3-1、引理 3-2 及定理 3.3.1 仍然适用，结论成立。 $\square$

引理 5-2. 设证据结构更新后得到新的公开版本根 R_{v+1} ，查询期仍按第四章定义的审计关系

$$\mathcal{R}((R_{v+1}, p, s, \Phi, y), \mathbf{w}) = 1$$

构造零知识证明，其中私有见证 $\mathbf{w}$ 包含对象 (p, s) 的语义见证及其认证路径。若所采用的证明系统满足第四章引理 4-1 的标准性质，则围绕 R_{v+1} 的查询期审计证明仍不泄露关于私有见证的额外信息。

证明. 增量维护仅将公开输入中的版本根由 R_v 更新为 R_{v+1} ，并未改变第四章中的证明系统、公开输入与私有见证划分，以及关系 $\mathcal{R}$ 的构造方式。因此，第四章引理 4-1 与定理 4.4.1 仍然适用，结论成立。 $\square$

定理 5.3.1. 在半诚实模型下，若生成期局部重算继续采用第三章的秘密分享与数据无关执行框架，查询期审计继续采用第四章的零知识证明接口，则第五章提出的版本化维护与增量更新机制不会降低系统既有的安全标准。具体而言，生成期仍满足第三章的输入隐私与执行轨迹隐私，查询期仍满足第四章的零知识隐私性。

证明. 由引理 5-1 与引理 5-2 直接得到。由于第五章未引入新的密码学原语或新的公开泄露接口，故其整体安全性可归约为第三章与第四章既有安全结论的组合。 $\square$

5.4 本章小结

本章聚焦于版本化维护与增量更新机制，旨在解决系统在长期运行中如何以低成本持续维护数据面语义证据。本章首先提出了生成期的增量更新机制，通过监控配置变更与前缀输入变化这两类驱动方式，定义受影响的对象集合。并详细介绍如何对该集合执行隐私语义推演，而对其他未受影响的对象直接复用上一版本的见证结果。随后，本章进一步探讨了证据结构层的增量维护方式。AegisPath 仅需更新受影响对象对应的叶子承诺，并沿 Merkle 路径自底向上同步节点状态，从而高效锚定新的公开版本根 R_{v+1} 。这一设计确保了版本根能持续绑定最新的全局见证状态，同时避免了全量重建证据树的开销。最后，本章从安全性层面分析了增量维护机制。因此，本章所提出的版本化维护机制为系统面向长期运行、持续修复与重复审计的实际部署提供了关键支撑。

第6章 系统实现与性能评估

在前文介绍 AegisPath 的系统架构、生成机制、零知识审计接口以及版本化维护方法之后，本章进一步给出 AegisPath 的原型实现与实验评估。本章首先介绍原型系统的实现 (§6.1)，然后介绍实验环境、实验所使用的数据集以及对比工具 (§6.2)。随后，本文在实验室的测试环境中部署 AegisPath 原型系统，并围绕其生成期、查询期与版本维护阶段开展实验分析。

结合本文的研究目标，本章重点回答以下三个问题：

- 1) 生成期的隐私协作计算在不同规模网络中的开销如何？ (§6.3)
- 2) 查询期零知识证明审计与验证时的开销如何？ (§6.4)
- 3) 版本化与增量更新机制在局部变化场景下能否显著降低维护开销？ (§6.5)

围绕上述问题，本章将分别从生成期性能、查询期审计开销以及增量维护收益三个方面，对 AegisPath 的实际运行代价与可扩展性进行系统评估。

6.1 系统实现

AegisPath 的原型实现分为生成期与查询期两部分。生成期负责在秘密共享条件下生成覆盖对象对应的数据平面语义见证，查询期负责围绕版本根、Merkle 认证路径与路径不变量输出可公开验证的证明。基于这一划分，本文分别对生成期与查询期的底层框架进行了选型比较。

对于生成期，本文主要从协议模式、安全计算能力与工程能力三个方面进行考察。比较结果如表 6.1 所示。

表 6.1 生成期 SMPC 框架选型比较

候选框架	协议模式	安全计算原语	工程能力
ABY3 ^[89]	三方协议	混合共享	场景专用
CrypTen ^[90]	多方计算	张量导向	任务定向
SecretFlow-SPU ^[91]	多方计算	张量导向	任务定向
MpyC ^[92]	多方计算	能力完备	批量受限
MP-SPDZ ^[47]	多方计算	能力完备	工程成熟

ABY3^[89] 主要面向固定三方场景，其协议组织方式与本文按门限秘密共享开

展多方语义生成的设定并不一致。CrypTen^[90]与SecretFlow-SPU^[91]更偏向张量化隐私计算与模型相关任务，对于本文所需的前缀匹配、最长前缀匹配优先级消解、边界策略处理与逐跳路径推进等逻辑，支撑方式不够直接。MpyC^[92]支持安全整数算术、比较与位运算等能力，原型实现较为灵活，但在大规模批量输入、固定长度循环与多轮执行场景下，工程控制能力相对有限。相比之下，MP-SPDZ^[47]能同时支持算术与二进制共享，并可围绕比较、条件选择与控制流组织程序实现，更适合作为本文生成期SMPC的底层框架。

对于查询期，本文主要从电路表达、证明接口与工程集成三个方面进行考察。比较结果如表6.2所示。

表 6.2 查询期零知识证明框架选型比较

候选框架	电路表达	证明接口	工程集成
Circom ^[93]	专用语言	链路复杂	封装繁多
arkworks ^[94]	底层库式	接口灵活	实现复杂
bellman ^[95]	底层库式	接口灵活	实现复杂
halo2 ^[96]	底层库式	接口灵活	实现复杂
gnark ^[97]	程序组织	接口完整	集成友好

Circom^[93]采用专用电路语言描述约束系统，在原型系统中通常需要维护电路描述、见证生成与证明调用之间的完整处理链路。arkworks^[94]、bellman^[95]与halo2^[96]具有较强灵活性，但整体上更偏底层库式开发，从原型实现角度看，其工程组织与调试复杂度相对更高。相比之下，gnark^[97]便于围绕结构化见证对象组织电路，并能较好衔接证明生成、验证与序列化流程，更适合与本文的版本根构造、认证路径生成和查询服务模块统一实现。

据此，AegisPath在原型实现上形成了生成期采用MP-SPDZ^[47]、查询期采用gnark^[97]的分层技术路线。前者负责在隐私约束下生成版本化数据面语义见证，后者负责围绕已承诺见证输出与版本根绑定的可公开验证证明。

在实现语言与模块划分方面，AegisPath采用Python、Go、MP-SPDZ的多语言实现方式。其中，Python主要负责数据集处理、输入构造、配置转换与实验辅助脚本；MP-SPDZ部分负责生成期秘密共享输入上的数据平面语义推演；Go语言主要用于实现查询期的版本根构造、Merkle路径生成、零知识证明生成与验证接口；按当前原型统计，MP-SPDZ代码约为2500行，Go代码约为4000行，Python

代码约为 3900 行，此外，系统还包含约 600 行 Shell 脚本，主要负责跨模块的编译优化。

从模块功能上看，原型系统主要包括以下四个部分：

- **数据预处理与输入构造模块：**负责将公开控制面输入与本地配置整理为生成期所需的整数化中间表示，并生成秘密共享输入文件。
- **生成期 SMPC 计算模块：**在 MP-SPDZ 中实现第三章的数据平面语义生成算法，针对覆盖对象 (p, s) 生成对应的路径见证。
- **版本承诺与证明生成模块：**在 Go/gnark 中实现叶子承诺计算、Merkle 根构造、认证路径生成以及查询期零知识证明逻辑。
- **实验自动化模块：**负责组织不同拓扑规模、不同参数设置以及不同更新场景下的批量运行与结果记录。

6.2 实验设置

6.2.1 测试环境

本实验在实验室的多机环境中完成，部署方式按照 AegisPath 的目标运行形态进行组织。具体而言，生成期采用分布式部署：各自治系统对应的输入预处理与秘密共享输入分布在不同节点上，SMPC 计算节点运行于至少 5 台服务器组成的实验网络中，用于模拟多管理域场景下各参与方围绕共享输入联合生成数据平面语义见证的过程。查询期采用与 RPKI 相近的中心托管与发布、外部依赖方独立验证的部署方式：证明与托管服务部署于独立服务器上，负责保存见证、计算承诺、维护版本根并按查询生成零知识证明；外部依赖方仅获取公开版本根、查询参数与证明，并在本地独立完成验证，而不需要重新参与生成期的多方计算。

实验中使用的服务器为同构配置：每台服务器配备双路 Intel Xeon Gold 6138 处理器，共 40 个物理核心、80 个逻辑核，主频为 2.00 GHz，最大睿频可达 3.70 GHz，内存为 93 GiB；操作系统为 Ubuntu 22.04.5 LTS，内核版本为 Linux 5.15.0-157-generic；网络接口为 Intel X722 系列以太网控制器，支持 10 GbE/1 GbE 连接。通信组件方面，生成期多方计算依赖 MP-SPDZ 运行时提供的网络通信机制在各计算节点之间交换秘密共享数据；查询期的证明生成与验证模块基于 Go/gnark 实现，其中证明与托管服务器同时承担见证存储与私有输入注入两项功能：一方面，本地存储模块负责保存版本化见证及其对应的 Merkle 认证路径；另一方面，证明服务进程在

接收到查询后，从本地见证库中读取相应版本下的见证 $W_{v,p,s}$ 及认证路径 $\text{auth}_{p,s}$ ，并在内存中组装为 gnark 电路所需的私有见证（private witness），随后完成零知识证明生成。外部依赖方只需获取公开版本根、查询参数与证明文件，即可在本地执行验证过程。

6.2.2 数据集

为了获取公开且可观测的控制面信息，本文联合使用 RouteViews 归档数据与 CAIDA 发布的 pfx2as 数据集。RouteViews 提供路由表快照与更新归档，可作为控制面宣告和候选自治系统路径的公开来源^[17, 98]；CAIDA 的 pfx2as 数据集提供前缀到起源自治系统的映射信息，可用于前缀归属标准化与起源判定^[99]。两类数据在本文中的作用如表 6.3 所示。

表 6.3 实验中公开数据来源及其作用

数据来源	提供内容	本文用途
RouteViews ^[17, 98]	路由表快照，更新归档	提取前缀宣告与候选路径
CAIDA pfx2as ^[99]	前缀归属映射	进行起源判定与归属标准化

在拓扑组织上，本文按照自治系统层级关系构造分层实验拓扑。具体而言，本文优先选取 Tier-1 自治系统作为骨干层节点，再从与这些 Tier-1 自治系统存在连接关系的自治系统中选取 Tier-2 节点作为中转层，随后进一步选取与上述 Tier-2 节点相连的 Tier-3 节点作为接入层。通过这种方式，实验拓扑同时包含骨干转发、中间传递与边缘接入三类结构角色，从而能够较好地刻画多管理域网络中的层次化连接关系。

基于上述分层拓扑与公开控制面观测结果，本文进一步为各节点构造基础转发表。这里的基础转发表刻画的是公开控制面所能反映的默认转发骨架，即在不考虑自治系统内部私有策略干预时，由公开宣告、候选路径与起源归属共同决定的基本转发关系。RouteViews 与 CAIDA 数据共同提供了这一路径骨架的公开依据，本文据此获得实验环境中的基础可达关系与候选转发方向。

公开数据能够揭示的主要仍是前缀宣告、候选路径与可达关系等控制面信息，无法直接反映运营者内部实际部署的数据面规则。边界黑洞、访问控制与策略重定向等机制通常属于本地运维配置，其触发条件、作用范围与施加对象不会直接

体现在公开 BGP 观测中。基于这一差异，本文在基础转发表之上进一步叠加黑洞、访问控制与策略重定向规则，以构造更贴近现实的数据面环境。

本文采用较低比例的选择性附加策略，在基础转发骨架上叠加私有数据面规则。这样处理有两方面原因。其一，访问控制规则在工程实践中通常需要控制规模。已有研究指出，ACL 规则规模增长会带来额外的性能开销与管理复杂度，因此不会对全部对象持续铺设细粒度规则^[100-101]。其二，黑洞缓解、策略引流与访问控制更接近面向局部对象的例外性配置，其部署通常围绕局部冲突、异常流量或特定防护需求展开。若在实验中对全部对象大比例叠加私有规则，公开控制面骨架与局部私有偏离之间的层次关系将被削弱，不利于刻画现实网络中控制面可观测而数据面受局部私有策略影响的场景。

基于上述考虑，本文采用比例化的规则注入方式。具体设定如表 6.4 所示。这里的比例描述的是实验中的规则注入参数，而非对真实网络规则分布的统计刻画。

表 6.4 实验中附加数据面规则的注入比例设定

规则类型	注入比例
黑洞规则	10%
访问控制规则	7%
策略重定向规则	3%
合计	20%

上述设定使实验输入同时包含两部分信息：一部分来自 RouteViews 与 CAIDA 所提供的公开控制面观测结果，用于确定基础转发骨架；另一部分来自在该骨架之上按比例附加的私有数据面规则，用于模拟现实网络中由局部运维策略引入的数据面语义偏离。基于这样的输入组织方式，后续实验既能够保持控制面来源的公开可解释性，又能够体现多管理域网络中数据面语义受私有配置影响的特征。

6.2.3 对比工具

多管理域隐私保护数据平面审计作为一个新兴的研究方向，目前学术界与工程界均缺乏能够直接承担同等任务的同类开源系统。因此，本文的实验对比需要结合任务目标分别设置。现有可参考的方法大体可以分为两类：一类是经典的数据平面验证工具，另一类是少量隐私保护下的跨域验证方法。前者在算法模型上默认验证方能够在统一明文全局视图下集中获取网络配置、策略规则与转发表，并

据此执行符号推演、依赖传播或增量检查；其核心计算过程并非围绕门限分片、密文共享或数据无关执行而设计，因而无法直接迁移到本文所关注的多管理域、互不信任场景。即使在实验室环境中集中运行，这类工具也不提供面向外部审计者的可验证证据接口，因而不能满足事故复盘、责任划分与持续询证场景中的审计需求。

在现有少量隐私保护下的跨域验证方法中，IVeri^[5]与Seguall^[6]是与本文最接近的两类代表工作。IVeri的核心算法建立在控制面语义之上，其验证对象主要围绕路由传播、候选路径与控制面收敛结果展开，本质上仍以路由表或由控制面导出的转发结果作为验证依据；这一语义层级无法直接迁移到本文所关注的真实数据平面语义生成任务中。Seguall则以AS级转发表为输入，构建转发图并基于安全多方计算检查可达性、前缀挟持和路由泄露等性质。但Seguall无法应对因访问控制列表（ACL）、黑洞或策略重定向等引起的流量切割问题，因此其输出的结果可能在这些情境下不准确。由于这些限制，Seguall与IVeri在语义层级上有所不同，无法直接应用于本文所需的对象级数据平面语义生成任务。基于上述分析，由于当前尚无系统能够在多管理域隐私保护下，完整建模边界过滤与策略规则并生成对象级数据平面语义见证，因此数据平面语义见证生成模块无法设置同类基线进行对比。

查询期的对比基线设置为基于Seguall^[102]提出的安全多方计算查询方案。考虑到其开源实现中存在可能影响隐私约束的实现细节，本文在复现该基线时对相关部分进行了数据无关化处理，以避免实现层面的信息泄露行为影响比较结果的影响。与此同时，为排除生成期语义来源不同所带来的干扰，本文在查询期实验中统一使用AegisPath生成的数据平面语义结果，并分别输入到AegisPath的零知识证明查询流程与Seguall的安全多方计算查询流程中执行。由此得到的对比结果能够更集中地反映，在相同语义输入和相近查询目标下，基于零知识证明的在线审计机制相对于Seguall的基于安全多方计算的在线查询机制在时间开销上的性能对比。

增量更新机制的对比基线设置为状态变化后执行全量重算的方案。该基线在每次配置变更或路由更新后，重新对全部覆盖对象执行完整语义生成，并重建整棵版本化证据树；本文方法则仅对受影响对象执行局部重算，并同步更新相应的Merkle分支与版本根。通过这一对比，可以衡量增量维护机制在频繁局部变化场景下相对于全量重算所带来的时间收益

6.3 实验一：隐私协作计算的性能表现

本实验评估 AegisPath 在生成期执行完整 SMPC 数据平面语义生成时的总体开销与可扩展性，重点回答三个问题：其一，完整生成期能否在不同规模的多管理域网络中稳定输出覆盖对象上的见证结果；其二，运行时间如何随拓扑规模与前缀数量的增长而变化；其三，在生成期后续的对象级 MiMC 承诺与版本根构造过程中，单次承诺计算与聚合操作相对于完整 SMPC 生成的总体开销处于何种量级。

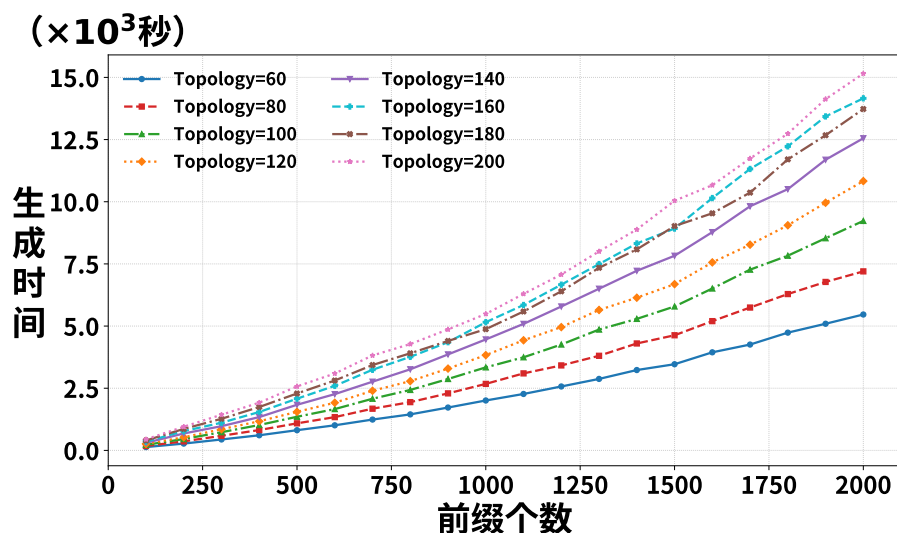


图 6.1 生成期隐私协作计算的时间开销

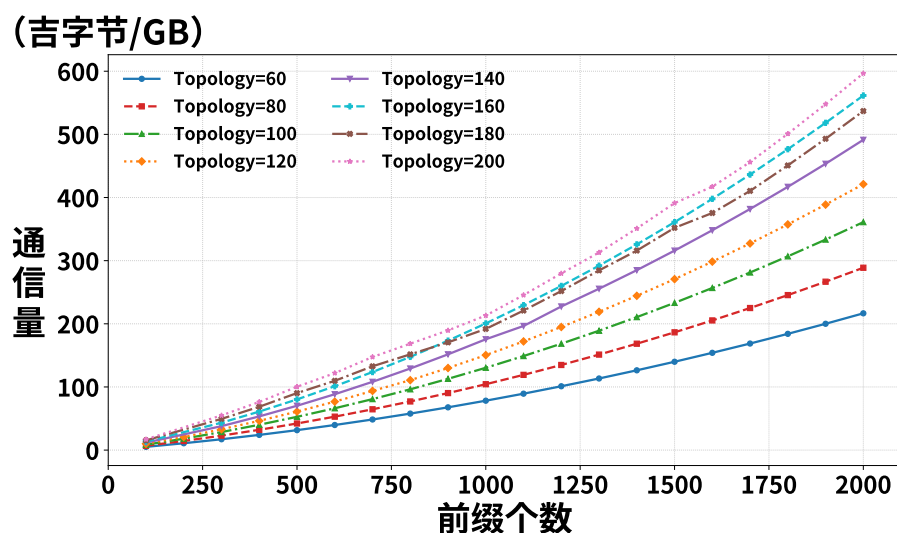


图 6.2 生成期隐私协作计算的通信量

图 6.1 展示了不同拓扑规模下完整生成期运行时间随前缀数量变化的结果。结果表明，在网络拓扑规模为 60-200 个自治系统、前缀数量为 100-2000 的范围内，

完整生成期均能够稳定完成见证生成。这说明，本文第三章提出的共享域数据平面语义推演过程不仅在较大规模的多管理域网络拓扑上具有可执行性，而且能够稳定支持覆盖对象上的见证输出。同时，生成期总耗时随前缀数量增加而显著上升，且网络拓扑规模越大，增长趋势越为明显。例如，在 60 个自治系统规模的网络拓扑下，运行时间大致由约 1.5×10^2 秒增长至约 5.4×10^3 秒；而在 200 个自治系统规模的网络拓扑下，运行时间则由约 3.5×10^2 秒增长至约 1.5×10^4 秒。其余拓扑规模下的曲线也呈现一致的增长规律，表明完整安全多方计算生成的主要开销同时受到前缀规模与拓扑规模的共同影响。

图 6.2 进一步展示了完整生成期在不同规模条件下的通信量变化情况。结果显示，随着前缀数量与拓扑规模同步增长，全局通信量呈现显著上升趋势。在较小规模配置下，通信量已达到 10 GB 量级；当前缀数量增至 2000 时，不同拓扑规模下的通信量整体进入 100 GB 量级。其中，在 60 个自治系统规模的网络拓扑下，累计通信量约为 220 GB；而在 200 个自治系统规模的网络拓扑下，该值约为 610 GB。该图与图 6.1 的变化趋势基本一致，说明完整生成期总时间的增长不仅源于本地计算规模的扩大，更与多方协议在共享域中的持续在线交互及由此带来的通信膨胀密切相关。

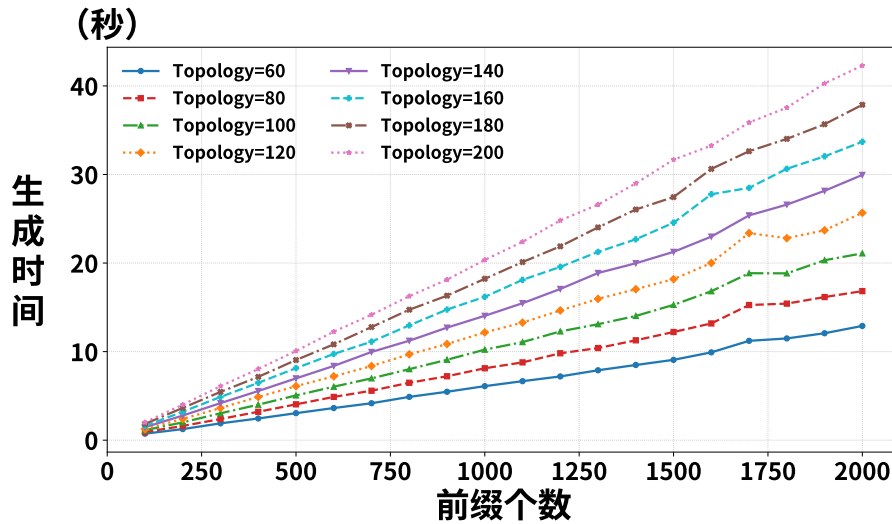


图 6.3 生成期构造 MiMC 承诺和版本根的时间开销

图 6.3 给出了对象级 MiMC 承诺与版本根构造阶段的时间开销。可以看到，随着前缀规模增加，该阶段的耗时虽有所上升，但整体仍显著低于完整生成期的总时间。即使在较大规模拓扑下，其总耗时也仅处于几十秒量级，而在较小规模拓扑下通常仅为数秒至十余秒。与图 6.1 中完整生成期达到数千秒乃至数万秒的总

耗时相比, 该部分代价明显较低。这表明, 在当前实现条件下, 生成期的主要性能瓶颈并不在于单次 MiMC 承诺计算或 Merkle 树聚合本身, 而主要来自共享域内对象级语义推演、多方交互过程及其引发的通信开销。并且对象级 MiMC 承诺与版本根构造过程属于生成期的一次性预处理操作, 其执行时机发生在验证对象生成阶段, 而非查询到来后的在线验证阶段。该过程并不会在每次查询时重复执行, 因此不会直接影响查询期的在线查询的性能开销。

综合图 6.1、图 6.3 与图 6.2 的结果可以看出, 完整 SMPC 方案能够为 AegisPath 提供正确的离线见证生成能力, 但其主要代价集中在共享域内的数据平面语义推演与多方协议执行阶段, 而非后续的对象级承诺与版本根构造阶段。这一结果具有两方面含义。一方面, 完整生成期在较大规模多管理域网络拓扑上仍可稳定执行, 说明基于 SMPC 的多管理域数据平面语义见证生成路线具有现实可实现性。另一方面, 随着网络规模进一步扩大, 完整生成期的总耗时与通信量均增长较快, 而承诺与聚合阶段仅占较小比例。这意味着, 若将完整生成过程直接作为可重复调用的查询接口, 则难以满足运行期审计对低时延与多轮复验的要求。基于这一观察, AegisPath 将完整 SMPC 生成定位为离线阶段的一次性高开销计算, 而非供查询期反复调用的在线接口。

总体而言, 实验一表明, 完整 SMPC 生成能够在较大规模多管理域网络拓扑上稳定输出见证结果, 但其时间开销与通信开销均较为显著, 可扩展性仍受到一定限制, 因此更适合作为离线见证生成机制。同时, 对象级 MiMC 承诺与版本根构造所引入的额外耗时相对较小, 说明后续版本化承诺过程不会成为生成期的主导性能瓶颈。

6.4 实验二：零知识证明审计的性能表现

本实验评估 AegisPath 在查询期采用零知识证明进行审计时的性能开销, 并将 Segull 中完整多方计算的查询作为基线进行对比, 重点回答以下四个问题: 第一, AegisPath 查询期中证明生成的时间开销处于什么量级; 第二, AegisPath 查询期中验证阶段的时间开销处于什么量级; 第三, Segull 的安全多方计算查询方案, 其时间开销和通信量会随拓扑规模与前缀数量如何增长; 第四, AegisPath 在查询期的时间开销和 Segull 方案时间开销的性能比较。

AegisPath 在查询期采用零知识证明机制。图 6.4 给出了查询期证明者生成证明的时间开销。可以看到, 随着前缀数量从较小规模逐步增加到 2000, 不同拓扑

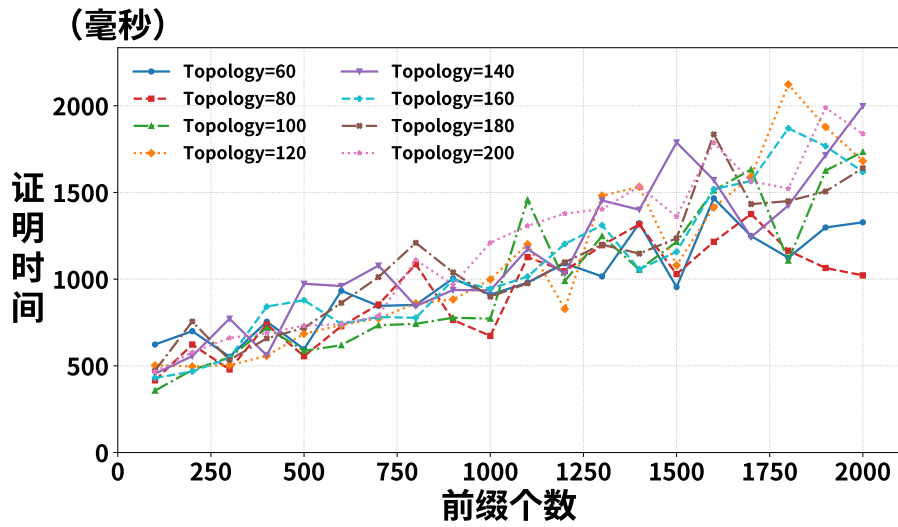


图 6.4 查询期证明者生成证明的时间开销

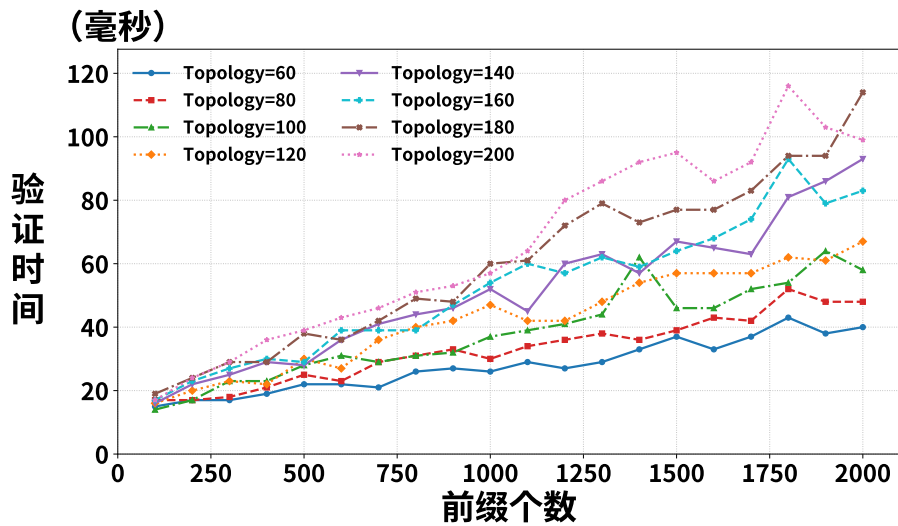


图 6.5 查询期时审计者进行验证的时间开销

下的证明生成时间整体呈上升趋势，但存在一定波动：在较小拓扑和较少前缀时，证明时间通常处于数百毫秒量级；当前缀规模与拓扑规模进一步增大后，证明时间大致上升到 1 秒-2.1 秒的范围内。整体来看，即使在较大拓扑和较大前缀规模下，证明生成仍稳定保持在秒级以内或接近 2 秒的范围。图 6.5 给出了查询期审计者执行验证的时间开销。与证明生成阶段相比，验证端开销明显更低，在不同拓扑与前缀规模下总体约处于 15 毫秒-115 毫秒的范围内。虽然验证时间同样会随前缀数量和拓扑规模增加而缓慢上升，但整体增长幅度远小于证明生成阶段，始终保持在百毫秒以内。

图 6.6 进一步给出了查询期证明者所对应电路的约束数量。可以看到，不同拓

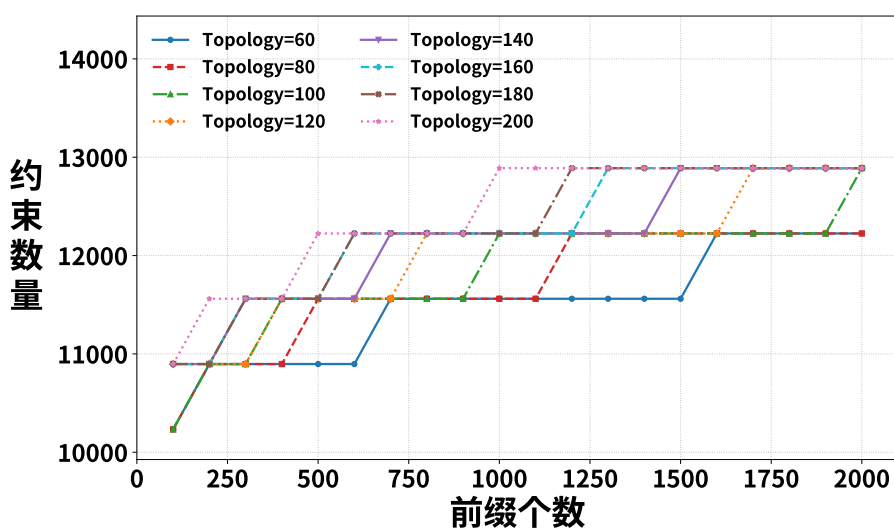


图 6.6 查询期时证明者的电路约束数量

扑与前缀规模下的约束数整体集中在约 10^4 到 1.3×10^4 之间，并随前缀数量增加呈现缓慢增长后逐渐趋于平稳的趋势，没有出现随前缀规模扩张而显著膨胀的现象。这说明，查询期电路复杂度主要受固定证明逻辑支配，前缀规模增长对约束规模的附加影响相对有限。其原因在于，见证数量对证明期电路规模的影响并不是直接线性体现的，而是主要通过影响 Merkle 树高度间接产生作用；而树高度随叶子数量的增长本身是对数级的，因此在一段输入规模范围内，即使见证数量继续增加，电路约束数量也可能保持不变，只有当见证规模增长到足以使 Merkle 树增加一层时，约束总量才会出现较明显的上升。

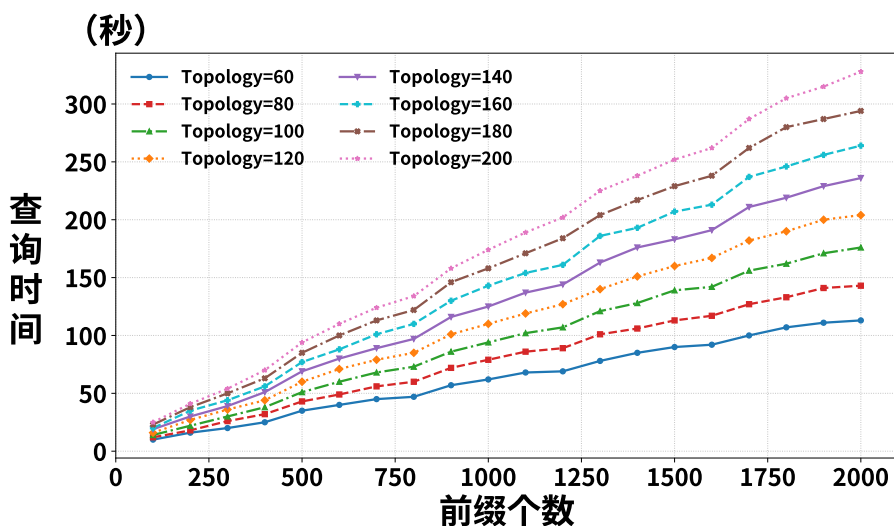


图 6.7 Segull 查询方案的时间开销

图 6.7 和图 6.8 分别给出了 Segull 在查询期重新组织完整安全多方计算时的

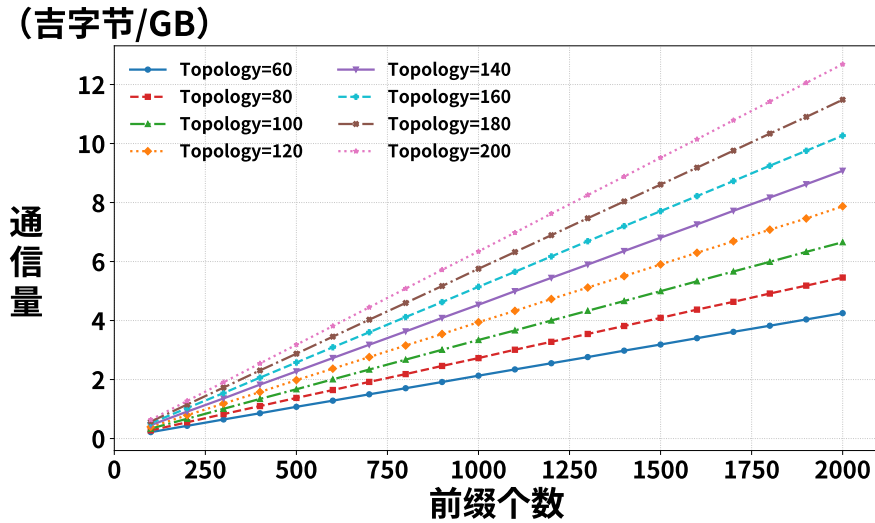


图 6.8 Seguall 查询时产生的通信发送量

单次查询时间与通信量。可以看到，随着前缀数量和拓扑规模同时增加，两项开销均呈现稳定上升趋势：在拓扑规模为 60 个 AS 时，单次查询时间由约 12s 增长至约 113s，通信量由约 0.3GB 增长至约 4.2GB；在拓扑规模为 200 个 AS 时，单次查询时间进一步上升至约 327s，通信量接近 12.7GB。两张图的变化趋势基本一致，说明当每次查询都重新在共享域内执行完整审计逻辑时，系统开销会随网络规模 and 对象规模同步放大，其中主要瓶颈并不在局部算术操作本身，而在于多方协议持续在线交互所带来的通信与同步成本。

图 6.10 和图 6.9 给出了 AegisPath 查询期与 Seguall 查询方案在时间开销上的对比结果，其中 Seguall 在每次查询时都需要重新执行一次完整的多方计算。由累积分布曲线可以看到，AegisPath 的查询时间整体集中在横轴左端的极小区间内，全部样本均落在约 2.1 秒以内；相比之下，Seguall 的查询时间分布则明显向右扩展，从约 12 秒一直延伸至约 330 秒。也就是说，AegisPath 的单次查询基本稳定在秒级以内，而 Seguall 普遍处于十秒到数百秒量级，说明在全部测试样本上，AegisPath 的查询时延整体显著低于 Seguall。结合图 6.9 可进一步看到，在不同拓扑规模和前缀规模下，AegisPath 相对于 Seguall 的加速比均保持在较高水平：在拓扑规模为 60 个 AS 时，加速比大致由十余倍逐步提升至约 90 倍；拓扑规模为 80 个 AS 时，最高可达约 140 倍；拓扑规模为 100-140 个 AS 时，大多数配置下稳定在约 100-150 倍区间；当拓扑规模进一步增长至 160-200 个 AS 时，加速比进一步升高，在部分配置下达到约 180-200 倍，其中最大值接近 200 倍。总体来看，随着网络规模和前缀数量增加，Seguall 因为需要在查询期重新执行完整多方计算，其开销增长更快；

相比之下，AegisPath 将主要计算负担前移到离线生成阶段，在查询期仅执行证明生成与验证，因此能够持续保持显著的性能优势。这也说明，对于需要高频访问、重复复验和多主体持续询证的审计场景，若仍采用 Segull 这种每次查询都重新组织一次完整多方计算的方式，则总体等待时间将随请求数近似线性累积，难以满足实际在线可用性要求；相比之下，AegisPath 更接近可部署的在线审计接口形态。

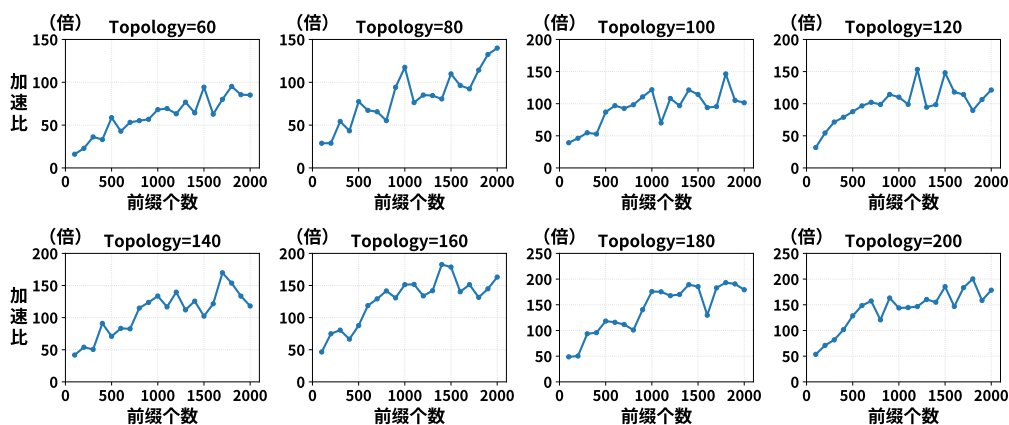


图 6.9 AegisPath 和 Segull 时间开销对比图

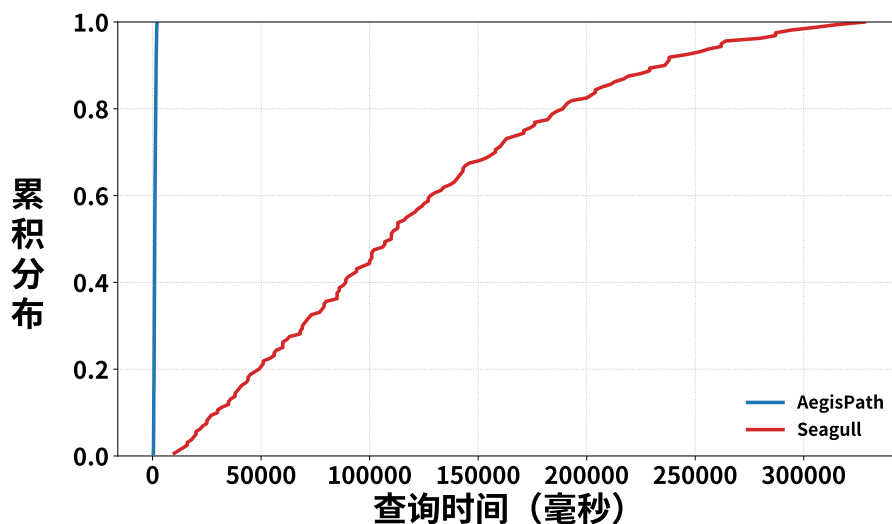


图 6.10 查询阶段时间开销累积分布图

6.5 实验三：增量更新机制的性能表现

本实验评估 AegisPath 在局部变化场景下的版本化与增量更新性能，重点回答以下四个问题：第一，生成期在配置变更场景下，增量更新相对于全量重算能够获

得多大加速；第二，生成期在**覆盖对象增加**场景下，增量更新相对于全量重算能够获得多大加速；第三，查询期在**Merkle 树维护**过程中，局部更新相对于整棵树重建能够带来多大时间收益；第四，当查询期电路规模变化时，增量机制的性能收益会呈现怎样的变化。

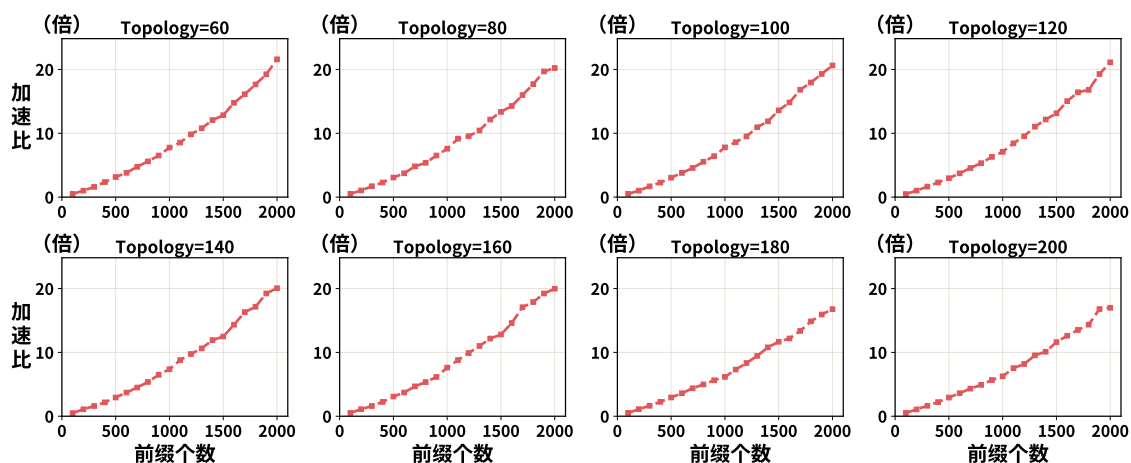


图 6.11 配置变更场景下生成期增量更新与全量更新的时间开销对比

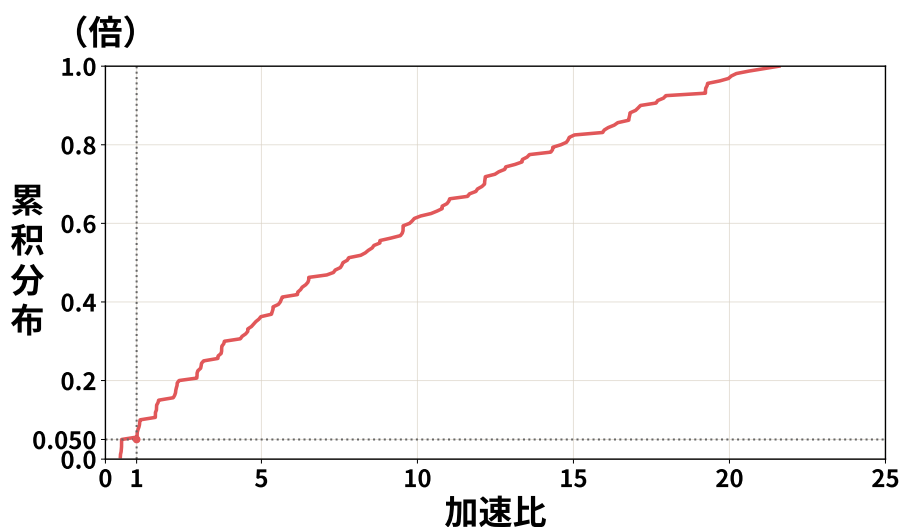


图 6.12 配置变更场景下生成期增量更新与全量更新时间加速比累积分布图

图 6.12 给出了配置变更场景下增量更新加速比的累积分布情况。可以看到，约 5% 的测试用例几乎没有优化效果，这是因为在小规模拓扑或核心节点变更时，局部扰动波及了全量范围。然而，绝大多数情况下增量更新表现优异。结合图 6.11 的不同拓扑规模子图可以发现，加速比与数据规模呈现显著的正相关性：当通告前缀数量较小时，由于基准方案本身开销较低，增量的优势尚不明显；但随着前缀数量逐步增加到 2000，各曲线的加速比普遍提升至 15 倍以上，部分场景最高可达

20 倍，且不同拓扑规模下的表现趋于一致。该结果说明，当变化来源于局部配置调整时，生成期仅对受影响对象执行重新推演，能够有效避免对全量对象重复执行 SMPC 计算，且这种只重算局部的优势会随着对象规模的增长而逐步放大。

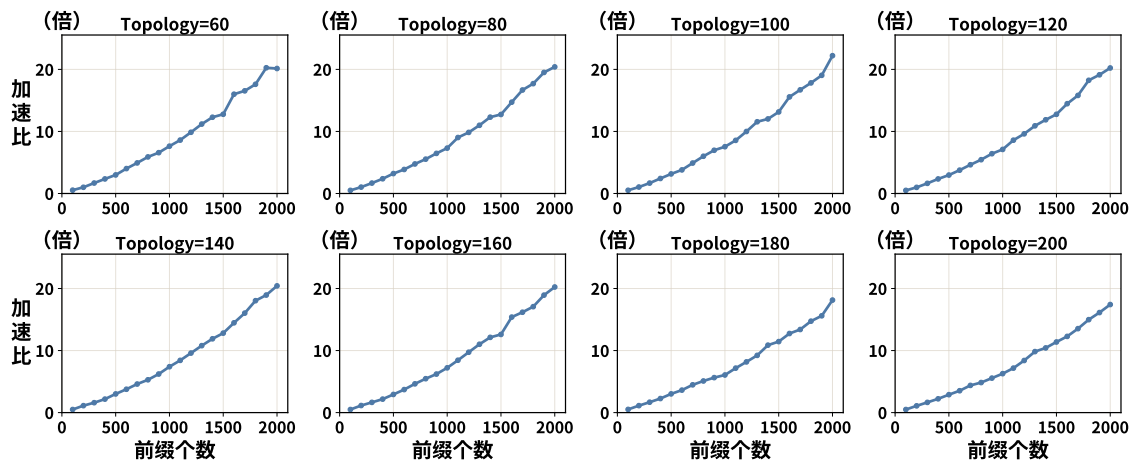


图 6.13 覆盖对象增加场景下生成期增量更新与全量更新的时间开销对比

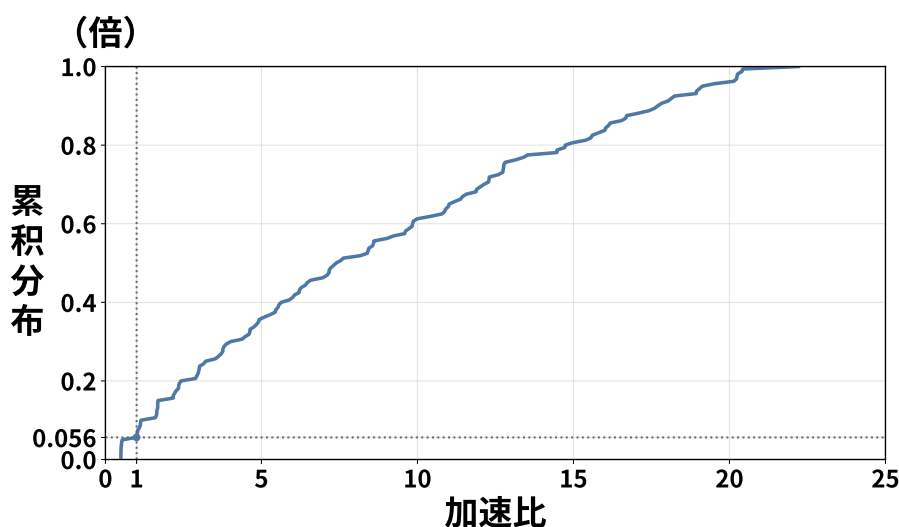


图 6.14 覆盖对象增加场景下生成期增量更新与全量更新时间加速比累积分布图

图 6.14 展示了覆盖对象增加场景下增量更新加速比的累积分布。可以看到，约 5.6% 的测试案例加速比接近 1，这主要源于新增前缀在小规模拓扑中诱发了全局性的重新推演。结合图 6.13 的不同拓扑规模趋势图可以进一步发现，加速比与网络中前缀的既有规模呈现显著的正相关性：当前缀数量较少时，全量重算的基准开销较低，增量更新的优势尚不突出；而随着前缀规模逐步增加至 2000，各拓扑场景下的加速比均普遍提升至 15 倍以上，部分峰值可达 20 倍。该结果表明，无论变化来源于本地配置调整还是覆盖对象范围扩展，只要受影响对象相对于全体覆

盖对象保持局部性，生成期增量维护机制都能持续规避冗余的 SMPC 计算，且在大规模场景下表现出更优的性能增益与可扩展性。

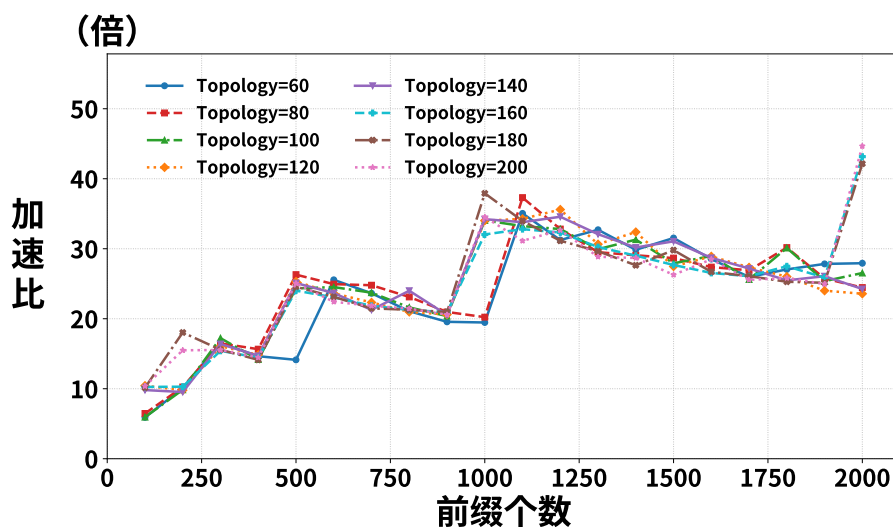


图 6.15 增量更新 Merkle 树和重建 Merkle 树的时间开销对比

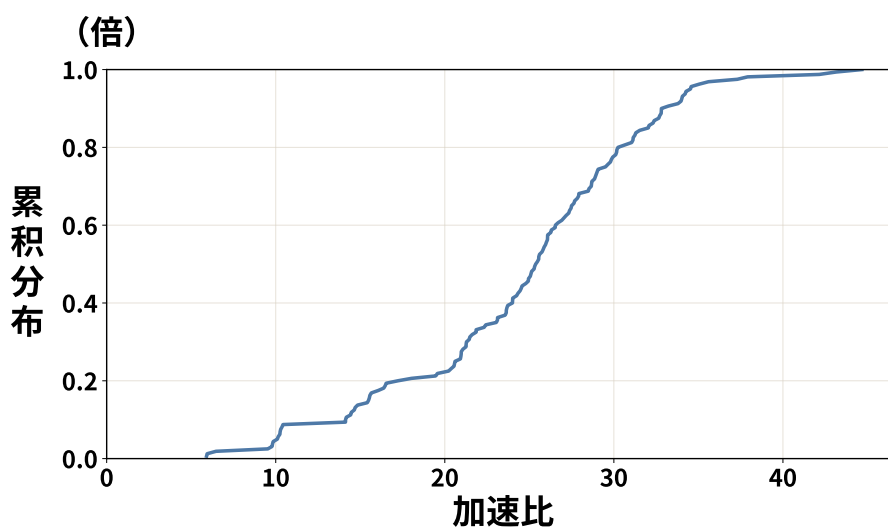


图 6.16 增量更新 Merkle 树相对于重建 Merkle 树的加速比累积分布图

图 6.15和图 6.16展示了查询期在 Merkle 树维护上的时间加速比，即仅更新受影响叶子及其认证分支，相对于重建整棵 Merkle 树所获得的性能收益。从图中可以看出，不同拓扑规模下的曲线整体变化趋势较为一致，说明增量维护带来的加速效果具有较好的稳定性。随着前缀数量增加，加速比总体呈现阶段性上升后趋于稳定的特征：在较小前缀规模下，加速比通常处于约 6–18 倍区间；当前缀数量增长到约 500 时，多数拓扑已提升至约 24–26 倍；在 1000–1200 个前缀附近，又进

一步跃升至约 32–37 倍。此后，绝大多数拓扑的加速比虽有一定波动，但总体保持在约 26–33 倍的较高区间内；当达到 2000 个前缀时，部分拓扑的加速比进一步提高到约 42–45 倍。累积分布结果也表明，查询期增量维护的时间收益主要集中在较高加速区间，进一步说明局部更新受影响叶子及其认证分支，能够在版本演化过程中持续降低查询期的维护成本，而无需为少量变化承担整棵 Merkle 树重建的开销。

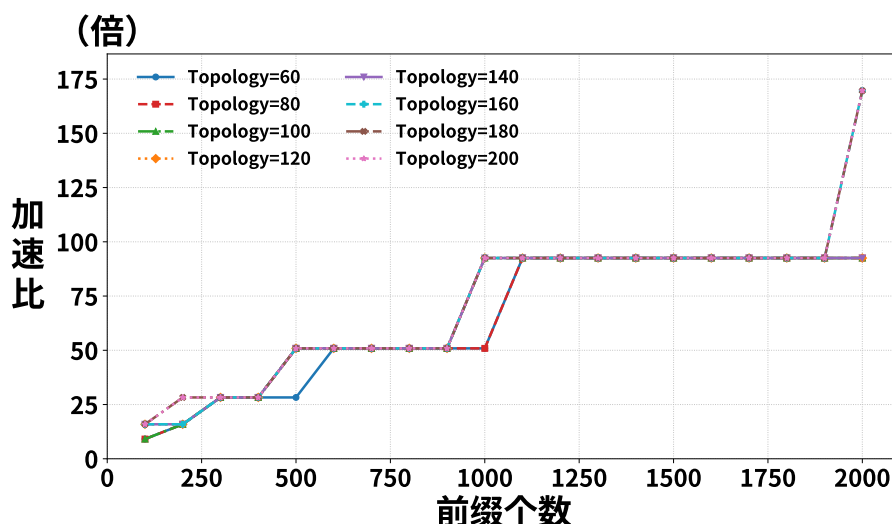


图 6.17 增量更新 Merkle 树和重建 Merkle 树的电路约束数量对比

图 6.18 首先给出了查询期增量机制在电路约束数量加速比上的整体分布情况。从累积分布结果可以看出，样本主要集中在若干离散的高加速区间，而不是连续平滑分布，这表明该指标的收益并非随前缀数量线性变化，而是更多受到电路结构层级变化的影响。结合图 6.17 可以进一步看到，不同拓扑下的曲线几乎完全重合，说明在约束数量这一指标上，增量维护的收益主要由 Merkle 树相关电路的结构特征决定，而与具体拓扑规模关系较小。随着前缀数量增加，加速比呈现出明显的阶梯式提升：在较小前缀规模下，各拓扑的加速比大致位于约 10–17 倍区间；随后很快上升并稳定在约 28 倍左右；当前缀数量达到约 500 时，进一步提升到约 51 倍；在 1000–1100 个前缀附近再次跃升至约 92–93 倍，并在之后的大部分区间内保持稳定；当达到最高前缀规模 2000 时，加速比又进一步提高到约 168–170 倍。该结果表明，查询期采用仅更新受影响叶子及其认证分支的方式，不仅能够降低维护时间开销，也能够显著压缩增量更新所需电路的约束规模，从而避免在少量局部变化下重复承担整棵 Merkle 树重建对应的电路代价。

综合以上结果可以看出，实验三验证了第五章提出的版本化与增量更新机制在生成期与查询期两侧都具有实际价值。生成期中，增量更新通过仅重算受影响对

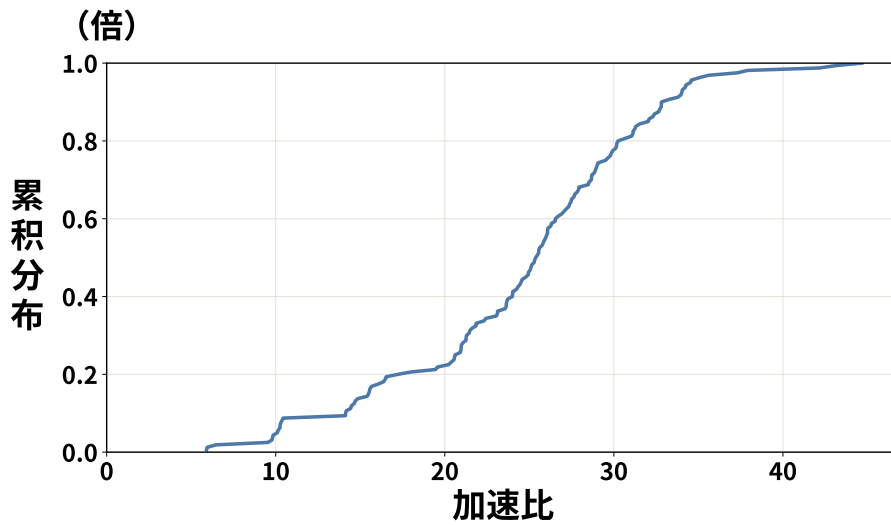


图 6.18 增量更新 Merkle 树和重建 Merkle 树的电路约束数量对比累积分布图
象显著降低了 SMPC 推演的重复开销；在版本维护过程中，增量更新通过仅更新受影响叶子及其到根分支避免了整棵 Merkle 树的全量重建；而当查询电路规模进一步增大时，这种局部维护机制的收益仍能保持，甚至更加明显。因此，AegisPath 的版本化与增量更新机制不仅在语义上成立，而且在局部变化场景下能够稳定取得显著的性能提升。

6.6 本章小结

本章围绕 AegisPath 的原型实现与性能表现，对生成期、查询期以及版本化增量维护三方面进行了实验评估。

实验一主要考察了生成期完整隐私协作计算的总体开销与可扩展性。结果表明，完整 SMPC 数据平面语义生成能够在 60-200 个 AS、100-2000 个前缀的范围内稳定输出覆盖对象上的见证结果，说明本文第三章提出的共享域语义推演过程在中等规模多管理域网络上具有可执行性。同时，生成期总耗时与通信量均随拓扑规模和前缀数量显著增长，说明完整生成过程更适合作为离线阶段的一次性高开销计算。相较之下，对象级 MiMC 承诺与版本根构造仅占较小比例，表明后续的版本化承诺过程并不是生成期的主导瓶颈。

实验二主要考察了查询期零知识审计的性能开销，并与 Segull 的查询方案进行了对比。结果表明，若每次查询都重新在共享域中执行一次完整审计流程，则单次查询时间可达到几十秒到数百秒，通信量也会显著膨胀；而 AegisPath 在查询期采用零知识证明机制后，证明生成时间稳定处于毫秒级，验证时间进一步降低到更低的毫秒级范围。该结果说明，将重计算前移到离线生成阶段、并在运行期

仅围绕版本根进行证明与验证，能够显著降低重复查询的边际成本，从而更适合作为可部署的在线审计接口。

实验三则进一步评估了版本化与增量更新机制在局部变化场景下的性能收益。结果表明，无论变化来源于局部配置调整还是覆盖对象增加，生成期增量更新相对于全量重算都能够取得稳定加速；在查询期的证据结构维护中，仅更新受影响叶子及其到根分支，也明显优于整棵 Merkle 树重建。当查询电路规模进一步增大时，这种局部维护机制的收益仍然保持，甚至更加明显。由此可以看出，第五章提出的版本化与增量更新机制不仅在语义上成立，而且在性能上具备实际价值。

综合上述实验，本章验证了本文整体技术路线的合理性：生成期利用 SMPC 完成隐私保护下的数据平面语义见证生成，查询期利用零知识证明提供低开销、可独立复验的审计接口，而版本化与增量更新机制则进一步支持了长期运行环境中的持续维护与反复复核。这些结果共同说明，AegisPath 能够在保持隐私保护目标的同时，为多管理域数据平面审计提供具备实际可行性的系统实现路径。

第7章 总结与展望

本章对全文工作进行总结与回顾。首先，围绕本文提出的系统目标与技术路线，对多管理域数据平面语义生成、版本化证据组织、查询期零知识审计以及增量维护机制等核心内容进行归纳；随后，结合本文当前实现与分析结果，总结其主要技术特点与应用价值；最后，对后续值得进一步研究的方向进行展望。

7.1 总结

本文围绕如何在不暴露多管理域内部数据面配置与路径细节的前提下，为多管理域数据平面验证结果提供可独立复验的审计能力这一问题展开研究，提出了一个结合隐私协同计算与零知识证明的版本化数据平面审计框架 AegisPath。全文的主要工作可概括为以下三个方面。

1. **多管理域隐私约束下的数据平面语义见证生成。**针对真实数据面语义难以集中获取的问题，本文研究了在隐私计算框架下如何围绕覆盖对象生成可用于审计的数据平面语义见证。具体而言，各自治系统以秘密共享方式提交本地数据面配置与策略信息，计算方在有限域上对转发表项、边界过滤规则与策略约束进行统一编码，并按照覆盖对象逐一执行数据面语义推演。与直接依赖控制面可观测路径的做法不同，本文生成的是综合前缀匹配、最长前缀匹配、边界过滤、黑洞、访问控制以及策略路由等因素后的有效数据平面语义。对于每一个覆盖对象，系统输出其在当前版本下的实际转发结果，包括逐跳转发路径、路径终止状态以及与该对象关联的结构化语义字段，并将这些信息封装为后续承诺、证明与查询所使用的对象级语义见证，从而为后续审计阶段提供可直接绑定到特定对象和特定版本的数据面证据。
2. **面向事后复核的版本化审计接口构建。**针对重复核验成本较高、外部主体难以独立复验的问题，本文研究了如何将生成阶段得到的语义见证组织为面向查询阶段的版本化证据结构。具体而言，系统首先对对象级见证进行密码学绑定，并通过 Merkle 聚合形成与特定版本状态对应的版本根；在查询阶段，证明方围绕相应版本输出零知识证明，使外部主体在不接触真实转发路径与域内策略细节的前提下，能够独立核验某一数据平面不变量结论是否确实来源于该版本下的已生成见证。基于这一设计，原本需要多方在线重算的重复

核验过程被转化为面向外部主体的低交互验证过程，从而为事后复核、责任界定与跨主体独立验证提供了可实现的技术接口。

3. **面向持续审计的版本化维护与增量更新。**针对多管理域环境中配置与策略频繁变化、审计证据需要持续维护的问题，本文进一步研究了运行期变更下的局部更新方法。该部分包括两个方面：一是在生成阶段建立增量输入接口与驱动方式，并设计面向受影响覆盖对象的增量推演算法，使配置变化后仅需对相关对象重新生成语义见证；二是在证据结构层对受影响叶子的承诺进行更新，沿对应 Merkle 分支局部重算并发布新版本根，使后续查询验证能够显式绑定到新的版本状态。通过上述设计，系统能够在避免全量重算与全量重建的前提下支持运行期持续核验，并使不同时间点的审计结论能够被清晰地区分、追溯与复验。

综上，本文从隐私计算、可验证证明与版本化维护三个层面，系统性地构建了一套面向多管理域数据平面审计的技术框架。相关设计表明，在不泄露多管理域内部配置与路径细节的前提下，多管理域数据平面语义的生成、承诺、证明与维护可以被统一组织到一个可实现、可扩展且可复验的体系之中。这为面向长期运行环境的多管理域网络审计与责任追溯提供了新的技术路径。

7.2 未来展望

尽管本文已经围绕多管理域数据平面语义生成与版本化审计给出了较为完整的设计与实现，但仍有若干值得进一步研究和完善的方向。

更丰富的数据平面语义建模。本文当前的数据平面语义模型主要覆盖前缀级转发、最长前缀匹配、边界过滤以及有限形式的策略处理，能够支撑本文关注的可达性、路径经过性和无环性等审计场景。但是针对真实的生产网络，数据平面还可能包含多路径负载均衡、报文头字段改写、端口级访问控制、细粒度流级处理以及更加灵活的策略组合等复杂行为。这些行为会进一步扩展语义对象的字段结构、状态转移逻辑和约束系统规模。后续工作可在保持语义结果可承诺、可证明和可复验的前提下，进一步扩展数据平面语义表示能力，从而提升框架对真实生产网络复杂行为的适配范围。

增量识别与局部更新策略精化。本文已经提出了基于版本化承诺的数据平面语义维护机制，并以覆盖对象为基本单元支持增量更新。当前增量识别主要依据配置变化、控制面输入变化以及对象级路径关联关系确定受影响范围。后续工作

可进一步引入网络配置依赖关系分析，将转发表项、边界策略、前缀匹配关系和路径传播关系之间的依赖显式建模，从而更精确地判断某一配置变化实际影响的覆盖对象集合。通过缩小重算集合并减少不必要的见证重新生成，可进一步降低长期运行场景下的增量维护开销。

多管理域审计中的全盲验证机制研究。本文目前在设计上通过密码学机制硬性约束了审计结论的真实性，而将路径隐私保护部分交由合约约束与事后溯源机制。尽管这种分层防御在当前互联网算力环境下具有极高的工程实用性，但如何进一步消除证明与托管服务对路径见证的观测权，实现全生命周期的盲验证（Blind Verification），仍是未来的重要研究方向。

后续工作将探索引入全同态加密或不变量无关的路径隐藏结构。其核心挑战在于：如何在不显著增加验证延迟的前提下，使 Prover 能够在完全盲态（即无法获知路径槽位、前缀映射及中间跳序列）的情况下，计算并生成满足特定网络不变量要求的零知识证明。

此外，针对盲验证带来的计算开销挑战，未来可研究基于硬件可信执行环境（TEE）与密码学算法的混合增强方案。通过在飞地（Enclave）内部执行不变量核验逻辑，并利用远程度量（Remote Attestation）产生可信证明，实现在保护路径全盲隐私的同时，将审计性能提升至工业级实时响应水平。

查询期证明系统与电路优化。本文在查询期采用 ZK-SNARK 提供高效的独立验证能力，但可信设置、约束规模与证明生成开销仍然是系统工程化落地中需要持续优化的问题。未来可进一步探索更适合该场景的证明系统选型，例如通用设置或透明设置体系下的实现方案，并结合电路裁剪、承诺结构优化和哈希原语替换等手段，进一步降低证明生成与验证的实际成本。

总体而言，本文的工作为多管理域数据平面的可验证审计提供了一个初步但完整的研究框架。未来围绕语义扩展、证明系统优化、全盲验证机制研究、增量维护精化以及系统化评估等方向的持续推进，将有助于进一步提升该框架的理论完备性与工程实用性。

参考文献

- [1] 中国互联网络信息中心 (CNNIC). 第 56 次《中国互联网络发展状况统计报告》[EB/OL]. 2025. <https://www.cnnic.cn/NMediaFile/2025/0730/MAIN1753846666507QEK67ZS9DH.pdf>.
- [2] Uptime Institute. Annual outage analysis 2025 (executive summary)[EB/OL]. 2025. https://uptimeinstitute.com/uptime_assets/d7c049ef5b02a6e0a15540a3e5cb8fbf742c7fa54a1af6caea-aab32b7c15d443-GA-2025-05-annual-outage-analysis.pdf.
- [3] Zulan A, Miron O, Shapira T, et al. IRR-based AS type of relationship inference[J]. arXiv preprint arXiv:2504.10299, 2025.
- [4] Gao L. On inferring autonomous system relationships in the internet[J]. IEEE/ACM Transactions on networking, 2001, 9(6): 733-745.
- [5] Fang M, Zhao Y, Qin Q, et al. Iveri: A scalable privacy-preserving interdomain configuration verification tool via secure multi-party computation[C]//IWQoS'25. IEEE, 2025: 1-10.
- [6] Daneshamooz J, Yu M, Maddury S. Seagull: Privacy preserving network verification system [J]. arXiv preprint arXiv:2402.08956, 2024.
- [7] Xu H, Qin Q, Fang X, et al. Toward Privacy-Preserving Interdomain Configuration Verification via Multi-Party Computation[C]//APNet'23. ACM, 2023: 28-33.
- [8] Wang Y, Men Q, Xiao Y, et al. Confmask: Enabling privacy-preserving configuration sharing via anonymization[C]//SIGCOMM'24. ACM, 2024: 465-483.
- [9] RIPE NCC. RIPE NCC Routing Information Service (RIS)[EB/OL]. 2026. <https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris/>.
- [10] Cogent Communications. Cogent Communications Looking Glass[EB/OL]. 2026. <https://www.cogentco.com/en/looking-glass>.
- [11] University of Oregon RouteViews Project. RouteViews Project[EB/OL]. 2026. <http://www.routeviews.org/>.
- [12] Zeng H, Zhang S, Ye F, et al. Libra: Divide and conquer to verify forwarding tables in huge networks[C]//NSDI'14. USENIX, 2014: 87-99.
- [13] Internet Society Pulse. North korea downed by faulty roa[EB/OL]. 2025. <https://pulse.internetsociety.org/blog/north-korea-downed-by-faulty-roa>.
- [14] Roth A. Russia blocks millions of ip addresses in battle against telegram app[EB/OL]. 2018. <https://www.theguardian.com/world/2018/apr/17/russia-blocks-millions-of-ip-addresses-in-battle-against-telegram-app>.
- [15] Reuters. Russia lifts ban on telegram messaging app after failing to block it[EB/OL]. 2020. <https://www.reuters.com/article/technology/russia-lifts-ban-on-telegram-messaging-app-after-failing-to-block-it-idUSKBN23P2DY/>.
- [16] Okui P. The routeviews project: Update[EB/OL]. 2025. <https://assets.routeviews.org/presos/RV-AfPIF2025.pdf>.

- [17] RouteViews. Routeviews api documentation[EB/OL]. 2025. <https://api.routeviews.org/docs/>.
- [18] 方星, 胡波, 马超, 等. 网络验证研究综述[J]. 软件学报, 2023, 34(01): 351-380.
- [19] Mai H, Khurshid A, Agarwal R, et al. Debugging the Data Plane with Anteater[C]//SIGCOMM'11. ACM, 2011: 290-301.
- [20] Kazemian P, Varghese G, McKeown N. Header Space Analysis: Static Checking for Networks [C]//NSDI'12. USENIX, 2012: 113-126.
- [21] Kazemian P, Chang M, Zeng H, et al. Real Time Network Policy Checking Using Header Space Analysis[C]//NSDI'13. USENIX, 2013: 99-111.
- [22] Lopes N P, Bjørner N, Godefroid P, et al. Checking beliefs in dynamic networks[C]//NSDI'15. USENIX, 2015: 499-512.
- [23] Khurshid A, Zou X, Zhou W, et al. Veriflow: Verifying network-wide invariants in real time [C]//NSDI'13. USENIX, 2013: 15-27.
- [24] Horn A, Kheradmand A, Prasad M. Delta-net: Real-time network verification using atoms[C]//NSDI'17. USENIX, 2017: 735-749.
- [25] Zhang P, Liu X, Yang H, et al. Apkeep: Realtime verification for real networks[C]//NSDI'20. USENIX, 2020: 241-255.
- [26] Beckett R, Gupta A. Kutra: Realtime verification for multilayer networks[C]//NSDI'22. USENIX, 2022: 617-634.
- [27] Guo D, Chen S, Gao K, et al. Flash: Fast, consistent data plane verification for large-scale network settings[C]//SIGCOMM'22. ACM, 2022: 314-335.
- [28] Xiang Q, Wen R, Huang C, et al. Network can check itself: scaling data plane checking via distributed, on-device verification[C]//HotNets'22. ACM, 2022: 85-92.
- [29] Xiang Q, Huang C, Wen R, et al. Beyond a centralized verifier: Scaling data plane checking via distributed, on-device verification[C]//SIGCOMM'23. ACM, 2023: 152-166.
- [30] Yang H, Lam S S. Collaborative verification of forward and reverse reachability in the internet data plane[C]//ICNP'14. IEEE, 2014: 320-331.
- [31] Laurie B, Messeri E, Stradling R. Certificate Transparency Version 2.0[Z]. 2021.
- [32] Laurie B, Langley A, Kasper E. Certificate Transparency[Z]. 2013.
- [33] Melara M S, Blankstein A, Bonneau J, et al. CONIKS: Bringing key transparency to end users [C]//USENIX Security'15. USENIX, 2015: 383-398.
- [34] Wang X, Liu Z, Li Q, et al. Secure inter-domain routing and forwarding via verifiable forwarding commitments[J]. arXiv preprint arXiv:2309.13271, 2023.
- [35] Anahory Y E, Kong J, Scaglione N, et al. Suppressing BGP zombies with route status transparency[C]//NSDI'25. USENIX, 2025: 1349-1366.
- [36] Orsini C, King A, Giordano D, et al. Bgpstream: a software framework for live and historical bgp data analysis[C]//IMC'16. ACM, 2016: 429-444.
- [37] Katz-Bassett E, Madhyastha H V, John J P, et al. Studying black holes in the internet with hubble[C]//NSDI'08. USENIX, 2008: 247-262.

- [38] Quan L, Heidemann J, Pradkin Y. Trinocular: Understanding internet reliability through adaptive probing[C]//SIGCOMM'13. ACM, 2013: 255-266.
- [39] Rekhter Y, Li T, Hares S. A Border Gateway Protocol 4 (BGP-4)[Z]. 2006.
- [40] Yao A C. Protocols for secure computations[C]//FOCS'82. IEEE, 1982: 160-164.
- [41] Goldreich O, Micali S, Wigderson A. How to play any mental game or a completeness theorem for protocols with honest majority[C]//STOC'87. ACM, 1987: 218-229.
- [42] 蒋凯元. 多方安全计算研究综述[J]. 信息安全研究, 2021, 7(12): 1161-1165.
- [43] Bellare M, Hoang V T, Rogaway P. Foundations of garbled circuits[C]//CCS'12. ACM, 2012: 784-796.
- [44] 邵航, 高思琪, 钟离, 等. 同态加密在隐私计算中的应用综述[J]. 信息通信技术与政策, 2022(08): 75-88.
- [45] 徐科鑫, 王丽萍. 多方全同态加密研究进展[J]. 密码学报 (中英文), 2024, 11(04): 719-739.
- [46] Shamir A. How to share a secret[J]. Communications of the ACM, 1979, 22(11): 612-613.
- [47] Keller M. MP-SPDZ: A Versatile Framework for Multi-Party Computation[C]//CCS'20. 2020: 1575-1590.
- [48] Nishide T, Ohta K. Multiparty computation for interval, equality, and comparison without bit-decomposition protocol[C]//PKC'07. Springer, 2007: 343-360.
- [49] Toft T. Constant-rounds, almost-linear bit-decomposition of secret shared values[C]//CT-RSA'09. Springer, 2009: 357-371.
- [50] Goldreich O, Ostrovsky R. Software protection and simulation on oblivious rams[J]. J. ACM, 1996, 43(3): 431-473.
- [51] Evans D, Kolesnikov V, Rosulek M. A pragmatic introduction to secure multi-party computation[J]. Foundations and Trends in Privacy and Security, 2018, 2(2-3): 70-246.
- [52] Goldwasser S, Micali S, Rackoff C. The knowledge complexity of interactive proof systems [J]. SIAM Journal on Computing, 1989, 18(1): 186-208.
- [53] Blum M, Feldman P, Micali S. Non-interactive zero-knowledge and its applications[C]//STOC'88. ACM, 1988: 103-112.
- [54] Fiat A, Shamir A. How to prove yourself: Practical solutions to identification and signature problems[C]//Advances in Cryptology – CRYPTO '86. Springer, 1987: 186-194.
- [55] Parno B, Howell J, Gentry C, et al. Pinocchio: Nearly practical verifiable computation[C]//S&P'13. IEEE, 2013: 238-252.
- [56] Ben-Sasson E, Chiesa A, Genkin D, et al. Snarks for c: Verifying program executions succinctly and in zero knowledge[C]//CRYPTO'13. Springer, 2013: 90-108.
- [57] Groth J. On the size of pairing-based non-interactive arguments[C]//EUROCRYPT'16. Springer, 2016: 305-326.
- [58] 张正铨, 胡森, 莫晓康. 零知识证明研究综述[J]. 数字通信世界, 2023(06): 79-81.
- [59] Bowe S, Gabizon A, Miers I. Scalable multi-party computation for zk-snark parameters in the random beacon model[Z]. 2017.

- [60] Albrecht M R, Grassi L, Rechberger C, et al. Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity[C]//ASIACRYPT'16. Springer, 2016: 191-219.
- [61] Merkle R C. A digital signature based on a conventional encryption function[C]//Advances in Cryptology – CRYPTO '87. Springer, 1988: 369-378.
- [62] Lepinski M, Kent S. An infrastructure to support secure internet routing[Z]. 2012.
- [63] Huston G, Loomans R, Michaelson G. A profile for resource certificate repository structure [Z]. 2012.
- [64] Allison P D. Discrete-time methods for the analysis of event histories[J]. Sociological Methodology, 1982, 13: 61-98.
- [65] Google Cloud. Cloud Dedicated and Partner Interconnect Service Level Agreement (SLA) [EB/OL]. 2019. <https://cloud.google.com/network-connectivity/docs/interconnect/sla-20190307>.
- [66] Google Cloud. Network Connectivity Pricing[EB/OL]. 2026. <https://cloud.google.com/network-connectivity/pricing>.
- [67] Dziembowski S, Faust S, Lizurej T, et al. Secret sharing with snitching[C]//CCS'24. ACM, 2024: 840-853.
- [68] Faust S, Hazay C, Kretzler D, et al. Financially backed covert security[C]//Public-Key Cryptography (PKC 2022). Springer, 2022: 99-129.
- [69] Cho S, Fontugne R, Cho K, et al. BGP hijacking classification[C]//TMA'19. IEEE, 2019: 25-32.
- [70] Ballani H, Francis P, Zhang X. A study of prefix hijacking and interception in the internet[C]//SIGCOMM'07. ACM, 2007: 265-276.
- [71] Cyber Security Agency of Singapore. Ransom Distributed Denial-Of-Service Attacks[EB/OL]. 2020. <https://www.csa.gov.sg/resources/publications/ransom-distributed-denial-of-service-attacks/>.
- [72] Cloudflare. DDoS Threat Report for 2024 Q2[EB/OL]. 2024. <https://blog.cloudflare.com/ddos-threat-report-for-2024-q2/>.
- [73] Tao Y, Zhang C, An C, et al. Ares: Comprehensive path hijacking detection via routing tree [C]//USENIX Security'25. USENIX, 2025: 803-821.
- [74] 国家互联网信息办公室. 中华人民共和国网络安全法[EB/OL]. 2025. https://www.cac.gov.cn/2025-12/29/c_1768735112911946.htm.
- [75] Legal Information Institute. U.S. Code Title 18[EB/OL]. 2026. <https://www.law.cornell.edu/uscode/text/18>.
- [76] European Union. Regulation (EU) 2016/679[EB/OL]. 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [77] Help Net Security. MyEtherWallet Users Robbed After Successful DNS Hijacking Attack [EB/OL]. 2018. <https://www.helpnetsecurity.com/2018/04/25/myetherwallet-dns-hijacking/>.

-
- [78] Madory D. What Can Be Learned from Recent BGP Hijacks Targeting Cryptocurrency Services?[EB/OL]. 2022. <https://www.kentik.com/blog/bgp-hijacks-targeting-cryptocurrency-services/>.
- [79] Holterbach T, Alfroy T, Phokeer A, et al. A system to detect forged-origin bgp hijacks[C]//NSDI'24. USENIX, 2024: 1751-1770.
- [80] McMillan R. One of the Largest Bitcoin Mining Pools Has Been Hit With a Novel Attack [EB/OL]. 2014. <https://www.wired.com/2014/11/bgp-hijacking-bitcoin/>.
- [81] Chen Y, Yin Q, Li Q, et al. Learning with semantics: Towards a semantics-aware routing anomaly detection system[C]//USENIX Security'24. USENIX, 2024: 5143-5160.
- [82] Gilad Y, Cohen A, Herzberg A, et al. Are we there yet? on rpki's deployment and security.[C]//NDSS. ISOC, 2017.
- [83] Hlavacek T, Jeitner P, Mirdita D, et al. Stalloris:RPKI downgrade attack[C]//USENIX Security'22. USENIX, 2022: 4455-4471.
- [84] Mirdita D, Schulmann H, Vogel N, et al. The cure to vulnerabilities in rpki validation[C]//NDSS'24. ISOC, 2024.
- [85] Elmokashfi A, Kvalbein A, Dovrolis C. Bgp churn evolution: A perspective from the core[J]. IEEE/ACM Transactions on Networking, 2011, 20(2): 571-584.
- [86] Reitblatt M, Foster N, Rexford J, et al. Abstractions for network update[C]//SIGCOMM'12. ACM, 2012: 323-334.
- [87] Zhang P, Gember-Jacobson A, Zuo Y, et al. Differential Network Analysis[C]//NSDI'22. USENIX, 2022: 601-615.
- [88] Zhao C, Guo Y, Wang J, et al. EPVerifier: Accelerating update storms verification with Edge-Predicate[C]//NSDI'24. USENIX, 2024: 979-992.
- [89] Mohassel P, Rindal P. ABY3: A mixed protocol framework for machine learning[C]//CCS'18. ACM, 2018: 35-52.
- [90] Knott B, Venkataraman S, Hannun A, et al. Crypten: Secure multi-party computation meets machine learning[C]//NeurIPS'21. NeurIPS Foundation, 2021: 4961-4973.
- [91] Ma J, Zheng Y, Feng J, et al. SecretFlow-SPU: A Performant and User-Friendly Framework for Privacy-Preserving Machine Learning[C]//ATC'23. USENIX, 2023: 17-33.
- [92] Schoenmakers B. MPyC—Python package for secure multiparty computation[EB/OL]. 2018. <https://berry.win.tue.nl/mpyc/TPMPC2018.pdf>.
- [93] Bellés-Muñoz M, Isabel M, Muñoz-Tapia J L, et al. Circom: A circuit description language for building zero-knowledge applications[J]. IEEE Transactions on Dependable and Secure Computing, 2022, 20(6): 4733-4751.
- [94] arkworks contributors. arkworks zksnark ecosystem[EB/OL]. 2022. <https://arkworks.rs>.
- [95] zkcrypto. bellman: zk-snark library[EB/OL]. 2026. <https://github.com/zkcrypto/bellman>.
- [96] Zcash Foundation. The halo2 book[EB/OL]. 2022. <https://zcash.github.io/halo2/index.html>.
- [97] Consensys. gnark: A highly optimized library for zero-knowledge proof systems[EB/OL].

2024. <https://github.com/Consensys/gnark>.
- [98] RouteViews. Routeviews collector data and archive access[EB/OL]. 2023. https://assets.routeviews.org/presos/RV_EPF2023.pdf.
- [99] CAIDA. Routeviews prefix to as mappings dataset (pfx2as) for ipv4 and ipv6[EB/OL]. 2008. <https://www.caida.org/catalog/datasets/routeviews-prefix2as/>.
- [100] Liu A X, Torng E, Meiners C R. Compressing network access control lists[J]. IEEE Transactions on Parallel and Distributed Systems, 2011, 22(12): 1969-1977.
- [101] Tian B, Zhang X, Zhai E, et al. Safely and automatically updating in-network acl configurations with intent language[C]//SIGCOMM '19. ACM, 2019: 214-226.
- [102] Jaber Daneshamooz. Seagull-Network-verification[EB/OL]. 2024. <https://github.com/jaber-the-great/Seagull-Network-verification>.

致 谢

回望三年的研究生生涯，心中感慨万千。首先，我要感谢那个在实验室无数个深夜里依然坚持、在每一个实验数据前反复推敲、在无数次论文修改中从未放弃的自己。这段奋斗的青春岁月，将成为我未来步入职场、面对挑战时最宝贵的财富。

特别要向我的导师向乔老师致以最崇高的敬意和最诚挚的感谢。向老师在学术研究上对我始终保持着极高标准의严谨要求，无论是实验设计的严密性，还是论文措辞的精确度，向老师都言传身教，给予了精辟而深刻的指导。正是向老师这种精益求精、一丝不苟的科学精神，磨炼了我的学术意志，也让这篇论文在反复的磨砺中日益完善，使我受益终生。

在此，还要特别感谢实验室的宋庆禹老师和苏强老师。在论文的撰写与修改过程中，两位老师投入了大量的时间与精力，细致入微地审阅了每一个章节，并针对论文的逻辑结构、实验对比以及核心贡献点提出了极具建设性的修改意见。正是因为有了两位老师在细节处的悉心指导，才使得本文的措辞更加规范。

最后，感谢 SNG 实验室全体同学的陪伴与支持。在组会上的思想碰撞、在实验室里的互助探讨、以及生活中点点滴滴的欢声笑语，共同营造了这样一个充满活力、积极向上的科研氛围。这份并肩作战的情谊是我研究生生活中最温暖的一抹亮色。

感谢所有曾经关心、支持和帮助过我的人。

在学期间完成的相关学术成果

学术论文：

- [1] **Mingjun Fang**, Yuntao Zhao, Qiuyue Qin, Xing Fang, Huisan Xu, Lizhao You, Qiao Xiang, and Jiwu Shu. IVeri: A Scalable Privacy-Preserving Interdomain Configuration Verification Tool via Secure Multi-Party Computation[C] Proceedings of the IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS '25). pp. 1-10. doi: 10.1109/IWQoS65803.2025.11143374. (CCF-B, 共同第一作者)
- [2] Qiao Xiang, Yuling Lin, **Mingjun Fang**, Bang Huang, Siyong Huang, Ridi Wen, Franck Le, Linghe Kong and Jiwu Shu. Toward Reproducing Network Research Results Using Large Language Models[C] Proceedings of the 22nd ACM Workshop on Hot Topics in Networks(HotNets '23). pp. 56-62. doi: 10.1145/3626111.3628189. (CCF-C, 第三作者, 第一作者为导师)
- [3] **Mingjun Fang**, Shuhao Zheng, Zonglun Li, Letian Zhu, Qingyu Song, Lizhao You, Qiang Su, Lu Tang, Wanjian Feng, Fei Yuan, Qiao Xiang, Xue Liu and Jiwu Shu. AegisPath: Privacy-Preserving Interdomain Data-Plane Auditing with Versioned Verifiable Evidence[C]. (在投, 第一作者)

